

Diplomarbeit

Bestellservice „HungerSatt“

Entwicklung und Design eines Online-Bestellservices
für das Schulbuffet

5AHINF-2025/26-DA05

Johannes Ellmer

Christos Magos

Betreuer

Mag. Robert Gröbl

Dipl. Ing. (FH) Dietmar Winkler

Ausgeführt im Schuljahr 2025/26

Eidesstattliche Erklärung

Die unterfertigten Kandidaten / Kandidatinnen haben gemäß § 34 (3) SchUG in Verbindung mit § 22 (1) Zi. 3 lit. b der Verordnung über die abschließenden Prüfungen in den berufsbildenden mittleren und höheren Schulen, BGBl. II Nr. 70 vom 24.02.2000 (Prüfungsordnung BMHS), die Ausarbeitung einer Diplomarbeit mit der umseitig angeführten Aufgabenstellung gewählt.

Die Kandidaten / Kandidatinnen nehmen zur Kenntnis, dass die Diplomarbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können.

Die Abgabe der vollständigen Diplomarbeit hat bis spätestens

07.04.2026

beim zuständigen Betreuer zu erfolgen.

Die Kandidaten / Kandidatinnen nehmen weiters zur Kenntnis, dass gemäß § 9 (6) der Prüfungsordnung BMHS nur der Schulleiter bis spätestens Ende des vorletzten Semesters den Abbruch einer Diplomarbeit anordnen kann, wenn diese aus nicht beim Prüfungskandidaten (bei den Prüfungskandidaten) gelegenen Gründen nicht fertiggestellt werden kann.

Kandidaten / Kandidatinnen	Unterschrift
Johannes Ellmer	
Christos Magos	

Kurzfassung

Im Rahmen dieser Diplomarbeit wird ein digitales Bestell- und Verwaltungssystem für das Schulbuffet entwickelt und umgesetzt. Ausgangspunkt ist der bisher manuelle Bestellprozess, der durch lange Wartezeiten, handschriftliche Zettel und mangelnde Übersicht geprägt ist. Ziel ist es, diesen Ablauf zu digitalisieren und dadurch effizienter sowie übersichtlicher zu gestalten.

Dazu wird eine Webanwendung entwickelt, über die Benutzerinnen und Benutzer ihre Speisen im Voraus auswählen und bestellen können. Dadurch werden Wartezeiten während der Pausen reduziert und das Buffet erhält eine bessere Planungsgrundlage. Die benötigten Portionen sind frühzeitig bekannt, wodurch sowohl Überproduktion als auch Engpässe vermieden werden können.

Zusätzlich steht dem Buffetpersonal eine Verwaltungsoberfläche zur Verfügung. Diese ermöglicht das Erstellen von Menüs, das Verwalten von Bestellungen sowie die Organisation der Essensausgabe. Ein integriertes Guthabensystem sorgt zudem für eine einfache und bargeldlose Zahlungsabwicklung. Die technische Umsetzung basiert auf einer modernen Webarchitektur mit *NestJS*, einer *REST-API* und einer *relationalen* Datenbank. Das Frontend wird benutzerfreundlich gestaltet und ist auf verschiedenen Endgeräten nutzbar.

Das Ergebnis ist ein funktionsfähiges System, das den Buffetbetrieb digital unterstützt und organisatorische Abläufe im Schulalltag effizienter gestaltet.

Abstract

As part of this diploma thesis, a digital ordering and management system for the school cafeteria is developed and implemented. The starting point is the previously manual ordering process, which involves long waiting times, handwritten notes, and limited transparency. The goal is to digitize this process and make it more efficient and structured.

A web application is developed that allows users to select and pre-order meals in advance. This reduces waiting times during school breaks and provides the cafeteria with a better basis for planning. Required portions are known beforehand, helping to avoid both overproduction and shortages.

In addition, an administrative interface is provided for cafeteria staff. It allows menus to be created, orders to be managed, and the distribution process to be organized efficiently. A credit-based system is also integrated to enable simple and cashless payment. The system is implemented using a modern web architecture based on *NestJS*, a *REST API*, and a *relational* database. The frontend is designed to be user-friendly and accessible on different devices.

The result is a functional system that supports cafeteria operations digitally and improves efficiency in everyday school processes.

Gendererklärung

In dieser Diplomarbeit wird aus Gründen der besseren Lesbarkeit überwiegend die männliche Form verwendet. Dies geschieht ausdrücklich ohne die Absicht, andere Geschlechter oder Identitäten auszuschließen. Die gewählte Sprachform dient allein der Vereinfachung und folgt gängigen wissenschaftlichen Schreibkonventionen. Sämtliche Personenbezeichnungen sind daher geschlechtsneutral zu verstehen und beziehen alle Menschen gleichermaßen ein. Die Verwendung der männlichen Form impliziert keinerlei Wertung oder Diskriminierung.

Inhaltsverzeichnis

1	Einleitung	1
2	Planung	2
2.1	Aufgabenstellung	2
2.1.1	Projektfindung und Ausgangslage.....	2
2.1.2	Konzept und Rahmenbedingungen.....	2
2.1.3	Anforderungen und Ziele	3
2.2	Projektorganisation	6
2.2.1	Meilensteine und Zeitplan	6
2.2.2	Rollen- und Aufgabenverteilung	8
2.3	Stakeholder (<i>Ellmer</i>).....	10
2.3.1	Stakeholder-Analyse.....	10
2.3.2	Personas	12
2.4	Machbarkeitsstudie und Variantenbildung (<i>Ellmer</i>)	14
2.4.1	Hosting mittels <i>Proxmox</i> -Server.....	14
2.4.2	Cloud-Dienste	15
2.4.3	Gewählte Hosting-Lösung	16
2.4.4	Schlussfolgerung der Variantenbildung	16
3	Projektrecherche	17
3.1	Technologien (<i>Magos</i>).....	17
3.1.1	Angular	17
3.1.2	NestJS	18
3.2	Programmiersprache <i>Typescript</i> (<i>Magos</i>).....	20
3.3	Datenschutz und Verifizierung (<i>Ellmer</i>)	20
3.3.1	Datenschutz und DSGVO	20
3.3.2	Verifizierung und Authentifizierung.....	21
3.3.3	Zusammenfassung.....	22
3.4	PostgreSQL als relationale Datenbank (<i>Ellmer</i>)	22
3.4.1	Relationale Datenbanken im Allgemeinen.....	22
3.4.2	PostgreSQL als DBMS	25
3.4.3	PostgreSQL in containerisierten Umgebungen	25
3.4.4	Zusammenfassung.....	26

3.5	Grundlagen zur Microservice-Architektur	26
3.5.1	Grundlagen und wesentliche Merkmale	27
3.5.2	Kommunikation / Koordination von Microservices	28
3.5.3	Datenmanagement in Microservices	29
3.5.4	Frontend-Backend Kommunikation	29
3.5.5	Zusammenfassung.....	30
3.6	Containerisierung und Docker	30
3.6.1	Docker als Technologie der Containerisierung	31
3.6.2	Orchestrierung von containerisierten Microservices	33
3.6.3	Herausforderungen containerisierter Microservices.....	33
3.6.4	Zusammenfassung.....	33
3.7	Cloud Computing und Cloud-Plattformen (<i>Ellmer</i>).....	34
3.7.1	Grundlagen des Cloud Computing	34
3.7.2	AWS vs. Microsoft Azure	35
3.7.3	Cloud-Hosting-Alternativen.....	36
3.7.4	World4You als nationaler Hosting-Anbieter	36
3.7.5	Render als Cloud-Plattform	36
3.7.6	Netlify als Cloud Plattform	36
3.7.7	Cloud Anbieter im direkten Vergleich.....	37
3.7.8	Zusammenfassung.....	37
3.8	Einsatz sinnvoller Libraries und Technologien	37
3.8.1	PDF-Generierung mit jsPDF (<i>Ellmer</i>)	38
3.8.2	QR-Code-Erkennung mit html5-qrcode	38
3.8.3	Bedienung mit Angular CDK Drag-and-Drop (<i>Magos</i>)	38
3.8.4	Verifikation und Authentifizierung via E-Mail (<i>Ellmer</i>)	39
3.8.5	Zuverlässiger E-Mail-Versand (<i>Ellmer</i>)	40
3.8.6	Zusammenfassung.....	40
4	Umsetzung.....	41
4.1	Grundstruktur des Projekts	41
4.1.1	Systemüberblick	41
4.1.2	Architekturübersicht	43
4.1.3	Container-Ansicht.....	44

4.1.4	Interne Struktur des Backend.....	45
4.1.5	Sicherheit und Authentifizierung	45
4.1.6	Funktionale Ansicht – Use-Cases	46
4.1.7	Systemabläufe – Aktivitätsdiagramme	48
4.1.8	C4-Diagramme.....	50
4.2	Frontend-Entwicklung	53
4.2.1	Menüverwaltung (Magos).....	53
4.2.2	Menüplanung (Magos).....	55
4.2.3	Verwaltung von Gerichten (<i>Magos</i>).....	59
4.2.4	Menüauswahl (Menüansicht der Kunden) (<i>Ellmer</i>)	61
4.2.5	Warenkorb (<i>Ellmer</i>).....	63
4.2.6	Bestellübersicht – Kunden (<i>Ellmer</i>)	65
4.2.7	QR-Scan und Bestellabschluss - Inhaber	67
4.2.8	Auswertungen und Benutzerverwaltung (Ellmer)	69
4.2.9	Authentifizierung und E-Mail-Bestätigung.....	71
4.2.10	Zugriffskontrollen und Sicherheit (Ellmer).....	74
4.3	Backend und API-Schnittstellen	78
4.3.1	Einstiegspunkt und globale Konfiguration (<i>Ellmer</i>).....	78
4.3.2	Authentifizierung mit JWT.....	79
4.3.3	Rollenbasierte Autorisierung	79
4.3.4	Fachliche Zugriffsbeschränkung	80
4.3.5	Benutzerverwaltung und Profilzugriffe	81
4.3.6	Bestellprozess und Datenkonsistenz (<i>Orders</i>)	81
4.3.7	QR-Abschluss und Statuswechsel.....	83
4.3.8	Guthabenverwaltung und QR-Requests	85
4.3.9	Account- <i>Lifecycle</i> : Erstellung - Löschung	88
4.3.10	E-Mail-Service (Express + Gmail API).....	88
4.3.11	Zusammenfassung.....	92
4.4	PostgreSQL DB und DB-Zugriffe (<i>Ellmer</i>).....	92
4.4.1	Datenbankschema und ER-Diagramm	93
4.4.2	Organisation der Datenzugriffe.....	95
4.4.3	Integrität und Transaktionsmanagement	95

4.4.4	Performance und Skalierbarkeit	95
4.4.5	Zusammenfassung.....	96
4.5	Deployment des Bestellservices (<i>Ellmer</i>)	96
4.6	Rechtliche Absicherung	99
5	Softwarequalitätssicherung (Testing)	101
5.1	Unit-Tests (<i>Magos</i>)	101
5.1.1	Testframework	101
5.1.2	Order Service	101
5.1.3	Order Overview Component	103
5.1.4	Cart Page Component	105
5.2	Integration-Tests (<i>Ellmer</i>).....	107
5.2.1	Verwendetes Testframework.....	108
5.2.2	Testarchitektur und Rollenmodell.....	108
5.2.3	Aufbau und Struktur der Tests	111
5.2.4	Datenkonsistenz und Testisolierung	111
5.2.5	Zielsetzung und Mehrwert	112
5.3	Akzeptanz-Tests.....	112
5.4	Ergebnisse Testphase und Evaluation	113
6	Projektabschluss	115
6.1	Ergebnisse	115
6.2	Projektverlauf.....	121
6.3	Zukunftsausblick.....	124

Abkürzungsverzeichnis

Tabelle 1: Abkürzungsverzeichnis

Abkürzung	Bedeutung
ACID	Atomicity, Consistency, Isolation, Durability
AGB	Allgemeine Geschäftsbedingungen
API	Application Programming Interface
AWS	Amazon Web Services
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database (Datenbank)
DBMS	Database Management System
DSGVO	Datenschutz-Grundverordnung
ECS	Elastic Container Service
EC2	Elastic Compute Cloud
EU	Europäische Union
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaaS	Infrastructure as a Service
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MFA	Multi-Factor Authentication
MVCC	Multi-Version Concurrency Control

Abkürzung	Bedeutung
NIST	National Institute of Standards and Technology
OAuth	Open Authorization
ORDBMS	Object-Relational Database Management System
PaaS	Platform as a Service
PDF	Portable Document Format
QR	Quick Response
RDS	Relational Database Service
REST	Representational State Transfer
SaaS	Software as a Service
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
VPS	Virtual Private Server
VM	Virtual Machine

1 Einleitung

Diese Diplomarbeit beschreibt die Planung, Konzeption und Umsetzung einer webbasierten Bestellapplikation. Ziel ist es, den Entwicklungsprozess einer modernen Softwarelösung nachvollziehbar darzustellen und dabei sowohl technische als auch organisatorische Aspekte zu berücksichtigen.

Kapitel 1 (Planung) legt die inhaltliche und organisatorische Grundlage des Projekts. Es beschreibt die Ausgangslage, die Projektidee sowie die definierten Anforderungen und Ziele. Darüber hinaus werden die Projektorganisation, die zeitliche Planung und die Rollenverteilung im Projektteam erläutert. Ergänzend dazu werden relevante Stakeholder analysiert, Personas erstellt und im Rahmen von Machbarkeitsstudien verschiedene technische und infrastrukturelle Umsetzungsvarianten untersucht und bewertet.

Aufbauend darauf behandelt **Kapitel 2 (Projektrecherche)** die theoretischen und technologischen Grundlagen der Arbeit. Vorgestellt werden die eingesetzten Werkzeuge, Programmiersprachen und Frameworks sowie deren Auswahlkriterien. Weitere Schwerpunkte bilden Datenschutz, Verifizierung und Authentifizierung, der Einsatz von *PostgreSQL* als *relationales* Datenbanksystem, grundlegende Konzepte der *Microservice-Architektur* sowie *Containerisierung* und *Cloud-Hosting*. Zusätzlich wird der Einsatz ausgewählter *Libraries* und unterstützender Technologien erläutert.

Die praktische Umsetzung der Anwendung steht im Mittelpunkt von **Kapitel 3 (Umsetzung)**. Dieses Kapitel beschreibt die Projektstruktur sowie die Implementierung des *Frontends*, des *Backends* und der *Datenbankanbindung*. Zudem wird das *Deployment* der Anwendung in einer produktiven Umgebung dargestellt.

Kapitel 4 (Sicherung der Softwarequalität) befasst sich mit der *Qualitätssicherung* der entwickelten Anwendung. Es werden die eingesetzten *Teststrategien* für *Frontend* und *Backend* vorgestellt und deren Bedeutung für die Stabilität und Zuverlässigkeit der Software erläutert.

Den Abschluss bildet **Kapitel 5 (Projektabschluss)**, in dem die erzielten Ergebnisse zusammengefasst, der Projektverlauf reflektiert und ein Ausblick auf mögliche Weiterentwicklungen gegeben wird. Eine kurze Anleitung zur Nutzung der Bestellapplikation rundet die Arbeit ab.

Insgesamt bietet diese Arbeit einen strukturierten und praxisnahen Einblick in die Planung und Umsetzung eines modernen Softwareprojekts.

2 Planung

In diesem Kapitel wird die Planung und Entstehung des Projektes thematisiert. Dabei wird die Ausgangslage, aber auch die Ziele und die Aufgabenverteilung näher beschrieben.

2.1 Aufgabenstellung

2.1.1 Projektfindung und Ausgangslage

Im schulischen Alltag stellt die Organisation von Bestellungen am Buffet eine wiederkehrende Herausforderung dar. Der bestehende Bestellprozess ist durch lange Wartezeiten, weite Wege zum Buffet sowie durch die Verwendung handschriftlicher Zettel geprägt. Diese Vorgangsweise ist nicht nur ineffizient, sondern erschwert auch eine strukturierte Erfassung und Auswertung der Bestellungen.

Derzeit werden Bestellungen ausschließlich persönlich und handschriftlich aufgenommen. Dadurch entstehen ein erhöhter Zeitaufwand, mangelnde Übersichtlichkeit sowie zusätzliche Fehlerquellen. Zudem fehlt eine digitale Unterstützung, die einen schnellen und transparenten Bestellprozess ermöglichen würde.

Genau hier kann eine digitale Lösung Abhilfe schaffen: Bestellungen können einfach und schnell erfasst und in Echtzeit an den Buffetbetreiber übermittelt werden. Auf diese Weise werden Wartezeiten verkürzt und die Übersichtlichkeit des Bestellprozesses optimiert.

2.1.2 Konzept und Rahmenbedingungen

Benutzer / Zielgruppe

Die Bestell-App „HungerSatt“ richtet sich primär an die Kunden des Schulbuffets „HungerSatt“. Unter die Zielgruppe fallen Schüler, Lehrer sowie das Verwaltungspersonal. Auf der Bedienerseite führt die Anwendung beim Buffet-Inhaber selbst zur Effizienzsteigerung.

Allgemeine Zieldefinition

Ziel dieser Diplomarbeit ist die Entwicklung eines Online-Bestellsystems, das es den Kundinnen und Kunden des Schulbuffets ermöglicht, ihre Bestellungen bequem und schnell, beispielsweise direkt aus den Klassenzimmern, aufzugeben. Dabei sollten lange Warteschlangen in den Pausen vermieden und gleichzeitig ein effizienterer, übersichtlicherer Bestellprozess sowie eine optimierte Buffetverwaltung gewährleistet werden.

Für das Schulbuffet bedeutet dies die Umsetzung und Implementierung eines Bestellsystems, mit dessen Hilfe Wartezeiten reduziert und der gesamte Buffetalltag effizienter gestaltet werden kann. Die Benutzer der App profitieren von einem schnellen und unkomplizierten Bestellvorgang, können Speisen komfortabel vorbestellen, und die Abwicklung sowie Speicherung

ihrer Daten erfolgt unter höchsten Sicherheitsstandards und entspricht der Datenschutzgrundverordnung (DSGVO).

Systemvoraussetzungen

Aufgrund der relativ großen Zielgruppe wird die Bestell-App so entwickelt, dass sie von unterschiedlichen Geräten und Betriebssystemen bequem über den Browser gestartet werden kann. Wichtig ist auch der *serverseitige Aufbau der Anwendung (Backend)*, dieser soll ständig bereitstehen und dennoch kostengünstig sein. Das *Frontend und Backend* sollte also immer gleichermaßen erreichbar sein und muss daher entsprechend gehostet werden.

2.1.3 Anforderungen und Ziele

Die Bestell-App „HungerSatt“ ist eine innovative Anwendung, die einen effizienten und schnellen Bestellservice verspricht. Dabei profitieren Kunden von kürzeren Wartezeiten beim Bestellen und für das Buffetmanagement wird der gesamte Alltag optimiert.

Funktionelle Anforderungen

Muss-Ziele

- **Benutzerfunktionen**
 - **Sign-up / Sign-in:** Benutzer können sich mit E-Mail-Adresse und Passwort registrieren und anmelden. Dabei wird ihnen beim Registrieren automatisch eine Rolle zugewiesen. Die verwendete E-Mail wird automatisch über das dazugehörige Microsoft-Konto verifiziert.
 - **Menüübersicht:** User können Gerichte auswählen. Es gibt auch Filtermöglichkeiten (z. B. vegetarisch).
 - **Guthaben und Bestellsystem:** Kunden verwenden ihr Guthaben, welches sie zuvor persönlich aufladen, um Bestellungen zu tätigen. Anschließend erhalten sie eine Bestellbestätigung. Die Kunden können das Guthaben jederzeit online einsehen und auffüllen. Der gesamte Vorgang wird als Bestellung gespeichert und ist zusätzlich mit einem *Loggingsystem* ausgestattet.
 - **Warenkorb:** Die User können im Warenkorb noch einmal eine Zusammenfassung ihrer Bestellung ansehen - mit Mengenangaben, Zwischensummen und etwaigen Anmerkungen.
- **Inhaberfunktionen**
 - **Dashboard:** Das Dashboard stellt dem Inhaber in Echtzeit sämtliche offenen Bestellungen übersichtlich dar. Dadurch ist jederzeit eine aktuelle Übersicht über den Bestellstatus gewährleistet.
 - **Menüplanung:** Über diese Funktion können Menüs erstellt, angepasst und verwaltet werden. Die erstellten Menüs sind anschließend für die Benutzer in der Menüübersicht einsehbar.

- **Verwaltung von Gerichten:** Einzelne Gerichte können detailliert definiert werden. Dabei werden zentrale Informationen wie Name, Preis, Allergene sowie die jeweilige Kategorie festgelegt.
- **Benutzerverwaltung:** Diese Funktion ermöglicht dem Inhaber einen umfassenden Überblick über alle aktiven Benutzerkonten. Zusätzlich besteht hier die Möglichkeit, das Guthaben der Kunden aufzuladen oder bei Bedarf Konten zu sperren.
- **Backend-Services & Datenbanken**
 - Über eine Representational State Transfer (Rest)-API (Application Programming Interface) werden Bestellungen, Benutzer, Gerichte und Zahlungen abgewickelt.
 - Die Daten werden in einer PostgreSQL-Datenbank persistent gespeichert.
 - Das gesamte System wird auf einem Proxmox-Server gehostet, daher muss dieser verwendet und implementiert werden.
- **Identity & Access Management**
 - Das System ist rollenbasiert. Es besteht aus Inhaber, User und Admin Rollen. Das verwendete Konto wird überprüft und verifiziert.

Soll-Ziele

- **Benachrichtigungen:** Kunden erhalten eine Benachrichtigung, wenn eine Bestellung erfolgreich war oder neue Menüs ersichtlich sind. Der Inhaber bekommt ebenfalls eine Benachrichtigung, wenn eine Bestellung eingegangen ist.
- **Bestellbestätigung:** Der Inhaber soll eine Bestellung per *Quick-Response Code (QR-Code)* abschließen können.
- **Finanzübersicht für den Anbieter**
 - Umsatzstatistiken pro Tag/Woche/Monat: Der Inhaber selbst kann eine bestimmte Zeitspanne wählen, um den Gesamtumsatz einzusehen.
 - Die Finanzübersicht gibt einen Überblick über einzelne Bestellungen bzw. Bestellungskategorien.

Nicht-funktionelle Anforderungen

Soll-Ziele

- **Performance:** Das System soll auch bei hoher Nutzauslastung noch gut ansprechbar sein.
- **Automatisierte Backups zur Datensicherheit**
 - Tägliche Backups der Datenbank
 - Notfall-Wiederherstellung
- **Deployment und Hosting:** Bereitstellung des Frontend und auch Verfügbarkeit des Backend über den Server
- Aufbau der *serverseitigen Backend-Anwendung* entsprechend einer modernen *Multi-Service Architektur*

Kann-Ziele

- **Zentrales Logging & Monitoring**, um etwaige Fehler überprüfen zu können

- **Progressive Web-App**

Technische Risiken

- Unerwartete Komplexität von *Frontend und Backend*
- Hosting der Multiservice-Architektur kann kompliziert werden
- Probleme und Schwierigkeiten bei *Datensicherung und Zugriffskontrolle*
- Abhängigkeit von *Proxmox-Server*
- Probleme mit *Netzwerken und Firewall*

Schätzung des Arbeitsaufwandes

Tabelle 2: Schätzung des Arbeitsaufwandes

Bereich	Aufgabe / Inhalt	Stunden
Frontend	Login / Registrierung	17
	Menüübersicht (Tages-/Wochenansicht, Filtermöglichkeiten)	20
	Gericht-Details & Warenkorb	23
	Benutzerverwaltung + Bestellbestätigung	25
	Dashboard	17
	Menüplaner CRUD & Gerichte verwalten (Name, Preis, Allergene, Kategorien)	36
Backend-Services	REST-API (Bestellungen, Benutzer, Gerichte, Zahlungen)	40
	PostgreSQL-Datenbankmodell, CRUD & Persistenz	30
	Business-Logik, Validierung, Services	20
Hosting	PostgreSQL-Datenbank und Container einrichten	35
Identity & Access Management	Keycloak Integration, Rollenmodell (Kunde, Inhaber, Admin), Token-Handling	25
Qualitätssicherung	Testphase, Unit-Tests, Integrationstests und Systemtests	30
Dokumentation	Recherche und vollständige Dokumentation des Projektes	60

2.2 Projektorganisation

2.2.1 Meilensteine und Zeitplan

Tabelle 3: geplante Projektphasen und Meilensteine

Vorbereitungsphase	
Design und Entwurf	24.09.2025
Anfertigung eines Pflichtenheft	30.09.2025
Recherche	30.10.2025
Implementierung und Test	
Erstellung des Frontends mit Funktionen und Design	16.10.2025
API-Schnittstelle mit Backend und MultiService-Architektur	30.11.2025
Vorbereitungen für das Hosting	30.11.2025
Benutzer- und Rollenverwaltung	06.01.2026
Hosting der Microservicearchitektur	26.01.2026
Implementierung	
Frontend und Backend hosten und TestBuild	19.02.2026
Voraussichtliche Fertigstellung	04.03.2026

Gantt-Chart

Der geplante Projektzeitplan ist in *Abbildung 1: Zeitplan Gantt-Chart* als Gantt-Chart dargestellt.

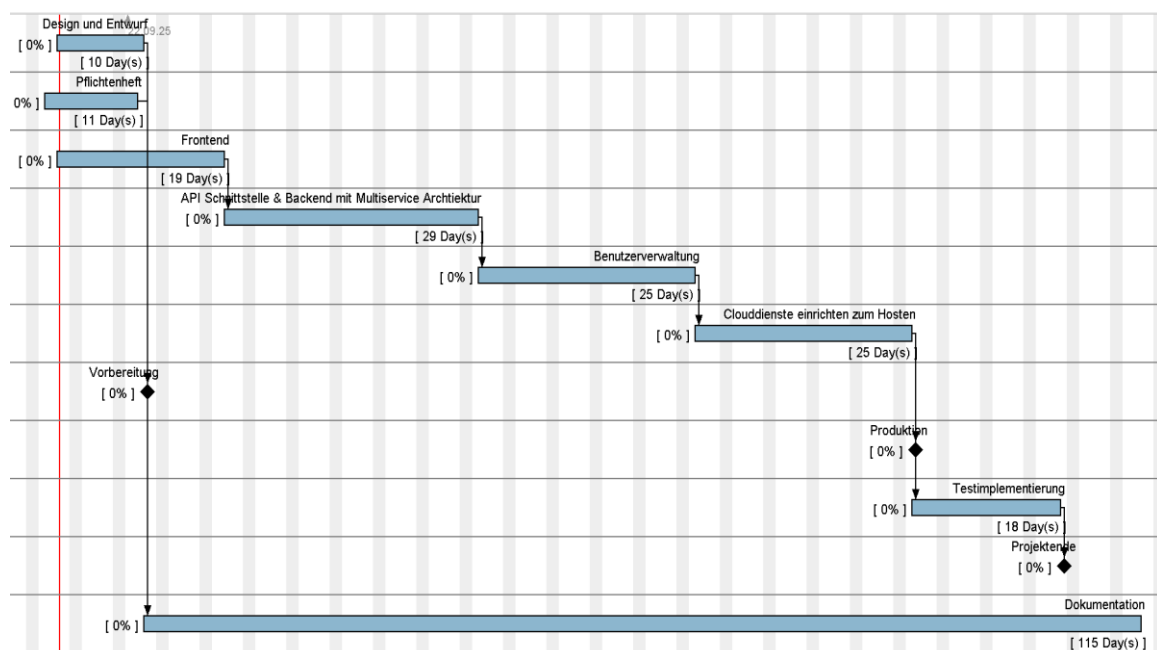


Abbildung 1: Zeitplan Gantt-Chart

Projektmeilensteine

Vorbereitungsphase

- **Design & Entwurf**
 - Erstellung von Mockups für alle Frontend-Seiten
 - Architektur und Design der Backend-Services (*REST-API*, Datenbankanbindung)
- **Pflichtenheft (30.09.2025)**
 - Fertigstellung und Abgabe des Pflichtenhefts als verbindliche Grundlage
- **Recherche (30.10.2025)**
 - *PostgreSQL, Datenschutz, Datensicherung und Rollenverwaltung, Proxmox Server und Deployment*
 - *NestJS, Application Programming Interface (API) und Entwicklungsumgebung (IDE), Angular Framework*

Implementierung

- **Frontend (16.10.2025)**
 - Entwicklung aller Kernseiten mit Angular:
 - Login/Registrierung
 - Menüübersicht
 - Warenkorb mit Mengen, Zwischensumme, Checkout
 - Benutzerverwaltung für Admins (Rollenverwaltung)
 - Erste Tests auf Usability & Responsiveness
- **Backend-Anwendung (30.11.2025)**
 - API-Schnittstellen
 - Entwicklung der MultiService-Architektur mit NestJS
 - Anbindung an der PostgreSQL DB
 - Umsetzung von *CRUD-Funktionen (Create-, Read-, Update-, Delete-Funktionen)* für Gerichte, Menüplaner und Bestellungen
- **Hosting und Deployment planen (30.11.2025)**
- **Benutzer und Rollenverwaltung (06.01.2025)**
 - Einrichtung der Rollen Kunde, Inhaber, Admin
- **Vorbereitungen fürs Hosting (30.01.2026)**

Testphase und Production

- Testing
- Testlauf mit Fehlerbehebung

Projektabschluss

Projektablauf

Der geplante Projektablauf ist in folgender *Abbildung 2* noch einmal im Überblick dargestellt.

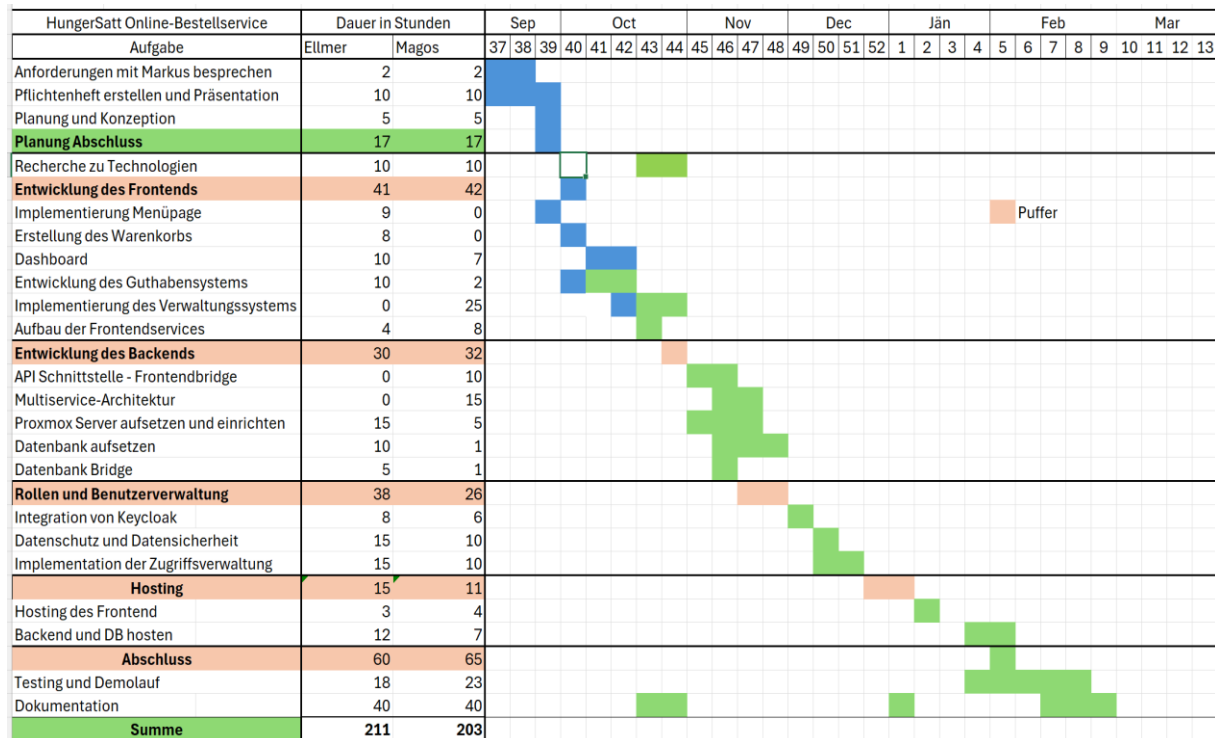


Abbildung 2: Überblick Projektablauf

2.2.2 Rollen- und Aufgabenverteilung

Rollenverteilung

Tabelle 4: Rollenverteilung

Auftraggeber	Schulbuffet „HungerSatt“ – Markus Gruber
Hauptbetreuer	Mag. Robert Gröbl
Nebenbetreuer	Dipl. -Ing. (FH) Dietmar Winkler
Projektleiter	Johannes Ellmer
Projektteam	Christos Magos, Johannes Ellmer

Aufgaben – Johannes Ellmer

Dokumentation und Recherche

PostgreSQL-Datenbank, Datenschutz, Rollenverwaltung, Hosting

Frontend

- Entwicklung der Kundenansicht und alle dazugehörigen Pages.
- Implementierung eines Guthabensystems. Dies ermöglicht Kunden ihr Konto aufzuladen, um anschließend Bestellungen abwickeln zu können.

Backend

- Architektur und Design der *PostgreSQL*-Datenbank und Entwicklung der Datenbank-Bridge, mit der die DB mit dem restlichen Backend kommuniziert. Außerdem erstellt das System regelmäßige Backups und Datensicherungen und verfügt über ein Logging-system.

Rollenverwaltung

- Rollenbasierte *Zugriffskontrollen*. Der Zugriff auf bestimmte Pages aber auch bestimmte Daten ist nur ausgewählten Rollen und Benutzern möglich.
- Datenschutz und Datensicherung: Daten werden verschlüsselt versendet und generell werden moderne Protokolle verwendet. Zusätzlich wird der Datenverkehr geloggt, um etwaige Fehler analysieren bzw. frühzeitig erkennen zu können. Des Weiteren werden regelmäßig automatische Datenbackups zur Datensicherung erstellt.

Deployment

- Bereitstellung und *Hosting*

Aufgaben – Christos Magos**Dokumentation und Recherche**

NestJS und APIs, Angular Framework

Frontend

- Entwicklung der Ansicht für den Inhaber, in der alle grundlegenden Informationen wie offene Bestellungen, Menüplanung sowie Benutzer- und Guthabenverwaltung angezeigt werden. Außerdem ermöglicht das System einen übersichtlicheren und effizienteren Buffetalltag.

Backend

- Implementierung einer API-Schnittstelle zum Frontend sowie Ausarbeitung des Backends das aus einer Multiservice-Architektur besteht. Diese soll für eine bessere Skalierbarkeit sorgen.

Benutzerverwaltung

- Authentifizierung mit Microsoft-Konto und Sign-up / Sign-in

Deployment

- Das Frontend wird mit dem Server gehostet, um den Zugriff auf das Bestellservice aus verschiedenen Netzwerken, kostengünstig zu ermöglichen.

2.3 Stakeholder (Ellmer)

Im Folgenden werden die wichtigsten Stakeholder des Projekts beschrieben, ihre Interessen und Ziele erläutert sowie ihr Einfluss auf den Projekterfolg analysiert.

2.3.1 Stakeholder-Analyse

Schüler: Die Schüler stellen die primäre Nutzergruppe dar. Sie wollen sich bequem im System ortsunabhängig anmelden können. Das Hauptaugenmerk liegt auf einer bedienerfreundlichen, intuitiven und zuverlässigen App. Der Einfluss auf das Projektziel ist in dieser Zielgruppe sehr hoch einzustufen, da die Nutzerzufriedenheit wesentlich für den Erfolg der App ist.

Lehrkräfte: Lehrkräfte und anderes Schulpersonal stellen eine weitere Nutzergruppe dar, die dieselben Ansprüche haben. Trotz ihrer recht kleinen Mitgliederanzahl, ist die Auswirkung auf das Projektziel dennoch hoch einzuschätzen. Besonders die Vorbildrolle der Lehrkräfte kann das allgemeine Feedback und das Nutzerverhalten beeinflussen.

Schulbuffet / Inhaber Markus Gruber: Das Buffet ist für die operative Umsetzung zuständig. Der Erfolg der App hängt stark davon ab, wie strukturiert und effizient Abläufe von Seiten des Schulbuffets abgewickelt werden können.

Entwickler: Das Entwicklerteam ist für die Konzeption und Implementierung der Anwendung verantwortlich. Die Qualität der Softwareentwicklung bestimmt im Wesentlichen den Projekterfolg.

Cloud Anbieter: Für das Hosten bzw. Bereitstellen der Software werden gängige Cloudanbieter genutzt. Die Wahl des Anbieters beeinflusst zwar nicht direkt den Projekterfolg, hat jedoch Auswirkungen auf die Zuverlässigkeit und Verfügbarkeit des Systems. Kommt es zu Störungen oder Ausfällen beim Hosting, kann dies die Erreichbarkeit der Anwendung und damit die Funktionsfähigkeit erheblich beeinträchtigen.

Als nächstes werden die identifizierten Stakeholder hinsichtlich ihres Einflusses auf das Projektziel sowie ihren Anteil am Projekterfolg eingeordnet. Die Kombination dieser beiden Dimensionen ermöglicht die Betrachtung und Einordnung der relevantesten Akteure im Rahmen der **Stakeholder-Matrix**, die in der folgenden Tabelle 5: Stakeholder-MatrixTabelle 5 ersichtlich ist.

Tabelle 5: Stakeholder-Matrix

Stakeholder	Interessen / Ziele	Einfluss	Interesse
Schüler	Einfache Nutzung, schnelle Anmeldung, stabile App	Sehr hoch	Hoch
Lehrkräfte	Reibungsloser Ablauf, einfache Integration	Hoch	Mittel
Schulbuffet	Effiziente Abläufe, klare Bestellübersicht	Sehr hoch	Sehr hoch
Cloud Anbieter	Stabile Infrastruktur, Sicherheit	Mittel	Niedrig

2.3.2 Personas

Persona 1: Schüler – Lukas Steiner (*Abbildung 3*)

Name: Lukas Steiner

Alter: 18 Jahre

Geschlecht: Männlich

Schulform: Höhere Technische Lehranstalt (HTL) für Informatik

Klassenstufe: 5. Jahrgang

Wohnort: St. Johann im Pongau

Technische Affinität: Sehr hoch

Genutzte Geräte: Smartphone, PC

Motivation / Ziele:

- Schneller, unkomplizierter Bestellvorgang über das Smartphone
- Echtzeitübersicht über verfügbare Speisen und mögliche Abholzeiten
- Modernes, responsives Design mit kurzen Ladezeiten

Frustrationen:

- Lange Warteschlangen am Buffet
- Langsame oder unübersichtliche Benutzeroberflächen
- Fehlende mobile Optimierung bei Apps

Persönliche Merkmale:

Lukas interessiert sich stark für Software, App-Design und digitale Innovationen. Er programmiert gelegentlich kleine Projekte und testet gerne neue Anwendungen. Er ist vertraut mit Cloud-Diensten wie Microsoft Azure, OneDrive und Teams. Er ist offen für Neues, aber schnell kritisch, wenn die App nicht flüssig funktioniert.



*Abbildung 3: Persona 1
(KI-generiert)*

Persona 2: Lehrkraft – Mag. Ingrid Bauer (*Abbildung 4*)

Name: Mag. Ingrid Bauer

Alter: 62 Jahre

Geschlecht: Weiblich

Beruf: Lehrerin für Mathematik und Physik

Schule: Bundeshandelsakademie St. Johann (JOHAK)

Wohnort: St. Johann im Pongau

Technische Affinität: Gering bis mittel



*Abbildung 4: Persona 2
(KI-generiert)*

Genutzte Geräte: Laptop, Handy

Motivation / Ziele:

- Bequemer Zugriff auf das Buffetangebot während der Pausen
- Zeitersparnis durch Vorbestellungen ohne Wartezeit
- Einfache, gut lesbare Benutzeroberfläche ohne technische Hürden

Frustrationen:

- Komplizierte oder unübersichtliche Apps
- Fehlende technische Unterstützung
- Wenn Funktionen nicht selbsterklärend sind

Persönliche Merkmale:

Frau Bauer nutzt digitale Medien im Unterricht, ist jedoch keine begeisterte Technik-Anwenderin. Sie arbeitet mit Microsoft Teams und E-Mail, hat aber gelegentlich Schwierigkeiten mit neuen Apps und komplexen Benutzeroberflächen. Ingrid Bauer ist engagiert und strukturiert, legt Wert auf Zuverlässigkeit und Einfachheit. Sie steht digitalen Lösungen offen gegenüber, bevorzugt jedoch klare und stabile Systeme.

Persona 3: Buffetinhaber/Angestellte (Abbildung 5)

Alter: 45 Jahre

Geschlecht: Männlich

Beruf: Inhaber und Betreiber/Angestellter des Schulbuffets

Technische Affinität: Mittel

Genutzte Geräte: Büro-PC oder Smartphone mit Kassensystem

Motivation / Ziele:

- Übersichtliche Darstellung der eingegangenen Bestellungen
- Verlässliche Prognose für Einkaufs- und Vorbereitungsplanung
- Minimierung von Wartezeiten und Fehlbestellungen

Frustrationen:

- Technische Ausfälle während Stoßzeiten
- Unübersichtliche oder fehlerhafte Bestelllisten
- Fehlende Möglichkeit, Änderungen in Echtzeit einzusehen

Persönliche Merkmale:

Markus Gruber/der Angestellte ist praktisch orientiert, effizient und kundenbewusst. Er legt Wert auf funktionierende, verständliche Systeme, die Arbeitsprozesse vereinfachen und Zeit sparen. Er ist offen für Digitalisierung, sofern sie seinen Betrieb effizienter macht.



Abbildung 5: Persona 3
(KI-generiert)

2.4 Machbarkeitsstudie und Variantenbildung (Ellmer)

Im Rahmen der Entwicklung des digitalen Bestellsystems für das Schulbuffet wurde untersucht, auf welche Weise die Anwendung technisch betrieben und öffentlich erreichbar gemacht werden kann. Ziel der Machbarkeitsanalyse war es, eine *Hosting*-Lösung zu finden, die technisch zuverlässig funktioniert und gleichzeitig innerhalb eines begrenzten Budgets betrieben werden kann.

Das System besteht aus mehreren Komponenten:

- *Frontend*: Webanwendung für Benutzerinnen und Benutzer
- *Backend*: Serveranwendung mit Geschäftslogik und *REST-API*
- *Datenbank*: *PostgreSQL*-Datenbank zur Speicherung von Benutzerkonten, Bestellungen und Speiseplänen

Für den produktiven Betrieb müssen diese Komponenten stabil erreichbar sein und möglichst wenig administrativen Wartungsaufwand verursachen.

2.4.1 Hosting mittels *Proxmox*-Server

Als erste Option wurde geprüft, ob der Bestellservice auf dem schuleigenen *Proxmox*-Server betrieben werden kann. *Proxmox Virtual Environment* ermöglicht den Betrieb von virtuellen Maschinen oder Containern und eignet sich grundsätzlich für das Hosting von Webanwendungen. [1]

Technisch wäre es möglich gewesen, *Frontend*, *Backend* sowie die *PostgreSQL*-Datenbank in separaten virtuellen Maschinen oder Containern bereitzustellen. Damit hätte das System vollständig innerhalb der bestehenden Schulserverinfrastruktur betrieben werden können.

Bewertung

In der Praxis scheitert dieser Ansatz an den Rahmenbedingungen des Schulnetzwerks. Netzwerkkomponenten wie *Firewall*, *Routing* und *Portfreigaben* werden zentral administriert und können im Rahmen des Projekts nicht flexibel angepasst werden. Dadurch wäre eine stabile externe Erreichbarkeit der Anwendung nicht gewährleistet.

Der Zugriff auf den Bestellservice wäre somit überwiegend auf das interne Schulnetz beschränkt gewesen. Lösungen wie Tunnel oder ähnliche Umgehungsmechanismen wären technisch möglich, jedoch für eine größere Anzahl an Nutzern im schulischen Umfeld nicht praktikabel. [1]

Ergebnis

Der Betrieb auf dem schuleigenen *Proxmox*-Server wurde daher verworfen, da die notwendige externe Erreichbarkeit nicht zuverlässig bereitgestellt werden kann. [1]

2.4.2 Cloud-Dienste

Da ein Hosting innerhalb des Schulnetzwerks nicht sinnvoll umsetzbar war, wurde der Einsatz externer *Cloud-* bzw. *Hosting*dienste untersucht. Ziel ist es, die Anwendung außerhalb der schulischen Infrastruktur zu betreiben und dadurch eine stabile öffentliche Erreichbarkeit zu gewährleisten.

Die bestehende Systemarchitektur umfasst:

- Frontend: *Single-Page-Application (SPA)*
- Backend: serverseitige Anwendung mit *REST-API*
- Datenbank: *PostgreSQL*-Datenbank

Nach Möglichkeit sollten *Managed Services* eingesetzt werden, um administrative Aufgaben wie Wartung, Updates und Monitoring zu reduzieren.

Variante 1: AWS

Amazon Web Services (AWS) stellt mit Diensten wie *Elastic Compute Cloud (EC2)* oder *Elastic Container Service (ECS)* Möglichkeiten zur Ausführung von Containeranwendungen bereit. Für relationale Datenbanken kann der Managed-Service *Amazon Relational Database Service (RDS)* verwendet werden, der Funktionen wie Backups, Monitoring und Updates integriert bereitstellt. [2, 3]

Kostenbewertung: Für einen produktiven Betrieb entstehen laufende Kosten durch Datenbankinstanzen, *Compute*-Ressourcen und Netzwerkkomponenten. Selbst minimal konfigurierte Systeme liegen häufig deutlich über dem vorgesehenen Budget. Eine typische Grundkonfiguration beginnt bei etwa 30 € pro Monat. [2, 3] Damit kann das Budget von maximal 10 € pro Monat nicht eingehalten werden.

Variante 2: Azure Cloud

Auch *Microsoft Azure* bietet mit *Azure App Service* eine Plattform zum Betrieb von Webanwendungen oder Containern. Für relationale Datenbanken steht der Dienst *Azure Database for PostgreSQL – Flexible Server* zur Verfügung, der unter anderem automatische Backups und Monitoring integriert. [4–6]

Kostenbewertung: In der untersuchten Konfiguration ergeben sich durch die Mindestpreise der Servicepläne sowie die Datenbankinstanz monatliche Kosten von rund 67 €. [4–6] Damit überschreitet auch Azure den vorgesehenen Kostenrahmen deutlich.

Variante 3: World4You

Als kostengünstigere Alternative wurde der Hostinganbieter *World4You* betrachtet. *World4You* bietet unter anderem *vServer*-Hosting an, auf denen eigene Anwendungen betrieben werden können. Dadurch wäre es möglich gewesen, die bestehende Architektur mit *Frontend*, *Backend* und *PostgreSQL*-Datenbank weiterhin containerbasiert zu betreiben, beispielsweise mittels *Docker*. Im Gegensatz zu großen Cloudplattformen handelt es sich hierbei

um eine klassische *Infrastructure as a Service (IaaS)*-Lösung. Das bedeutet, dass grundlegende Betriebsaufgaben wie Systemupdates, Sicherheitskonfigurationen, Backups oder Monitoring eigenständig durchgeführt werden müssen. [7, 8]

Kostenbewertung: Ein virtueller Server kann zwar innerhalb des vorgesehenen Budgets von maximal 10 € pro Monat betrieben werden. Im Vergleich zu vollständig verwalteten Cloudlösungen ist jedoch ein höherer administrativer Aufwand zu erwarten, da Wartung, Updates und Systemüberwachung selbst durchgeführt werden müssen. [7, 8]

2.4.3 Gewählte Hosting-Lösung

Da sowohl lokale Serverlösungen als auch große Cloudanbieter entweder technisch oder wirtschaftlich nicht geeignet waren, wurde eine alternative *Hosting*-Strategie gewählt. Dabei werden die einzelnen Komponenten der Anwendung auf spezialisierte Plattformen verteilt.

Die finale Systemarchitektur verwendet folgende Dienste:

- *Netlify* für das Frontend
- *Render* für das Backend
- *Supabase* für die *PostgreSQL*-Datenbank

Das *Frontend* wird über *Netlify* bereitgestellt. Diese Plattform ermöglicht das Hosting moderner Webanwendungen mit automatischer Bereitstellung aus einem *Git*-Repository sowie integrierter *HTTPS*-Unterstützung.

Das *Backend* wird auf der Plattform *Render* betrieben. *Render* stellt eine Umgebung für serverseitige Anwendungen bereit und übernimmt Funktionen wie automatisierte Builds, Neustarts bei Fehlern sowie die sichere Veröffentlichung über *HTTPS*.

Für die Datenbank wird *Supabase* eingesetzt. *Supabase* stellt eine vollständig verwaltete *PostgreSQL*-Datenbank bereit und übernimmt Aufgaben wie Backups, Wartung und grundlegendes Monitoring.

2.4.4 Schlussfolgerung der Variantenbildung

Die Machbarkeitsanalyse zeigt, dass mehrere Hostingvarianten technisch realisierbar gewesen wären, sich jedoch deutlich hinsichtlich Kosten, Wartungsaufwand und Erreichbarkeit unterscheiden. Der Betrieb auf dem schuleigenen *Proxmox*-Server wurde aufgrund der fehlenden externen Erreichbarkeit verworfen. Große Cloudanbieter wie *AWS* oder *Azure* erfüllen zwar die technischen Anforderungen, überschreiten jedoch den vorgesehenen finanziellen Rahmen deutlich. Die Kombination aus *Netlify*, *Render* und *Supabase* stellt daher eine geeignete Lösung dar. Sie ermöglicht eine stabile öffentliche Erreichbarkeit der Anwendung, reduziert den administrativen Aufwand durch *Managed Services* und kann gleichzeitig innerhalb eines sehr geringen Kostenrahmens betrieben werden.

3 Projektrecherche

Im folgenden Abschnitt werden die verwendeten Technologien genauer beschrieben und elementare Themen erläutert. Ziel ist es, dem Leser die verwendeten Technologien und Komponenten des Projektes zu vermitteln, darunter auch jene Bestandteile, die für den Anwender später möglicherweise nicht mehr sichtbar sind. Darüber hinaus fließen in diesem Kapitel alle im Rahmen der Recherche für diese Diplomarbeit gewonnenen Erkenntnisse ein, um ein allgemeines, besseres Verständnis für das gesamte Thema zu schaffen.

3.1 Technologien *(Magos)*

3.1.1 Angular

Im Rahmen dieser Diplomarbeit wird eine webbasierte Applikation umgesetzt, die von Endbenutzern sowie dem Buffetinhaber selbst verwendet werden soll. Es folgen daher besondere Anforderungen an die Benutzeroberfläche, wie eine übersichtliche Darstellung der Gerichte, eine einfache Verwendbarkeit auf verschiedenen Endgeräten, sowie die Trennung der Funktionen mithilfe der Verwendung von Rollen. Zusätzlich soll die Anwendung stabil und wartbar sein, sowie für zukünftige Erweiterungen entworfen werden.

Für die Erfüllung dieser Anforderungen ist ein geeignetes Webframework notwendig, das die strukturierte Entwicklung einer komplexen Benutzeroberfläche, sowie eine zuverlässige Anbindung an das Backend ermöglicht. Die Entscheidung für diese Technologie fiel dabei auf Angular. Im folgenden Kapitel werden die relevanten Grundlagen, Konzepte und Eigenschaften des Frameworks erläutert, um diese Entscheidung nachvollziehbar zu begründen.

Angular ist ein Webframework zur Entwicklung sogenannter *Single Page Applications (SPA)*. Es ist von Google entwickelt und basiert auf der Programmiersprache *Typescript*. Das Ziel und der hauptsächliche Vorteil von Angular ist die erhöhte Übersichtlichkeit von großen und komplizierten Webanwendungen, wodurch eine bessere Wartbarkeit und langfristige Erweiterbarkeit ermöglicht wird. Aufgrund dieser Eigenschaften wurde Angular als Framework für die Frontend Entwicklung des Projekts ausgewählt. [9, 10]

Angular folgt einem komponentenbasierten Architekturansatz. Jede Komponente beinhaltet jeweils eine *Typescript Datei*. Diese Datei ist für die Logik zuständig. Eine *HTML-Datei (Hypertext Markup Language)* kontrolliert die Ansicht der Datei und mit einer *CSS-Datei (Cascading Style Sheets)* kann das Design zusätzlich bearbeitet werden. Diese Trennung von Logik und Design macht die Funktionsweise der Website übersichtlicher und vereinfacht den Entwicklungsprozess. Im Kontext des Bestellsystems ist der komponentenbasierte Ansatz von großem Vorteil, da die Entwicklung von Seiten mit einem komplizierten Aufbau, wie zum Beispiel der Menüplan, ohne einen solchen Ansatz, sehr zeitaufwändig wäre. Auch für die Übersicht der zahlreichen Seiten des Systems ist dieser Architekturansatz von Vorteil. Dies ermöglicht das Wechseln von Ansichten, ohne dass die Webseite neu geladen werden muss. [9, 10]

Ein zugehöriger *URL-Pfad (Uniform Ressource Locator)* kann für jede Komponente definiert werden. Diese Pfade werden dann verwendet, um zwischen den Komponenten der Applikation zu wechseln. Die Zugriffskontrolle für die jeweiligen Komponenten lässt sich mit sogenannten *Route Guards* implementieren. Damit können spezifische Rollen definiert werden, die Zugang zu bestimmten Komponenten haben sollen. Diese Funktionalität ist für die Umsetzung der rollenbasierten Zugriffskontrolle des Bestellsystems essenziell. Mithilfe der *Route Guards* können die Rollen Kunde und Inhaber definiert werden, damit sichergestellt wird, dass jeweils die richtigen Komponenten zur Verfügung gestellt werden.[10]

Die Kommunikation mit dem Backend erfolgt mittels dem *HttpClient* von Angular. Mit diesen können Daten im *JSON-Format (Javascript Object Notation)* zwischen Frontend und Backend ausgetauscht werden. Damit können beispielsweise Authentifizierungsinformationen an das Backend übertragen werden. Im Rahmen dieser Diplomarbeit wird *HttpClient* für die Verbindung mit den, mit *NestJS* implementierten, *API-Gateway* genutzt. [11]

Insgesamt stellt Angular für die Entwicklung des Frontends der Bestellapplikation einen geeigneten technologischen Grundsatz. Es lässt die klare Trennung von Logik und Ansicht zu, bietet ein integriertes Navigationssystem und ermöglicht die Kommunikation mit dem Backend. Diese Eigenschaften sind für größere Systeme mit mehreren Rollen von großer Bedeutung.

3.1.2 NestJS

Neben dem Frontend wird für die Entwicklung des Bestellsystems auch ein geeignetes Backend benötigt. Dieses Backend soll eine zuverlässige Schnittstelle zum Frontend bereitstellen. Die hauptsächliche Arbeit davon ist, die Verarbeitung von Bestellungen, Speisekarten und Benutzerdaten. Aspekte wie Erweiterbarkeit und Wartbarkeit müssen auch berücksichtigt werden.

Für die Erfüllung dieser Anforderungen wird ein Framework benötigt, welches die Entwicklung einer Web-API ermöglicht und für die Implementation einer *Multiservicearchitektur* geeignet ist. Für die Umsetzung des *Backends* wurde daher *NestJS* gewählt. Im folgenden Kapitel werden die grundlegenden Konzepte von *NestJS* sowie dessen Bedeutung im Kontext des Backends des Bestellsystems erläutert. [12]

Grundlagen und Architektur

NestJS ist ein serverseitiges Webframework für die Entwicklung von Webanwendungen und *APIs*. Es basiert auf *NodeJS* und verwendet intern Technologien wie *Express* und *Fastify*. Ein zentrales Merkmal von *NodeJS* ist die Unterstützung von *Typescript*, wodurch eine gut strukturierte Codebasis entsteht. [12]

Bei der Architektur von *NestJS* liegt der Fokus auf Modulen. Eine Anwendung besteht aus verschiedenen Modulen. Jedes dieser *Module* ist für einen bestimmten Funktionsbereich zuständig. Jedes Modul beinhaltet *Controller* und *Services*. *Controller* verarbeiten *eingehende HTTP-Anfragen* und *Services* sind für die eigentliche Geschäftslogik zuständig. Diese Trennung der

Aufgaben macht die Anwendung wartbar und unterstützt eine klare Verantwortungsverteilung. [13]

Das *Dependency-Injection-System* von *NestJS* bildet auch einen zentralen Bestandteil seiner Architektur. Damit werden Abhängigkeiten wie Services und Datenzugriffe automatisch verwaltet und zur Verfügung gestellt. Dank dieses Systems können Komponenten unabhängig voneinander entwickelt und erweitert werden. [13]

Ein zentrales Ziel bei dem Aufbau des *Backends* ist ein *Multiservice-orientierter Architekturan-satz*, also die Trennung der einzelnen Funktionsbereiche. Die *modulare Architektur* von *NestJS* eignet sich für die Erfüllung dieses Ziels sehr gut. Die verschiedenen funktionale Bereiche des Projekts, wie Benutzerverwaltung, Bestellungen oder Menüverwaltung können in Modulen oder Services umgesetzt werden. Damit erfolgt eine klare Trennung dieser Funktionen, was die Grundlage einer Multiservice Architektur bildet. Diese Struktur verbessert die Übersichtlichkeit des Systems und erleichtert somit auch zukünftige Erweiterungen.

Kommunikation mit dem Frontend

NestJS eignet sich besonders für die Entwicklung von *REST-basierten APIs*. *HTTP-Endpunkte* können mit der Verwendung von sogenannten Decorators klar definiert werden. Damit können *GET-, POST-, PUT- und DELETE- Anfragen* implementiert werden. Diese *Endpoints* ermöglichen die Kommunikation zwischen Frontend und Backend bzw. den Austausch von Daten im *JSON-Format*. [12]

Für die Funktionalität der Bestellapplikation stellt *NestJS* die *API* bereit, die dem Frontend ermöglicht, Informationen wie Bestellungen, Menüs und Benutzerdaten abzurufen. Eine *strukturierte Definition der Endpunkte* ist dabei wichtig, um den Aufbau der Kommunikation zwischen den beiden Systemen so klar nachvollziehbar wie möglich zu machen. [12]

Zusammenfassung

NestJS ist ein leistungsfähiges und strukturiertes Framework für die Entwicklung von modernen backend Anwendungen. Der modulare Aufbau und die *klare Trennung von Controller* und Services machen das Framework zu einem geeigneten System für die Entwicklung größerer Webanwendungen mit komplizierter Logik.

Aufgrund dieser Eigenschaften bildet *NestJS* im Rahmen dieses Projekts die Grundlage für die serverseitige Verarbeitung aller zentralen Funktionen und ermöglicht eine saubere, skalierbare und wartbare Umsetzung der *REST-API*.

3.2 Programmiersprache *Typescript* (Magos)

Es werden im Frontend als auch im Backend Webframeworks verwendet, die auf die Programmiersprache *Typescript* basieren. Um die verwendeten Technologien besser einordnen zu können ist es wichtig, die Grundlagen von *Typescript* sowie dessen Bedeutung im Projektkontext zu betrachten.

Typescript ist eine von Microsoft entwickelte Programmiersprache, die auf *Javascript* aufbaut und dieses erweitert. Der *Typescript*-Code wird nicht direkt im Browser ausgeführt. Es wird vorher in *Javascript* übersetzt, was *Typescript* mit bestehenden *Javascript*-Laufzeitumgebungen kompatibel macht. Im Gegensatz zu *Javascript* können in *Typescript* Datentypen explizit definiert werden. Dies ermöglicht die Erkennung von Fehlern bereits bei der Entwicklung, nicht erst bevor der Code ausgeführt wird. [14]

Im Rahmen dieser Diplomarbeit bildet *Typescript* die Grundlage für das im Frontend eingesetzte Angular, sowie das im Backend eingesetzte *NestJS*. Die gemeinsame Programmiersprache der zwei Frameworks ermöglicht eine klare und konsistente Definition von Modellen, Datenstrukturen und Schnittstellen. Dadurch wird die Entwicklung deutlich erleichtert und mögliche Fehlerquellen werden reduziert. [15]

Typescript erweitert *Javascript* und fördert eine strukturierte und wartbare Codebasis. Der Einsatz der Programmiersprache in *Angular* und *NestJS* fördert die Übersichtlichkeit und Erweiterbarkeit des gesamten Systems und ist damit sinnvoll für die Umsetzung des Projekts. [15]

3.3 Datenschutz und Verifizierung (Ellmer)

In webbasierten Systemen, die personenbezogene Daten verarbeiten, sind Datenschutz und Zugriffssicherheit zentrale Qualitäts- und Schutzanforderungen. Für den in dieser Diplomarbeit betrachteten Bestellservice bedeutet dies, dass sowohl die rechtlichen Vorgaben der *Datenschutzgrundverordnung (DSGVO)* als auch geeignete technische und organisatorische Maßnahmen konsequent berücksichtigt werden müssen. Neben der datenschutzkonformen Erhebung, Verarbeitung und Speicherung von Kundendaten ist sicherzustellen, dass ausschließlich berechtigte Personen Zugriff auf geschützte Ressourcen erhalten. Daher werden im Folgenden wesentliche Aspekte der *DSGVO* sowie zentrale Sicherheitsmechanismen wie Verifizierung, Authentifizierung, tokenbasierte Verfahren und *Cross-Origin Resource Sharing (CORS)* in den Kontext moderner Webanwendungen eingeordnet. [16–18, 27, 28]

3.3.1 Datenschutz und DSGVO

Datenschutz ist im Projektkontext relevant, weil der Bestellservice personenbezogene Daten (z. B. Kontodaten, Bestellhistorien) verarbeiten muss. Die *DSGVO* beschreibt dabei die rechtlichen Rahmenbedingungen und verfolgt das Ziel, Missbrauch sowie unbefugte Weitergabe personenbezogener Daten zu verhindern. [16] Sie fordert insbesondere Zweckbindung,

Transparenz und Kontrollmöglichkeiten für Betroffene. Daten dürfen nur verarbeitet werden, wenn ein legitimer Zweck besteht und Nutzer darüber informiert werden; außerdem sollen Betroffene grundsätzlich Kontrolle über ihre Daten behalten. [16, 17]

Für die praktische Umsetzung reichen rechtliche Vorgaben allein nicht aus: Es braucht technische und organisatorische Maßnahmen, insbesondere Verschlüsselung beim Transport, Zugriffsbeschränkungen sowie eine nachvollziehbare Backup-Strategie. Gerade in Websystemen ist es sinnvoll, diese Maßnahmen früh in die Entwicklung zu integrieren („Security by Design“), weil spätere Nachbesserungen meist deutlich aufwändiger sind. [18]

Bei der Speicherung in Cloud-Umgebungen ist zusätzlich zu prüfen, wo Daten verarbeitet werden. Rechenzentren außerhalb der EU sind datenschutzrechtlich besonders sorgfältig zu bewerten. Seit 2023 gilt für Datenübermittlungen in die USA unter bestimmten Bedingungen das *EU–US Data Privacy Framework*, das Übermittlungen in definierte, geprüfte Rahmenbedingungen einordnet. [19]

3.3.2 Verifizierung und Authentifizierung

Verifizierung und *Authentifizierung* werden im Alltag häufig vermischt, verfolgen jedoch unterschiedliche Ziele: Verifizierung bestätigt, dass eine angegebene Identität bzw. ein Merkmal (z. B. eine E-Mail-Adresse) tatsächlich zur Person gehört; Authentifizierung prüft im Anmeldeprozess, ob jemand tatsächlich die behauptete Identität besitzt. [16, 20, 21] In der Praxis werden beide Schritte oft kombiniert, weil ein sicherer Zugriff erst dann sinnvoll ist, wenn sowohl die Identität plausibel ist als auch die Anmeldung zuverlässig abgesichert wurde.

In modernen Webanwendungen wird Authentifizierung häufig über externe Identitätsanbieter („Third-Party-Login“) umgesetzt, etwa über Microsoft oder Google. Das reduziert den Aufwand für Passwortverwaltung und ermöglicht zusätzliche Sicherheitsmechanismen wie *Multi-Faktor-Authentifizierung* (MFA). [20, 22] Im Bestellservice erfolgt die Anmeldung über die Schul-E-Mail-Adressen und einem eignen Benutzermanagement.

In webbasierten Architekturen wird Authentifizierung zudem häufig zustandslos realisiert. Dabei speichert der Server keine klassische Sitzung, sondern der Authentifizierungsstatus wird über kryptografisch geschützten Token übermittelt. Tokenbasierte Verfahren sind insbesondere bei *REST*-basierten Architekturen verbreitet. [23–25]

Tokenbasierte Authentifizierung und JSON Web Token (JWT)

Die tokenbasierte Authentifizierung stellt nach erfolgreicher Anmeldung ein Token aus, das bei weiteren Anfragen mitgesendet wird. Der Server prüft dieses Token und entscheidet anschließend, ob Zugriff gewährt wird. [23] Ein verbreitetes Format ist *JSON Web Token* (JWT). Ein *JWT* besteht aus *Header*, *Payload* und *Signatur*. Der *Header* enthält Metadaten, wie den *Payload* z. B. Benutzerinformationen (etwa Benutzer-ID oder Rollen) und eine digitale *Signatur* zur Sicherung der Integrität des Tokens. [23, 24] Tokens können außerdem für Verifizierungsprozesse eingesetzt werden, beispielsweise für E-Mail-Bestätigungen oder Passwort-Resets. [21, 26]

Entscheidend ist dabei, dass Tokens kryptografisch zufällig erzeugt, zeitlich begrenzt und sicher behandelt werden, um Missbrauch zu reduzieren. [23, 24]

Cross-Origin Resource Sharing (CORS)

Neben Authentifizierung ist auch die Zugriffskontrolle auf Browser-Ebene relevant. *Cross-Origin Resource Sharing (CORS)* ist ein Sicherheitsmechanismus, der kontrolliert, ob eine Webanwendung Anfragen an einen Server stellen darf, wenn diese von einer anderen *Origin* (z. B. anderer Domain oder Port) kommen. Damit lässt sich festlegen, welche Frontends auf eine *Server-Application Programming Interface (API)* zugreifen dürfen. [27, 28] Eine praxisnahe Konfiguration ist wichtig: Zu restriktive Regeln blockieren legitime Umgebungen (z. B. Test und Produktion), zu offene Regeln erhöhen unnötig die Angriffsfläche. [27, 28]

3.3.3 Zusammenfassung

Die *DSGVO* gibt die rechtlichen Leitlinien vor und verlangt zugleich technische Schutzmaßnahmen wie Verschlüsselung, Zugriffsbeschränkungen und Backups. [16–18] Verifizierung und Authentifizierung ergänzen diese Anforderungen technisch: Verifizierung sichert die Plausibilität der Identität (z. B. durch E-Mail-Bestätigung), Authentifizierung schützt den tatsächlichen Zugriff auf Ressourcen. [16, 20, 21] Tokenbasierte Verfahren – insbesondere *JWT* – sind in Webanwendungen verbreitet, sofern Token sicher erzeugt, zeitlich begrenzt und korrekt validiert werden. [23–26] Ergänzend reduziert *CORS* das Risiko unautorisierter Cross-Origin-Anfragen, indem zulässige Zugriffe kontrolliert werden. [27, 28]

3.4 PostgreSQL als relationale Datenbank (Ellmer)

Relationale Datenbanken sind für viele Informationssysteme unverzichtbar, weil sie Daten strukturiert speichern, zuverlässig verarbeiten und effizient abfragen können. Auch für die in dieser Diplomarbeit entwickelte Anwendung ist eine stabile *Persistenzschicht* wichtig: Bestell- und Kundendaten müssen dauerhaft gespeichert werden und auch bei gleichzeitigen Zugriffen korrekt und konsistent bleiben.

Dieses Kapitel stellt daher zuerst grundlegende Konzepte *relationaler* Datenbanken vor (z. B. *Primär- und Fremdschlüssel* sowie *Normalisierung*) und erläutert anschließend die *ACID*-Eigenschaften (*Atomicity, Consistency, Isolation, Durability*) als Grundlage transaktionssicherer Verarbeitung. Darauf aufbauend wird *PostgreSQL* als konkretes *Datenbankmanagementsystem (DBMS)* vorgestellt – mit Fokus auf Architektur, Erweiterbarkeit und Sicherheitsmechanismen – sowie der Zugriff darauf in containerisierten Umgebungen betrachtet.

3.4.1 Relationale Datenbanken im Allgemeinen

Relationale Datenbanken bilden eine wesentliche Grundlage moderner Softwarelösungen, da sie die strukturierte Speicherung und effiziente Abfrage großer Datenmengen ermöglichen.

Das relationale Datenmodell wurde 1970 von Edgar F. Codd entwickelt und gilt bis heute als Meilenstein der Informatik. Es basiert auf formalen Konzepten der *Mengenlehre* und *Prädikatenlogik*, die *konsistente Datenoperationen* erlauben. [29, 30]

Im *relationalen* Modell werden Daten in Tabellen (*Relationen*) organisiert, die aus *Zeilen* (*Tupeln*) und *Spalten* (*Attributen*) bestehen. Jede Tabelle beschreibt eine fachliche Entität, etwa einen *Kunden* oder eine *Bestellung*. [30]

Ein zentrales Merkmal *relationaler* Datenbanksysteme ist die *Datenunabhängigkeit*. Logische Datenstrukturen sind von der physischen Speicherung getrennt, wodurch Änderungen an der internen Organisation keine Auswirkungen auf Anwendungen haben und die Erweiterbarkeit des Systems unterstützt wird. [30]

Der Zugriff auf die Daten erfolgt über die *deklarative Abfragesprache Structured Query Language (SQL)*, die es erlaubt, benötigte Informationen klar zu beschreiben, ohne Details zur internen Umsetzung anzugeben. [30, 31]

Ein zentrales Element relationaler Datenbanken sind *Primär- und Fremdschlüssel*. Ein *Primärschlüssel* dient dazu, Datensätze eindeutig zu identifizieren und Mehrdeutigkeiten zu vermeiden. *Fremdschlüssel* stellen Beziehungen zwischen Tabellen her, indem sie auf den Primärschlüssel einer anderen Tabelle verweisen. Daraus ergeben sich verschiedene Beziehungen zwischen den Tabellen, die *Multiplizitäten* genannt werden. Infolgedessen ergibt sich beispielsweise eine 1:n-Beziehung. Das bedeutet, dass ein Datensatz mit mehreren anderen Datensätzen verknüpft werden kann. [30, 31]

In der Praxis ermöglicht dies beispielsweise, Bestellungen eindeutig einem bestimmten Kunden zuzuordnen. Gleichzeitig unterstützen diese Schlüssel die *referenzielle Integrität*, da definierte Regeln festlegen, wie sich Änderungen oder Löschungen auf abhängige Datensätze auswirken dürfen. [30, 31]

Diese *referenzielle Verknüpfung* ist eine zentrale Stärke relationaler Datenbanksysteme, da sie Konsistenz zwischen abhängigen Tabellen sicherstellt. Wird beispielsweise ein Kunde gelöscht, kann die Datenbank – abhängig von der definierten Regel (z. B. *Cascade* oder *Restrict*) – entweder automatisch alle zugehörigen Bestellungen entfernen oder den Löschvorgang verhindern. Dadurch können *Dateninkonsistenzen* vermieden werden. [30, 31]

Ein weiteres wichtiges Strukturprinzip relationaler Datenbanken ist die *Normalisierung*. Sie dient dazu, *Redundanzen* zu vermeiden und typische *Datenanomalien* zu reduzieren, indem Informationen logisch auf mehrere Tabellen verteilt werden. Dadurch bleiben Daten konsistent und leichter wartbar. [32]

Die klassischen *Normalformen* liefern dafür grundlegende Regeln: In der *ersten Normalform (1NF)* enthalten Attribute nur einen einzelnen Wert, etwa getrennte Felder für Straße und Hausnummer. Die *zweite Normalform (2NF)* stellt sicher, dass alle Nichtschlüsselattribute vollständig vom Primärschlüssel abhängen, sodass beispielsweise Produktinformationen nicht mehrfach in Bestellpositionen gespeichert werden. Die *dritte Normalform (3NF)* vermeidet indirekte Abhängigkeiten, etwa zwischen Postleitzahl und Ort. [32]

In der Praxis dienen diese Regeln als Orientierung für eine saubere Datenmodellierung. Abhängig von Performanceanforderungen kann es jedoch sinnvoll sein, gezielt von einer strikt normalisierten Struktur abzuweichen und einzelne Informationen bewusst redundant zu speichern. [32]

ACID-Transaktionen

Transaktionen sind ein zentrales Konzept relationaler Datenbanksysteme. Besonders bei geschäftskritischen Vorgängen und gleichzeitigen Zugriffen mehrerer Benutzer ist es wichtig, dass Datenänderungen konsistent und zuverlässig verarbeitet werden. Um dies sicherzustellen, müssen Transaktionen bestimmte Eigenschaften erfüllen, die unter dem Akronym *ACID* (*Atomicity, Consistency, Isolation, Durability*) zusammengefasst werden. Diese beschreiben die grundlegenden Anforderungen an robuste Transaktionssysteme. [33]

Atomicity (Atomarität) stellt sicher, dass eine Transaktion entweder vollständig ausgeführt oder vollständig zurückgesetzt wird. Teilweise ausgeführte Änderungen sind ausgeschlossen. Ein typisches Beispiel ist ein Geldtransfer: Erst wenn sowohl die Abbuchung als auch die Gutschrift erfolgreich sind, wird die Transaktion bestätigt, andernfalls wird der ursprüngliche Zustand wiederhergestellt. [33]

Consistency (Konsistenz) verlangt, dass eine Transaktion die Datenbank von einem gültigen Zustand in einen anderen gültigen Zustand überführt. Dabei dürfen keine definierten Integritäts- oder Geschäftsregeln verletzt werden. Tritt ein Regelverstoß auf, wird die gesamte Transaktion abgebrochen und zurückgesetzt. [31, 33]

Isolation beschreibt das Verhalten paralleler Transaktionen. Sie sollen sich so verhalten, als würden sie nacheinander ausgeführt, sodass sich gleichzeitige Zugriffe nicht ungewollt beeinflussen. Ohne ausreichende Isolation können inkonsistente Leseergebnisse entstehen. Zur Steuerung dieses Verhaltens definiert der SQL-Standard verschiedene Isolationsstufen. PostgreSQL nutzt zusätzlich *Multi-Version Concurrency Control (MVCC)*, wodurch parallele Lese- und Schreibzugriffe effizient möglich sind, ohne sich gegenseitig stark zu blockieren. [31, 33]

Durability (Dauerhaftigkeit) garantiert, dass bestätigte Änderungen dauerhaft gespeichert bleiben und auch bei Systemausfällen nicht verloren gehen. Technisch wird dies durch Protokollierungsmechanismen wie *Write-Ahead Logging* umgesetzt, die eine zuverlässige Wiederherstellung des konsistenten Zustands ermöglichen. [31, 34]

Die Bedeutung der *ACID*-Eigenschaften ergibt sich aus ihrem Zusammenspiel. Während Atomarität und Dauerhaftigkeit die physische Integrität der Daten sichern, gewährleisten Konsistenz und Isolation die logische Korrektheit der Verarbeitung. Gemeinsam bilden sie die Grundlage für verlässliche Datenverarbeitung in Mehrbenutzersystemen und sind insbesondere für Anwendungen mit hohen Anforderungen an Nachvollziehbarkeit und Sicherheit unerlässlich. [31]

3.4.2 PostgreSQL als DBMS

PostgreSQL ist ein objektrelationales Datenbankmanagementsystem (Object-Relational Database Management System, ORDBMS), das aus einem Forschungsprojekt an der University of California, Berkeley hervorgegangen ist. Dieses Projekt wurde unter der Leitung von Michael Stonebraker durchgeführt und trug den Namen POSTGRES. Ziel war es, das klassische relationale Datenmodell gezielt zu erweitern, etwa durch die Integration von Objekttypen, Vererbung und anpassbaren Operatoren, um komplexere Datenstrukturen besser abbilden zu können [35].

Im Jahr 1996 wurde PostgreSQL als Open-Source-Software veröffentlicht. Diese Entscheidung ermöglichte eine kontinuierliche Weiterentwicklung durch eine internationale Entwicklergemeinschaft, die bis heute von der PostgreSQL Global Development Group koordiniert wird. [35]

Dank seiner hohen Stabilität, Flexibilität und der Einhaltung moderner Standards hat sich PostgreSQL als eines der führenden Open-Source-Datenbanksysteme etabliert. Es findet breite Anwendung in unterschiedlichen Bereichen, darunter Wissenschaft, Industrie sowie öffentliche Verwaltung [35, 36].

Architektur und Systemaufbau

Architektonisch folgt *PostgreSQL* einem *Client-Server-Modell*, bei dem ein zentraler Serverprozess alle eingehenden Verbindungen verwaltet. Für jede *Client*-Verbindung wird ein eigener *Backend-Prozess* gestartet, wodurch einzelne Sitzungen voneinander isoliert sind und Fehler in einer Verbindung nicht unmittelbar das gesamte System beeinträchtigen.

Der interne Aufbau von *PostgreSQL* ist modular gestaltet. Verarbeitungsschritte wie *Parsing*, *Optimierung* und *Ausführung* von Abfragen sind klar voneinander getrennt, was eine strukturierte und nachvollziehbare Abfrageverarbeitung ermöglicht. Diese modulare Architektur trägt wesentlich zur Wartbarkeit und Erweiterbarkeit des Systems bei. [36, 37]

Für den gleichzeitigen Zugriff mehrerer Clients setzt *PostgreSQL* auf *Multi-Version Concurrency Control (MVCC)*. Dieses Verfahren erlaubt parallele *Lese- und Schreibzugriffe*, ohne dass umfangreiche Sperrmechanismen notwendig sind. Dadurch wird eine hohe Parallelität erreicht, während gleichzeitig *Konsistenz und Datenintegrität* gewährleistet bleiben. [37]

3.4.3 PostgreSQL in containerisierten Umgebungen

Mit der Verbreitung containerbasierter Architekturen (siehe [Kapitel 2.6](#)) hat sich auch der Betrieb von Datenbanksystemen verändert. *PostgreSQL* lässt sich aufgrund seiner stabilen *Client-Server-Architektur* in solchen Umgebungen typischerweise in einem eigenen Container betreiben, während Anwendungsdienste in separaten *Containern* laufen. [31, 33, 36, 38]

Die Kommunikation zwischen Anwendung und Datenbank erfolgt auch im Container-Kontext über standardisierte Netzwerkprotokolle, insbesondere *TCP/IP*. Container sind dabei häufig

über interne virtuelle Netzwerke verbunden, wodurch der Datenbankcontainer über einen festen Hostnamen erreichbar ist. Diese Trennung sorgt für eine lose Kopplung zwischen Anwendung und Datenbank und erleichtert Wartung sowie Skalierung des Gesamtsystems. [39, 40]

Der Zugriff auf *PostgreSQL* erfolgt technisch analog zu klassischen Installationen, etwa über definierte Verbindungsparameter wie Host, Port, Datenbankname und Zugangsdaten. In containerisierten Setups werden diese Informationen häufig über *Umgebungsvariablen* bereitgestellt, um Konfiguration und Anwendungscode sauber voneinander zu trennen. [38, 40, 41]

Da Container als kurzlebig gelten, wird der eigentliche Datenbestand in persistenten *Volumes* oder externen Speichersystemen abgelegt. Dadurch bleiben Daten auch bei Neustarts oder dem Austausch von Containern erhalten und die Anforderungen an Datenintegrität und Verfügbarkeit werden erfüllt. [39, 41]

Auch aus sicherheitstechnischer Sicht bleiben die etablierten Schutzmechanismen von *PostgreSQL* erhalten. Rollenbasierte Zugriffskontrollen, verschlüsselte Verbindungen sowie die Netzwerkisolation der Container tragen dazu bei, die Angriffsfläche zu reduzieren und den Datenbankzugriff auf autorisierte Dienste zu beschränken. [38, 39]

Insgesamt stellt der containerisierte Betrieb von *PostgreSQL* eine flexible und praxisnahe Möglichkeit dar, relationale Datenbanken in moderne Anwendungssysteme zu integrieren. Die klare Trennung von Anwendung und Datenbank unterstützt eine saubere Architektur und vereinfacht sowohl den Entwicklungs- als auch den Produktionsbetrieb. [38–41]

3.4.4 Zusammenfassung

Dieses Kapitel zeigt, warum relationale Datenbanken eine zentrale Grundlage für eine konsistente und nachvollziehbare Datenhaltung darstellen. *Primär- und Fremdschlüssel* sowie die *Normalisierung* tragen wesentlich zur Datenintegrität bei, indem sie Beziehungen eindeutig abbilden und Redundanzen reduzieren. *ACID*-Transaktionen stellen sicher, dass Datenänderungen auch bei parallelen Zugriffen oder in Fehlerfällen zuverlässig und korrekt verarbeitet werden.

Mit *PostgreSQL* kommt ein objektrelationales und erweiterbares *Datenbankmanagementsystem (DBMS)* zum Einsatz, das eine robuste Architektur, vielfältige Datentypen sowie differenzierte Sicherheits- und Zugriffskontrollen bietet. Durch den containerisierten Betrieb lässt sich *PostgreSQL* nahtlos in moderne Systemarchitekturen integrieren. Aspekte wie *Netzwerkisolation*, ausgelagerte Konfiguration und persistente *Volumes* sind dabei entscheidend, um Wartbarkeit, Verfügbarkeit und Datensicherheit im Entwicklungs- und Produktivbetrieb zuverlässig zu gewährleisten.

3.5 Grundlagen zur Microservice-Architektur

Die *Microservice-Architektur* ist ein zentrales Architekturkonzept moderner Softwareentwicklung. Im Gegensatz zu monolithischen Systemen wird eine Anwendung dabei in mehrere

unabhängigen Dienste zerlegt. Jeder Service übernimmt eine klar abgegrenzte Aufgabe, wird eigenständig betrieben und kann unabhängig entwickelt, bereitgestellt und skaliert werden. Dadurch steigt die *Flexibilität* des Systems, gleichzeitig verbessert sich die *Wartbarkeit* komplexer Anwendungen. [42–44]

3.5.1 Grundlagen und wesentliche Merkmale

Die Grundidee der *Microservice-Architektur* besteht darin, fachliche und technische Verantwortlichkeiten klar zu trennen. Anstelle einer großen Anwendung entstehen viele kleine, spezialisierte Services, die über definierte Schnittstellen zusammenarbeiten. Im Vergleich zu monolithischen Systemen ermöglicht dies eine gezieltere Weiterentwicklung einzelner Komponenten sowie eine effizientere Nutzung von Ressourcen. [42–44]

Zentrale Prinzipien der *Microservice-Architektur* sind:

- *Lose Kopplung*: Services sind möglichst unabhängig und kommunizieren nur über klar definierte Schnittstellen (z. B. *APIs*). Interne Änderungen bleiben dadurch lokal, solange die Schnittstelle stabil bleibt [42–44].
- *Hohe Kohäsion*: Ein Service bündelt fachlich zusammengehörige Funktionen und verfolgt einen klaren Zweck. Das erhöht Verständlichkeit und *Wartbarkeit*. [42–44]
- *Unabhängiges Deployment*: Services können separat entwickelt, getestet und ausgerollt werden. Änderungen an einem Service sollen nicht automatisch *Releases* anderer Services erzwingen. [42–44]
- *Verteilte Teamverantwortung*: Teams werden entlang von fachlichen oder technischen Servicegrenzen organisiert und übernehmen *Ownership* für „ihre“ Services (Entwicklung und Betrieb). [42–44]
- *Datenhoheit*: Jeder Service verwaltet seine eigenen Daten (Modell und Speicherung). Andere Services greifen nicht direkt auf diese Datenbank zu, sondern über Schnittstellen oder *Events*. [42–44]
- *Single-Responsibility-Prinzip*: Jeder Service hat genau eine Hauptverantwortung. Dadurch bleiben Umfang und Komplexität beherrschbar. [42–44]

3.5.2 Kommunikation / Koordination von Microservices

Die Zerlegung in viele Services erfordert klar definierte *Kommunikationsmechanismen*. Grundsätzlich wird zwischen *synchroner* und *asynchroner Kommunikation* unterschieden.

Bei der *synchronen Kommunikation* ruft ein Service einen anderen direkt über *REST-APIs* oder vergleichbare Schnittstellen auf und wartet auf dessen Antwort. Dies ist einfach umzusetzen, kann jedoch bei Ausfällen oder *Latenzen* zu Blockierungen im Gesamtsystem führen. [42, 43]

Die *asynchrone Kommunikation* nutzt *Message Broker* wie *RabbitMQ* oder *Apache Kafka*. Nachrichten werden in *Queues* abgelegt und unabhängig verarbeitet, was eine stärkere *Entkoppelung*, bessere *Lastverteilung* und höhere *Fehlertoleranz* ermöglichen. Gleichzeitig steigt der Aufwand für *Konsistenzsicherung* und Monitoring. [42, 43] Zusätzlich wird zwischen *direkter Kommunikation*, bei der Services ihre Zieladressen kennen, und *indirekter Kommunikation* über *Broker* oder Vermittlungsschichten unterschieden, wie in *Abbildung 6* dargestellt ist. Letztere erhöht *Resilienz* und *Skalierbarkeit*. [42, 43]

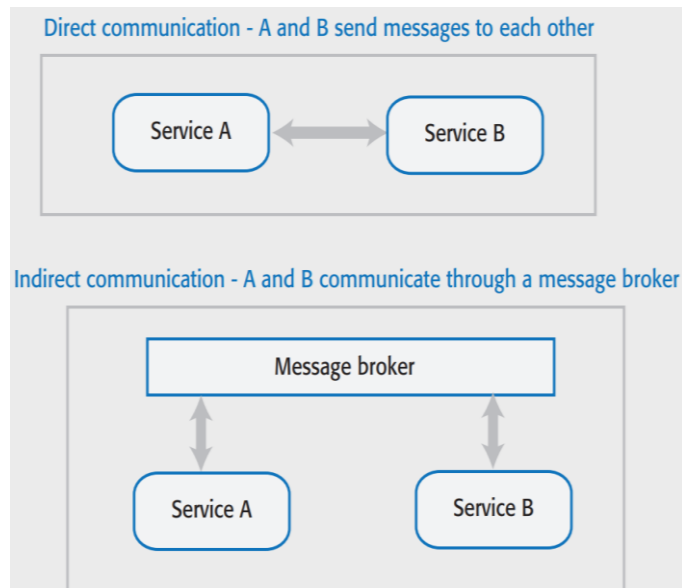


Abbildung 6: direct vs. indirect Kommunikation – Sommerville Microservice Architecture – F. 22

Bei der *Orchestrierung* steuert eine zentrale Komponente den Prozess: Sie ruft Services in der richtigen Reihenfolge auf, sammelt Ergebnisse und entscheidet über die nächsten Schritte. Das macht Abläufe meist gut nachvollziehbar und erleichtert *Monitoring* und *Tests*. Nachteil ist die stärkere Abhängigkeit von dieser Instanz, wodurch ein möglicher zentraler Ausfall- oder Engpasspunkt entsteht. Dies ist in *Abbildung 7* ersichtlich. [45]

Bei der *Choreographie* gibt es keine zentrale Steuerung. Services reagieren selbstständig auf *Events* anderer Services und lösen bei Bedarf weitere *Events* aus. Dadurch ist das System stärker *entkoppelt* und oft besser *skalierbar*. Gleichzeitig verteilt sich die Prozesslogik, was Nachvollziehbarkeit, *Debugging* und Fehleranalyse erschweren kann. [45]

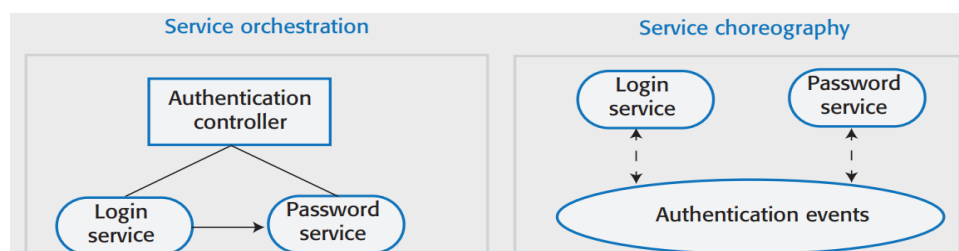


Abbildung 7: Orchestration vs. Choreography – Sommerville Microservices Architecture – F. 30

3.5.3 Datenmanagement in Microservices

Ein zentrales Prinzip der *Microservice-Architektur* ist die *Datenhoheit pro Service*. Jeder Service verwaltet sein eigenes Datenmodell und in der Regel auch eine eigene Datenbank. Dadurch wird die Kopplung zwischen Services reduziert, Releases werden unabhängiger und Änderungen an Datenstrukturen bleiben auf einen klar abgegrenzten Verantwortungsbereich beschränkt. Gleichzeitig entsteht die Herausforderung, fachliche Konsistenz über mehrere Services hinweg sicherzustellen. [43, 44, 46]

Klassische, serviceübergreifende *ACID*-Transaktionen lassen sich in verteilten Microservice-Systemen nur mit hohem Koordinationsaufwand umsetzen. Daher setzen viele Architekturen auf *eventual consistency*, bei der ein global konsistenter Zustand nicht sofort, sondern schrittweise erreicht wird. [43, 47]

Zur Unterstützung dieses Ansatzes werden etablierte Architekturprinzipien wie *Command Query Responsibility Segregation (CQRS)* und *Event Sourcing* eingesetzt. Dabei werden Schreib- und Lesezugriffe getrennt beziehungsweise Zustandsänderungen als Ereignisse modelliert. Dies verbessert Skalierbarkeit und Nachvollziehbarkeit, erhöht jedoch den Aufwand für Modellierung, Monitoring und Fehleranalyse. [43, 47, 48]

In Cloud-basierten *Software-as-a-Service (SaaS)*-Umgebungen sind zwei Datenhaltungsmodelle verbreitet: Beim *Multi-Tenant*-Modell teilen sich mehrere Mandanten eine Datenbank oder ein Schema, was Ressourcen spart, jedoch hohe Anforderungen an Isolation und Berechtigungen stellt. Das *Multi-Instance*-Modell nutzt getrennte Instanzen pro *Mandant*, bietet stärkere Isolation, verursacht jedoch höheren Betriebsaufwand. *Containerisierung* (z. B. mit *Docker*) kann diesen Mehraufwand durch standardisierte und automatisierte Bereitstellung reduzieren. [38, 41, 43, 48]

3.5.4 Frontend-Backend Kommunikation

In *Microservice-basierten* Webanwendungen erfolgt die Kommunikation zwischen *Frontend* und *Backend* über klar definierte Schnittstellen. Das Frontend ist primär für Darstellung und Benutzereingaben zuständig, während im Backend Geschäftslogik, Authentifizierung und Datenpersistenz umgesetzt werden. Diese klare Aufgabentrennung ist ein zentrales Architekturprinzip und unterstützt Wartbarkeit sowie Skalierbarkeit des Gesamtsystems. [46]

Die technische Kommunikation folgt in der Regel dem *Client-Server-Prinzip* über *HTTP/HTTPS*. Häufig wird ein *REST*-orientierter Ansatz verwendet, bei dem Funktionen und Daten als Ressourcen modelliert und über standardisierte HTTP-Methoden angesprochen werden. Als Austauschformat kommt meist *JSON* zum Einsatz, da es leichtgewichtig und plattformunabhängig ist. [23, 24, 27, 28, 49]

In Microservice-Architekturen stellt das Backend häufig eine zentrale *API*-Schicht bereit (siehe *Abbildung 8*), die als Zugriffspunkt für das Frontend dient. Dadurch bleibt das Frontend von internen Service-Strukturen entkoppelt, sodass die parallele Weiterentwicklung

verschiedener Clients (z. B. Web und Mobile) erleichtert wird. Die Absicherung dieser Schnittstellen erfolgt typischerweise über tokenbasierte Authentifizierung sowie *CORS*-Richtlinien, um unautorisierte Zugriffe zu verhindern. [23, 24, 27, 28, 43, 46, 49]

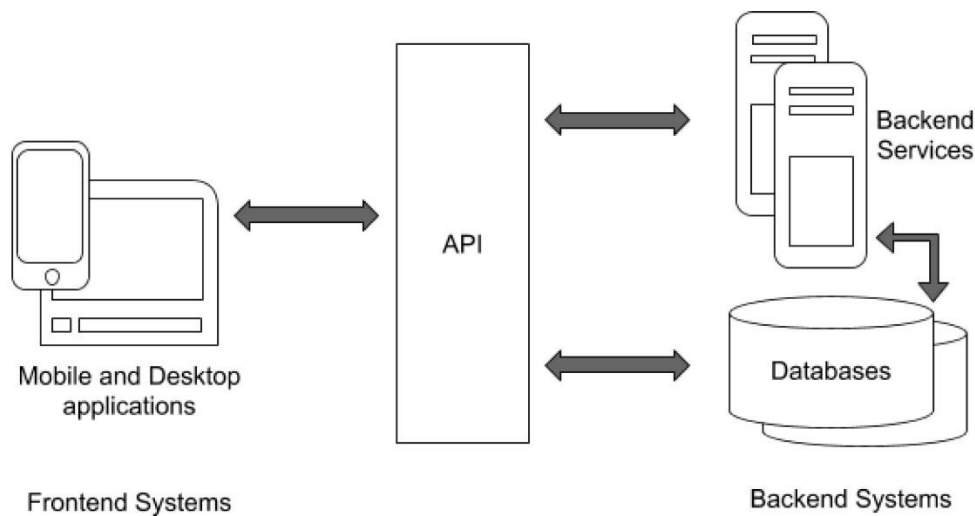


Abbildung 8: Client-Server Kommunikation [73]

3.5.5 Zusammenfassung

Die *Microservice-Architektur* zeichnet sich durch kleine, autonome Services mit klaren Verantwortlichkeiten, loser Kopplung und unabhängigem Deployment aus. Für die Zusammenarbeit der Services sind geeignete Kommunikationsformen entscheidend: Während synchrone Aufrufe eine einfache Umsetzung ermöglichen, bieten asynchrone Messaging-Ansätze eine stärkere Entkopplung und höhere Fehlertoleranz. [11]

Zur Koordination komplexer Abläufe kommen *Orchestrierung* oder *Choreographie* zum Einsatz, die jeweils unterschiedliche Stärken hinsichtlich Transparenz und Resilienz besitzen. Im Datenmanagement führt die serviceeigene Datenhoheit zu Herausforderungen bei der Konsistenz, weshalb in der Praxis häufig auf *eventual consistency* zurückgegriffen wird. Die *Frontend-Backend-Kommunikation* erfolgt typischerweise über standardisierte *HTTP(S)*-Schnittstellen (meist *REST/JSON*) und wird durch tokenbasierte Authentifizierung sowie *CORS*-Richtlinien abgesichert. Dadurch entsteht eine robuste und skalierbare Gesamtarchitektur.

3.6 Containerisierung und Docker

Die Vorteile der *Microservice-Architektur* lassen sich in der Praxis nur dann effizient umsetzen, wenn auch die zugrunde liegende Infrastruktur geeignet ist. *Containerisierung* bildet hierfür eine zentrale Basistechnologie, da sie *Isolation*, *Portabilität* und *Skalierbarkeit* unterstützt, ohne den *Overhead* vollständiger *virtueller Maschinen (VM)* zu verursachen. Dadurch können mehrere Dienste ressourcenschonend und flexibel parallel betrieben werden. [38, 39]

Container sind leichtgewichtige Laufzeitumgebungen, die Funktionen des *Host-Betriebssystems* wie *Namespaces* und *Control Groups (cgroups)* nutzen. Prozesse werden voneinander

isoliert und mit definierten Ressourcenlimits versehen, obwohl sie denselben *Kernel* verwenden. Dies ermöglicht kurze Startzeiten, einfache *Replikation* und einen effizienten Einsatz in verteilten Microservice-Systemen. [38, 39]

Ein wesentlicher Vorteil besteht darin, dass *Container* alle Abhängigkeiten eines Dienstes kapseln. Unterschiede zwischen *Entwicklungs-, Test- und Produktionsumgebungen* werden dadurch reduziert, während Services bei Bedarf schnell horizontal skaliert werden können (siehe *Abbildung 9*). [38, 39]

Wie in *Abbildung 8* dargestellt, können mehrere *Container* parallel auf demselben *Host-System* laufen. Jeder *Container* enthält dabei sowohl die Anwendung als auch die benötigten Laufzeit- und Serverkomponenten, bleibt jedoch logisch von anderen Containern getrennt. Die gemeinsame Nutzung des *Host-Betriebssystems* ermöglicht eine effiziente Ressourcenauslastung, während der *Container Manager* die Ausführung und *Isolation* der einzelnen Container koordiniert. Dadurch können mehrere unabhängige Anwendungen stabil auf derselben Hardware betrieben werden.

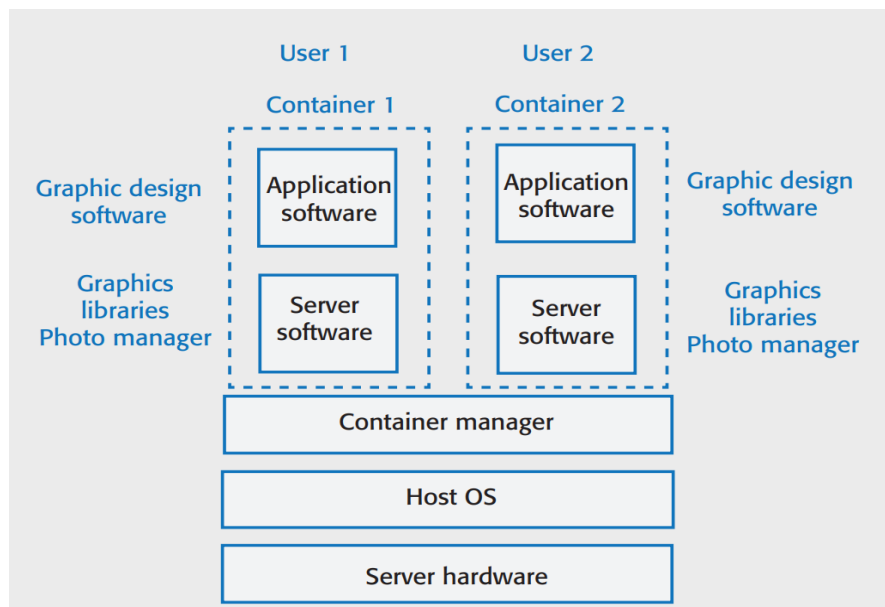


Abbildung 9: Aufbau Container - Cloud based Software Somemrville F. 8

3.6.1 Docker als Technologie der Containerisierung

Docker hat sich als verbreiteter Standard für Containerisierung etabliert und prägt die praktische Umsetzung vieler *Microservice-Architekturen*. Anwendungen werden gemeinsam mit ihren Bibliotheken und Laufzeitabhängigkeiten in *Container* verpackt, sodass sie in einer konsistenten Umgebung ausgeführt werden können. Die dafür notwendige *Isolation* und Ressourcensteuerung basiert – wie bei Containern allgemein – auf Betriebssystemmechanismen wie *Namespaces* und *Control Groups (cgroups)*. Dadurch bleibt die Laufzeitumgebung weitgehend unabhängig vom jeweiligen *Host-System*. [38, 39]

Zentrales Element des *Docker-Ökosystems* ist die *Docker Engine*. Der *Docker Daemon (dockerd)* verwaltet die Erstellung, Ausführung und Überwachung von *Containern* und stellt

hierfür eine *REST-basierte Schnittstelle* bereit. Die Trennung zwischen *Client* und *Daemon* ermöglicht sowohl lokal als auch remote gesteuerte *Deployments* und bildet eine wichtige Grundlage für automatisierte *Microservice-Umgebungen in Cloud-Infrastrukturen*. [38, 39]

Docker Images fungieren als grundlegende Verteilungseinheiten. Sie *kapseln* Anwendungen einschließlich aller Laufzeitabhängigkeiten und basieren auf einem schichtweisen Aufbau mittels *Union Filesystems*. Unveränderte Schichten können wiederverwendet werden, wodurch Speicherbedarf, Netzwerktransfer und *Update-Zeiten* reduziert werden. Die Definition eines *Images* erfolgt deklarativ in einem *Dockerfile*, was reproduzierbare *Builds* und eine hohe Nachvollziehbarkeit sicherstellt. [38, 39]

Docker Container stellen die *laufenden Instanzen dieser Images* dar. Sie nutzen den *Kernel* des *Hosts*, bleiben jedoch hinsichtlich Prozesse, Netzwerk und Dateisystem isoliert. Durch die Begrenzung von Ressourcen mittels *cgroups* entstehen kurze Startzeiten und geringe *Overheads*, was eine dynamische Skalierung von *Microservices* ermöglicht und *elastische Cloud-Architekturen* unterstützt. [38, 39]

Docker stellt zudem verschiedene Netzwerk- und Speichermechanismen bereit. Für verteilte *Microservice-Systeme* sind insbesondere *Overlay-Netzwerke* relevant, da sie eine transparente Kommunikation über mehrere *Hosts* hinweg ermöglichen. *Persistente* Daten werden außerhalb der *Container* über *Volumes* oder externe Speichersysteme verwaltet, was dem Prinzip *zustandsloser Services* entspricht. [38–41]

Die *Verteilung und Versionierung* von *Images* erfolgt über *Docker-Registries* wie *Docker Hub* oder *cloudbasierte* Dienste. Diese stellen eine zentrale Infrastrukturkomponente dar, da *Microservice-Architekturen* typischerweise eine Vielzahl *versionierter Services* umfassen (siehe *Abbildung 10*). [38, 39]

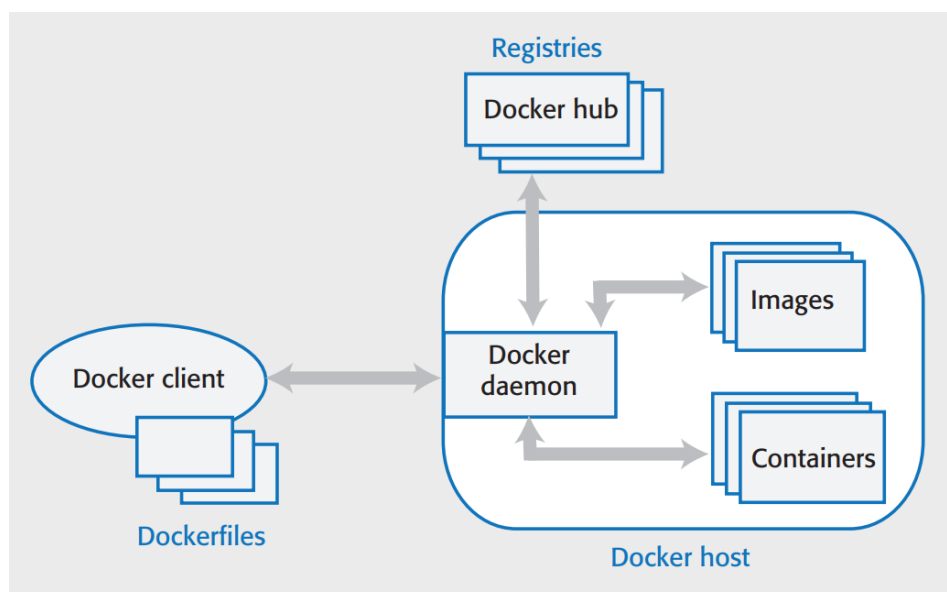


Abbildung 10: Docker Ecosystem - Cloud-based software (Sommerville) F. 11

3.6.2 Orchestrierung von containerisierten Microservices

Mit wachsender Anzahl containerisierter Services steigt der Betriebsaufwand. *Orchestrierungsplattformen* wie *Kubernetes* oder *Docker Swarm* automatisieren zentrale Aufgaben wie Start, Überwachung, Skalierung und Wiederherstellung von Containern. Anstatt einzelne Instanzen manuell zu verwalten, wird ein gewünschter Zielzustand definiert, den das System selbstständig einhält (*Self-Healing, Auto-Scaling*). [42, 44, 50, 51]

Zusätzlich stellen Orchestrierungssysteme Mechanismen für *Service Discovery*, *Load Balancing* und Netzwerkverwaltung bereit, sodass Dienste trotz dynamischer *Container-IPs* zuverlässig erreichbar bleiben. Für den produktiven Betrieb sind außerdem kontrollierte Deployment-Strategien wie *Rolling Updates* oder *Blue-Green-Deployments* wichtig, um Änderungen mit minimalen Ausfallzeiten auszurollen. [40, 44, 48, 50, 51]

3.6.3 Herausforderungen containerisierter Microservices

Trotz ihrer Vorteile erhöhen Microservices und Containerisierung die Komplexität eines Systems. Insbesondere *Observability* wird zur Schlüsselanforderung: *Monitoring*, zentrales *Logging* und verteiltes *Tracing* sind notwendig, um Fehler in verteilten Systemen nachvollziehen zu können. [38, 43, 44]

Auch Sicherheitsaspekte bleiben zentral, da Container zwar Prozesse isolieren, jedoch den Kernel des Hosts gemeinsam nutzen. Ein *restriktives* Berechtigungsmodell (z. B. *Least-Privilege*) sowie ergänzende Sicherheitsmaßnahmen sind daher erforderlich. [38, 52]

Darüber hinaus stellt das Datenmanagement eine dauerhafte Herausforderung dar. Da Microservices idealerweise zustandslos betrieben werden, müssen persistente Daten außerhalb der Container verwaltet werden. Gleichzeitig erfordern verteilte Systeme häufig Konsistenzmechanismen jenseits klassischer *ACID*-Transaktionen, etwa durch *eventual consistency* und kompensierende Abläufe. [41, 42, 45, 53]

3.6.4 Zusammenfassung

Containerisierung bildet eine zentrale Grundlage für *Microservice-Architekturen*, da sie einen isolierten, portablen und skalierbaren Betrieb bei geringem Ressourcenverbrauch ermöglicht. *Docker* unterstützt dies durch ein etabliertes Ökosystem aus *Engine*, *Images* und *Containern*, während *Orchestrierungsplattformen* wie *Kubernetes* oder *Docker Swarm* Skalierung, Service-Verfügbarkeit und Deployments automatisieren. Gleichzeitig steigen die Anforderungen an *Observability*, Sicherheit und Datenmanagement. Insgesamt ermöglichen *Containerisierung* und *Orchestrierung* jedoch einen flexiblen und robusten Betrieb moderner, verteilter Anwendungssysteme.

3.7 Cloud Computing und Cloud-Plattformen (Ellmer)

Die in den Abschnitten zur *Microservice-Architektur* und *Containerisierung* beschriebenen Konzepte entfalten ihre Vorteile besonders dann, wenn sie auf *Cloud-Plattformen* betrieben werden. *Cloud-Infrastrukturen* stellen Rechenleistung, Speicher und Netzwerk als abstrahierte Ressourcen bereit, die bedarfsgerecht zugewiesen und bei Laständerungen elastisch skaliert werden können. Dadurch eignen sie sich als stabile Grundlage für den Betrieb *verteilter, containerisierter Anwendungen*. [54]

Im Folgenden werden zunächst zentrale Grundlagen des *Cloud Computing* erläutert. Anschließend erfolgt eine konzeptionelle Einordnung von *Amazon Web Services (AWS)* und *Microsoft Azure* sowie eine Betrachtung alternativer *Hosting-Modelle*.

3.7.1 Grundlagen des Cloud Computing

Cloud Computing bezeichnet die Bereitstellung von IT-Ressourcen und IT-Diensten über das Internet nach dem On-Demand-Prinzip. Dazu zählen grundlegende Infrastrukturkomponenten (z. B. Rechenleistung, Speicher und Netzwerke) ebenso wie höherwertige Plattform- und Anwendungsdienste. Charakteristisch sind eine flexible, bedarfsgerechte Nutzung sowie eine nutzungsabhängige Abrechnung, wodurch der Betrieb eigener Hardware in vielen Szenarien entfallen kann. [48, 54]

Cloud-Systeme sind großskalierte, virtualisierte Rechenzentren, in denen zahlreiche Instanzen auf gemeinsamer physischer Infrastruktur betrieben werden [54]. Eine wesentliche technische Grundlage ist die Virtualisierung: Ein Hypervisor abstrahiert physische Ressourcen und weist diese isolierten Instanzen zu, die aus Sicht der Anwender wie eigenständige Server mit eigenem Betriebssystem und Software-Stack erscheinen. [48, 54]

Für den Einsatz in verteilten Architekturen sind insbesondere folgende Eigenschaften wesentlich:

- *Elastizität*: Ressourcen werden automatisch je nach Last hoch- oder heruntergefahren, so dass Leistungsspitzen abgefedert werden. [48, 54]
- *Skalierbarkeit*: Erweiterung bei Bedarf durch mehr Instanzen (*Scale-out*) oder stärkere Hardware (*Scale-up*). [48, 54]
- *Resilienz*: Höhere Ausfallsicherheit durch *Redundanz* und geografische Verteilung, damit ein Ausfall nicht den gesamten Dienst stoppt. [48, 54]
- *Messbarkeit*: Verbrauch von Rechenleistung, Speicher und Netzwerk wird transparent erfasst – Grundlage für Monitoring und nutzungsbasierte Abrechnung. [48, 54]

Diese Eigenschaften sind für Microservice-Architekturen besonders relevant, da einzelne Services häufig unterschiedliche Lastprofile aufweisen und daher unabhängig voneinander skaliert und betrieben werden sollen. [54, 55]

Cloud-Angebote werden üblicherweise in die Servicemodelle *Infrastructure as a Service (IaaS)*, *Platform as a Service (PaaS)* und *Software as a Service (SaaS)* unterteilt. Der zentrale Unterschied liegt dabei weniger in der eingesetzten Technik als in der Verantwortungsverteilung: Je nach Modell übernimmt entweder der Anbieter oder der Betreiber Aufgaben wie Betrieb, Skalierung, Absicherung und Monitoring. [54]

IaaS stellt grundlegende Infrastruktur wie *virtuelle Server, Netzwerk und Speicher* bereit. Dies bietet hohe *Flexibilität* und *Kontrolle* über *Betriebssystem* und Konfiguration, erfordert jedoch einen vergleichsweise hohen Betriebsaufwand für Wartung, Sicherheit und Überwachung. [54]

PaaS erweitert die Infrastruktur um *typische Betriebsfunktionen* wie *Deployment, Skalierung* und *Health-Checks*. Der Fokus liegt stärker auf der Anwendung selbst, während der Plattformanbieter den Standardbetrieb übernimmt. Dies reduziert den Wartungsaufwand, schränkt jedoch die technische Flexibilität durch plattformspezifische Vorgaben ein. [54]

SaaS stellt fertige Anwendungen als Dienst bereit. *Betrieb, Updates und Sicherheitsmaßnahmen* liegen weitgehend beim Anbieter, während Nutzer die Anwendung lediglich konfigurieren und über Weboberflächen oder APIs verwenden. Dadurch sinkt der technische Aufwand, allerdings auf Kosten individueller Anpassungsmöglichkeiten. [54]

Gerade im Kontext von *Microservice-Architekturen* ist diese Einordnung hilfreich, da Services je nach Lastprofil und Betriebsanforderungen unterschiedlich gut zu den jeweiligen Modellen passen. Unabhängig vom gewählten Ansatz bilden *Elastizität, Skalierbarkeit, Resilienz* und *Messbarkeit* den gemeinsamen Rahmen von *Cloud-Plattformen*. [48, 54]

3.7.2 AWS vs. Microsoft Azure

Amazon Web Services (AWS) und *Microsoft Azure* zählen zu den führenden Public-Cloud-Plattformen und werden häufig für den Betrieb verteilter und containerisierter Anwendungen eingesetzt. Beide Anbieter stellen Rechenleistung, Speicher und Netzwerk als standardisierte Dienste bereit, die bedarfsgerecht skaliert und nutzungsbasiert abgerechnet werden. Damit unterstützen sie zentrale Cloud-Eigenschaften wie Elastizität, Skalierbarkeit und Messbarkeit. [54]

Zur Orientierung bei Architekturentscheidungen bieten beide Plattformen sogenannte *Well-Architected Frameworks*. Diese bündeln Best Practices und bewerten Cloud-Architekturen anhand von Qualitätszielen wie Zuverlässigkeit, Sicherheit, Performance-Effizienz und Kostenoptimierung. Sie dienen als strukturierter Leitfaden, ohne konkrete Implementierungen vorzugeben. [56–59]

Für *Microservice-Architekturen* ist zudem relevant, dass AWS und Azure zahlreiche Betriebsfunktionen wie Skalierung, Monitoring und die Integration verwalteter Dienste bereitstellen. Dadurch wird der Übergang von klassischem Serverbetrieb zu serviceorientierten Cloud-Modellen erleichtert. [44, 57, 60]

3.7.3 Cloud-Hosting-Alternativen

Trotz ihres breiten Funktionsumfangs können *Public-Cloud-Plattformen* – insbesondere bei kleineren Projekten – durch Kosten- und Komplexitätsaspekte herausfordernd sein. Ursache ist vor allem das Prinzip der nutzungsbasierten Bereitstellung und Abrechnung. [54]

Vor diesem Hintergrund werden im Folgenden drei Alternativen betrachtet: ein klassischer *vServer/VPS* bei einem nationalen Anbieter (*World4You*) sowie eine entwicklerorientierte, *PaaS-nahe Plattform* (*Render*). Ziel ist eine Einordnung hinsichtlich Kontrolle, Betriebsaufwand und Kostenrahmen. [7, 61]

3.7.4 World4You als nationaler Hosting-Anbieter

World4You ist ein österreichischer Hosting-Provider mit Rechenzentren in Österreich und einem Angebot von Webhosting bis zu *vServer/VPS-Lösungen*. [7, 8] Für containerbasierte Setups sind insbesondere *Linux-vServer* relevant, weil sie typischerweise die nötige administrative Kontrolle bieten, um eine *Container-Runtime* wie Docker zu betreiben. [7, 8, 39]

Praktisch bedeutet das: Die Architektur ist weitgehend frei gestaltbar (z. B. getrennte Container für Backend, Datenbank und weitere Dienste). Gleichzeitig liegt der laufende Betrieb stärker in der eigenen Verantwortung. Insbesondere Updates, Backups sowie Monitoring und *Logging* müssen aktiv geplant und umgesetzt werden, damit Sicherheit und Verfügbarkeit im Produktivbetrieb gewährleistet bleiben. [7, 8, 52]

3.7.5 Render als Cloud-Plattform

Render verfolgt ein *PaaS-nahes Modell* mit Fokus auf *Developer Experience*. Anstatt virtuelle Maschinen direkt zu verwalten, werden Services über vordefinierte Service-Typen bereitgestellt, *Deployments* können Docker-basiert aus *Repositories* erfolgen und lassen sich gut in Git-Workflows integrieren. [61, 62]

Für datenbankgestützte Anwendungen ist außerdem relevant, dass *Render Managed PostgreSQL* anbietet und damit zentrale Betriebsaufgaben (je nach Service-Stufe) stärker standardisiert. Ebenso werden üblicherweise Funktionen wie *Custom Domains* und *TLS-Absicherung* unterstützt. Der Vorteil liegt vor allem im geringeren Betriebsaufwand, als Gegenstück dazu müssen Plattformlimits, Preisstufen und Rahmenbedingungen in die technische und wirtschaftliche Planung einbezogen werden. [61, 62]

3.7.6 Netlify als Cloud Plattform

Netlify ist eine *PaaS-nahe Cloud-Plattform* mit starkem Fokus auf die Bereitstellung von Web-Frontends. Anwendungen werden in der Regel direkt aus *Git-Repositories* gebaut und automatisch deployt, wodurch sich *Build- und Release-Prozesse* gut in bestehende Entwicklungsworkflows integrieren lassen. Die Auslieferung erfolgt über ein integriertes *Content Delivery Network (CDN)*, was kurze Ladezeiten und eine hohe Verfügbarkeit unterstützen. [69-72]

Zusätzlich bietet *Netlify* Funktionen wie *Custom Domains*, automatische *TLS*-Zertifikate sowie *serverlose Funktionen* für einfache *Backend-Logik*. Für komplexe, datenbankgestützte Backends ist die Plattform jedoch weniger geeignet und wird daher häufig in Kombination mit externen Backend-Diensten eingesetzt. Der Vorteil von *Netlify* liegt vor allem im geringen Betriebsaufwand für Web-Frontends, während Einschränkungen hinsichtlich Backend-Funktionalität und Plattformabhängigkeit bei der Architekturplanung berücksichtigt werden müssen. [69-72]

3.7.7 Cloud Anbieter im direkten Vergleich

Cloud Computing ist eine zentrale Grundlage für den Betrieb moderner Microservice-Systeme, weil Ressourcen flexibel bereitgestellt, skaliert und gemessen werden können. [54] *AWS* und *Azure* bieten hierfür ausgereifte Plattformmechanismen und geben mit ihren *Well-Architected-Leitlinien* einen strukturierten Rahmen, um Architekturentscheidungen an Qualitätszielen auszurichten. [56–59] *World4You* und *Render* zeigen, dass ähnliche Zielbilder auch außerhalb großer *Hyperscaler* realisierbar sind: Ein *vServer* bietet hohe Kontrolle, erfordert aber mehr Eigenbetrieb, eine PaaS-nahe Plattform reduziert den operativen Aufwand, bringt dafür stärkere Plattformabhängigkeiten mit. [7, 8, 61, 62]

3.7.8 Zusammenfassung

AWS und *Microsoft Azure* sind typische Vertreter großer *Public-Cloud-Plattformen*. Sie stellen weltweit verteilte Infrastruktur und standardisierte Dienste bereit, wodurch Skalierung, Verfügbarkeit und Betriebsautomatisierung weitgehend integriert sind. Zur Orientierung bei Architekturentscheidungen bieten beide Anbieter *Well-Architected Frameworks*, die Qualitätsziele wie Zuverlässigkeit, Sicherheit und Kosten adressieren. Als praxisnahe Alternativen zeigen *World4You*, *Render* und *Netlify*, dass Hosting auch außerhalb großer *Hyperscaler* sinnvoll umgesetzt werden kann. *World4You* (*vServer*) bietet hohe Kontrolle und flexible *Docker*-Setups, erfordert jedoch einen erhöhten Eigenbetrieb, etwa bei Updates, Backups sowie Monitoring. *Render* reduziert diesen Aufwand durch standardisierte Deployments und *Managed PostgreSQL*, während Plattformlimits berücksichtigt werden müssen. *Netlify* richtet sich primär an die Bereitstellung von Web-Frontends und ermöglicht automatisierte Deployments mit geringem Betriebsaufwand, ist jedoch für komplexe Backend-Szenarien nur eingeschränkt geeignet.

3.8 Einsatz sinnvoller *Libraries* und Technologien

Moderne Webanwendungen können heute Dinge, die früher fast nur mit Desktop-Programmen möglich waren – zum Beispiel *PDFs erzeugen*, *QR- und Barcodes* scannen oder Inhalte per *Drag-and-Drop* sortieren. Für solche Funktionen gibt es ausgereifte *JavaScript-Tools*, die direkt im Browser laufen. Diese Technologien werden auch in dem Bestellservice verwendet, um Abläufe und Funktionalitäten zu verbessern. Trotzdem müssen dabei praktische Grenzen wie

Browserrechte (z. B. Kamerazugriff) und Performance auf mobilen Geräten mitgedacht werden. [63–68]

In den folgenden Abschnitten werden die in diesem Projekt verwendeten *Bibliotheken (Libraries)* sowie einige sicherheitsrelevante Techniken beschrieben. Ziel ist es, ihre Funktionsweise in dem Bestellservices nachvollziehbar zu machen.

3.8.1 PDF-Generierung mit jsPDF *(Ellmer)*

Für die Erstellung von *PDF-Dokumenten (Portable Dokument Format)* im Browser wird *jsPDF* eingesetzt. Die Bibliothek ermöglicht, Dokumente clientseitig zu generieren und als Datei bereitzustellen, ohne dass ein serverseitiger *PDF-Generator* zwingend erforderlich ist. Im Bestellservice ist dies insbesondere für Exportfunktionen relevant, bei denen strukturierte Daten (z. B. Bestell- oder Übersichtslisten) bereits im Frontend vorliegen und in ein *transportier- bzw. archivierbares Format* überführt werden sollen. [63, 64]

Technisch arbeitet *jsPDF* dokumentorientiert: Ein *PDF* wird über ein *API* schrittweise aufgebaut, wobei Inhalte über ein Koordinatensystem positioniert werden. Dadurch ist die Ausgabe kontrollierbar und reproduzierbar, erfordert aber zugleich eine bewusste Modellierung des Layouts (z. B. Seitenumbrüche, Zeilenlängen, Skalierung). Im Projektkontext ist das vorteilhaft, weil Exporte nicht „Screenshot-artig“ von dem *User Interface (UI)* abhängen, sondern als eigenständige, konsistente Ausgabe erzeugt werden können. [63, 64]

3.8.2 QR-Code-Erkennung mit html5-qrcode

Für die clientseitige Erkennung von *QR-Codes* und *Barcodes* wird eine browserbasierte Scan-Bibliothek (*html5-qrcode*) eingesetzt, die *Live-Scans* über die Gerätekamera unterstützt. Dadurch lässt sich der Scanvorgang direkt in den *Webclient* integrieren, was den Zugang vereinfacht, da keine separate App installiert werden muss, und insbesondere für mobile Nutzungsszenarien geeignet ist. [66]

Technisch basiert der Scanprozess auf dem Zugriff auf einen *Videostream* sowie der fortlaufenden Auswertung einzelner *Frames*, bis ein Code erkannt und dekodiert werden kann. Für Stabilität und Performance sind Parameter wie *Scanbereich* und *Abtastrate* entscheidend, da sie den Ressourcenverbrauch und die Erkennungsrate je nach Endgerät maßgeblich beeinflussen. Zusätzlich ist der Scanvorgang an *Browser-Berechtigungen* gebunden, was eine transparente und nachvollziehbare Anforderung sowie Nutzung der Kamera voraussetzt. [66]

3.8.3 Bedienung mit Angular CDK Drag-and-Drop *(Magos)*

Um die Menüplanung im Bestellservice intuitiv und effizient zu gestalten, kommt das *Drag-and-Drop*-Modul des *Angular Component Development Kits (CDK)* zum Einsatz. Es ermöglicht, Elemente per Maus- oder *Touch-Eingabe* zwischen Listen zu verschieben, innerhalb von Listen neu anzuordnen oder zu entfernen. [65]

Ein wesentlicher Vorteil liegt in der engen Integration in *Angular*: Benutzerinteraktionen werden konsistent auf die zugrunde liegenden Datenstrukturen übertragen. Dadurch entsteht ein stabiler und nachvollziehbarer Datenzustand, während typische Eingabefehler reduziert werden, die bei rein formularbasierten Ansätzen häufiger auftreten. Im Projektkontext unterstützt dies insbesondere die flexible Zusammenstellung und Anpassung von Menüs durch berechnete Nutzer. [65]

3.8.4 Verifikation und Authentifizierung via E-Mail (Ellmer)

Die *E-Mail-Adresse* dient in vielen *Webanwendungen* sowohl als *Identifikator* als auch als *Kommunikationskanal*. Dieses Verfahren kommt auch in dieser Diplomarbeit zum Einsatz. Ohne *Verifikation* besteht das Risiko, dass Konten mit fremden oder fehlerhaften Adressen registriert werden, wodurch die *Datenqualität* sinkt und *Missbrauch* erleichtert wird. [21]

Ein verbreiteter Ansatz ist das *Double-Opt-In-Verfahren*: Nach der Registrierung wird ein Konto zunächst eingeschränkt angelegt; erst der Aufruf eines *Verifikationslinks* aktiviert es vollständig. Damit wird der Besitz der *E-Mail-Adresse* bestätigt und Missbrauch erschwert. Für die Schutzwirkung ist wesentlich, dass sicherheitsrelevante Funktionen (z. B. *Login* oder Zugriff auf *geschützte Ressourcen*) erst nach erfolgreicher Verifikation freigeschaltet werden. [21]

Technisch basiert die Verifikation auf *Tokens*, die kryptografisch zufällig erzeugt, zeitlich begrenzt und nur einmal verwendbar sein müssen. *OWASP* betont hierbei insbesondere klare *Ablaufzeiten* sowie die konsequente *Entwertung des Tokens* nach der Nutzung, um *Brute-Force*- und *Replay*-Szenarien zu reduzieren. [24] Das *NIST* ordnet *Authentifizierungs- und Identitätsmechanismen* zudem als zentrale Grundlage für sichere *Zugriffskontrollen* in *Websystemen* ein. [20, 21]

Beim Zurücksetzen von *Passwörtern* ist außerdem darauf zu achten, dass die Anwendung nicht preisgibt, ob zu einer *E-Mail-Adresse* ein Konto existiert. Andernfalls könnten Angreifer gezielt prüfen, welche Adressen registriert sind. Antworten auf *Reset-Anfragen* sollten daher stets neutral und identisch formuliert sein, unabhängig davon, ob die Adresse im System vorhanden ist oder nicht. [26]

3.8.5 Zuverlässiger E-Mail-Versand *(Ellmer)*

Der Versand von *Verifikations-* und *Passwort-Reset-E-Mails* kann über *SMTP (Simple Mail Transfer Protocol)* umgesetzt werden. *SMTP* ist asynchron konzipiert, sodass Nachrichten bei kurzfristigen Zustellproblemen zwischengespeichert und später erneut zugestellt werden können, was die *Zuverlässigkeit* erhöht. [68]

In der Praxis können jedoch organisatorische oder technische Rahmenbedingungen den *SMTP*-Versand einschränken, etwa in *Schul-* oder *Unternehmensnetzwerken*. So ist *SMTP*-Versand auf in unserem Schulnetzwerk beschränkt. Daher wird auch in diesem Projekt auf robuste Alternativen mit *HTTPS*-basiertem Versand, beispielsweise über die *Gmail API*, ausgewichen. Nur so kann ein dauerhaft stabiler *E-Mail*-Versand sichergestellt werden.

Im Gegensatz zu *SMTP* erfolgt die Autorisierung bei der *Gmail API* über *OAuth 2.0*. Dabei werden Versandrechte *tokenbasiert* vergeben, ohne dass das Anwendungssystem *Benutzerpasswörter* speichern muss. *OAuth 2.0* gilt als etablierter Standard für sichere Autorisierungsverfahren und unterstützt eine kontrollierte *Token*-Vergabe. [67]

3.8.6 Zusammenfassung

Die im Projekt eingesetzten *Libraries* erweitern die Webanwendung um zentrale, praxisrelevante Funktionen: *jsPDF* ermöglicht reproduzierbare und kontrollierte *PDF*-Exporte, browserbasiertes *QR-/* und *Barcode*-Scanning unterstützt scanbasierte Abläufe direkt im *Webclient*, und *Angular CDK Drag-and-Drop* verbessert die Menüplanung durch eine konsistente und effiziente Bearbeitung von Listen- und Strukturänderungen.

Sicherheitstechnisch stellt die *E-Mail*-Verifikation einen wichtigen Baustein zur Absicherung der *Benutzeridentität* und zur Verbesserung der allgemeinen *Datenqualität* dar. Sie wird über zeitlich begrenzte, einmal verwendbare *Tokens* umgesetzt. Beim *Passwort-Reset* ist zusätzlich darauf zu achten, dass aus Systemantworten keine Rückschlüsse auf die Existenz einzelner Konten gezogen werden können.

Der *E-Mail*-Versand kann je nach Infrastruktur entweder über *SMTP* oder über *API*-basierte Verfahren, etwa mittels *Gmail API*, erfolgen. Ergänzend trägt eine präzise konfigurierte *CORS-Policy* zur Absicherung der Schnittstellen bei und verhindert unbefugte Zugriffe aus nicht autorisierten Quellen.

4 Umsetzung

In diesem Kapitel wird zunächst die Grundstruktur des Systems nachvollziehbar dargestellt und in ihre zentralen Bausteine gegliedert. Ziel ist es, die einzelnen Systembestandteile, deren Aufgaben sowie das Zusammenspiel *zwischen Frontend, Backend und Datenbank* klar und nachvollziehbar darzustellen. Dazu werden *Komponentendiagramme, Architektordiagramme* und *C4-Diagramme* eingesetzt, um das System, je nach Betrachtungsebene sowohl übersichtlich als auch technisch präzise zu beschreiben. Die dazu verwendeten Diagramme wurden mittels *PlantUML* erstellt. Danach wird in den Unterkapiteln Frontend-Entwicklung, Backend-Entwicklung und Hosting detailliert beschrieben, wie das System technisch funktioniert und entwickelt wurde. Das Design und die entsprechenden *.css-Files* wurde mithilfe von KI erstellt. Auch im Zuge der Entwicklung der Systemlogik wurde in manchen Bereichen, vor allem für das eigene bessere Verständnis neuer Technologien und der Fehlerbehebung künstliche Intelligenz eingesetzt.

4.1 Grundstruktur des Projekts

4.1.1 Systemüberblick

Die Anwendung folgt einer klar getrennten *Client-Server-Architektur*. Auf oberster Ebene besteht das System aus einem *Angular-basierten Frontend*, einem *serverseitigen Backend* auf Basis von *NestJS* sowie einer relationalen *PostgreSQL-Datenbank* zur persistenten Speicherung der Anwendungsdaten.

Nutzer greifen über einen Webbrowser auf das Frontend zu, welches ausschließlich über *REST-basierte HTTP-/HTTPS-Schnittstellen* mit dem Backend kommuniziert. Das Backend übernimmt die Verarbeitung der Geschäftslogik, steuert den Zugriff auf die Datenbank und implementiert sicherheitsrelevante Mechanismen wie *die JWT-basierte Authentifizierung und Autorisierung*. Ergänzend wird *CORS* eingesetzt, um den Zugriff auf die API auf definierte Ursprünge zu beschränken.

Zusätzlich ist ein externer Dienst für den E-Mail-Versand angebunden. Das Backend nutzt hierfür *die Gmail API (Gmail)*, um systemrelevante E-Mails, etwa im Rahmen der Registrierung oder Profilverwaltung, automatisiert zu versenden.

Der gesamte Ablauf ist in folgendem Diagramm (*Abbildung 11*) ersichtlich.

Systemkontext - Bestell-/Menüsystem (UML)

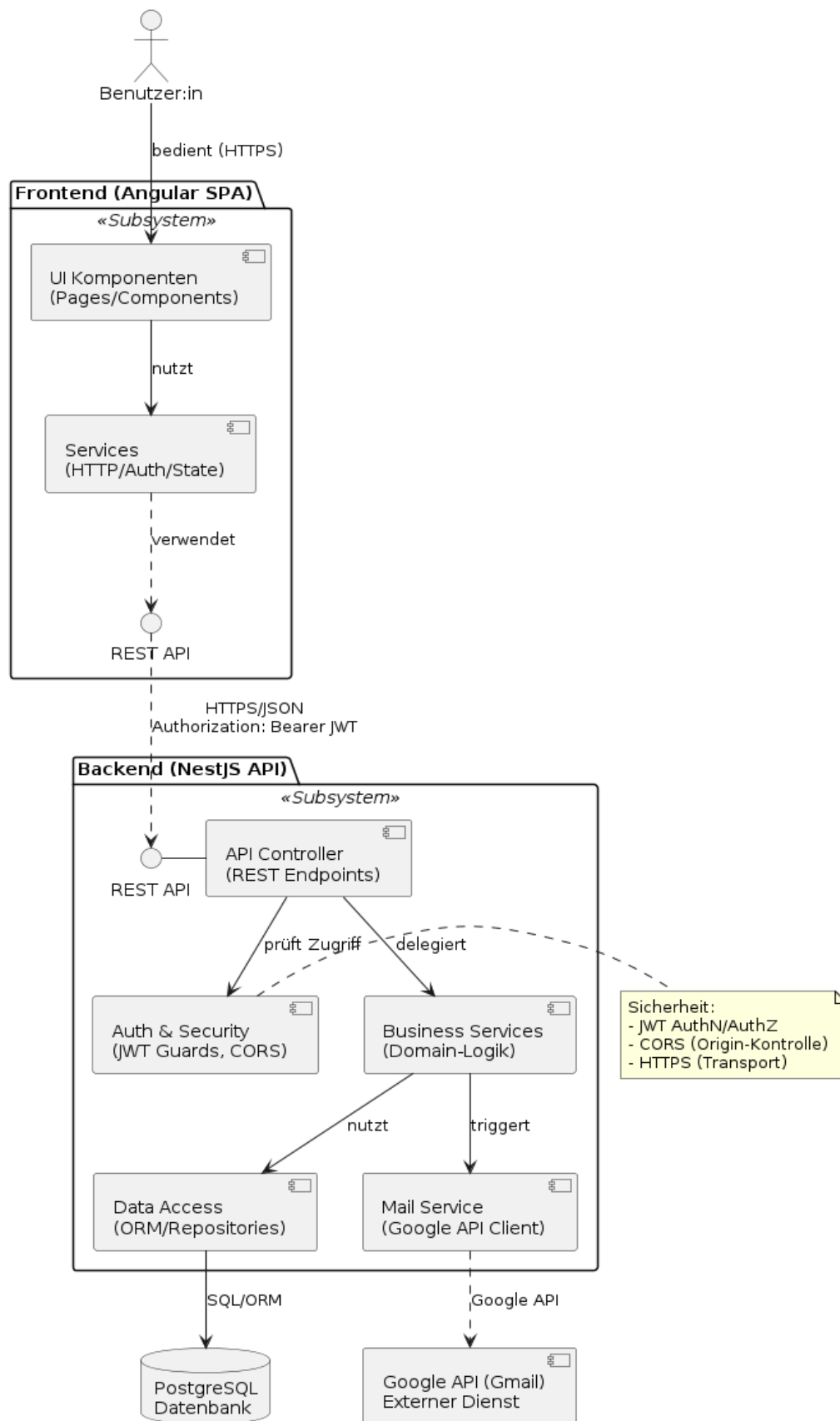


Abbildung 11: Kontextdiagramm – C4 Diagramm

4.1.2 Architekturübersicht

Die Architekturübersicht beschreibt die Laufzeitumgebung der Anwendung. Das *Angular-Frontend* wird direkt im Browser der Nutzer ausgeführt, während das *NestJS-Backend* innerhalb einer *Node.js-Runtime* auf einem Server betrieben wird. Die *PostgreSQL-Datenbank* ist auf einem separaten Datenbankserver untergebracht und dient der dauerhaften Speicherung aller relevanten Anwendungsdaten.

Der Versand von Bestätigungs-E-Mails für Benutzerfunktionen, wie Registrierung oder Passwort-Zurücksetzung erfolgt serverseitig über die *Gmail API*. Diese Darstellung macht die realen Systemgrenzen sowie die Kommunikationswege zwischen den einzelnen Komponenten deutlich, insbesondere die *HTTPS-basierte Kommunikation* zwischen Frontend und Backend, den Datenbankzugriff sowie die Anbindung externer Dienste (*Abbildung 12*).

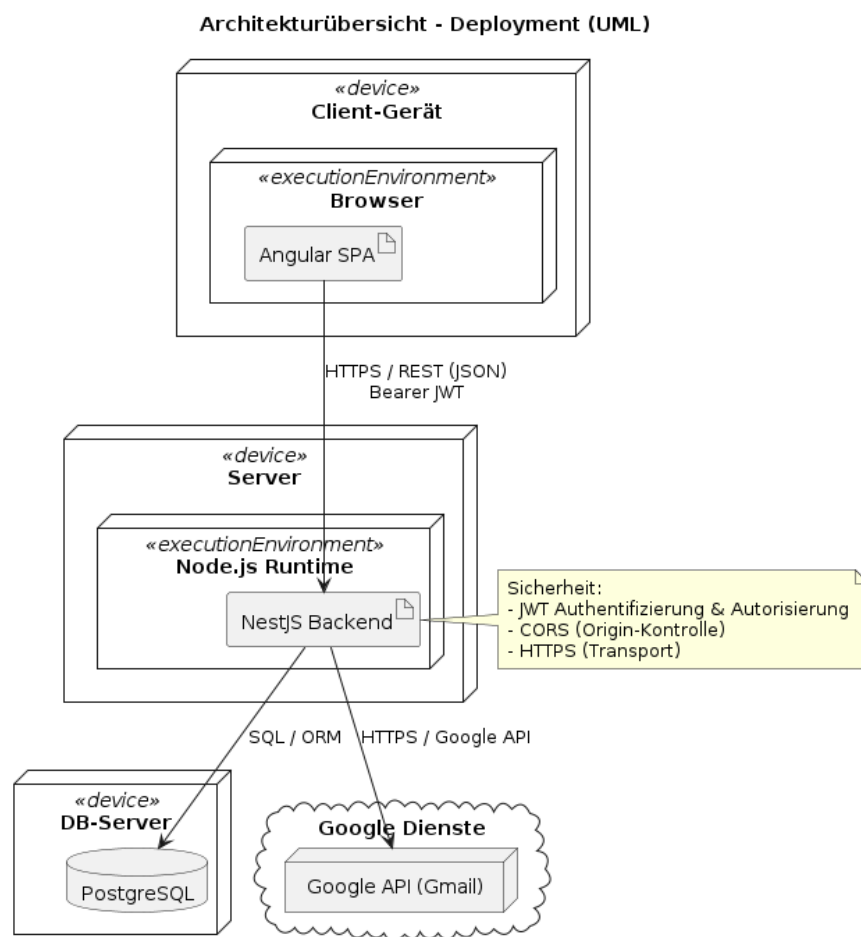


Abbildung 12: Architekturdiagramm

4.1.3 Container-Ansicht

In der *Container-Ansicht* werden die zentralen Bausteine der Anwendung so dargestellt, wie sie im laufenden Betrieb zusammenarbeiten. Im Fokus stehen dabei das *Frontend*, das *Backend*, die *Datenbank* sowie ein externer Dienst für den *E-Mail-Versand*.

Diese Darstellung bildet eine sinnvolle Brücke zwischen der *Systemkontext-Sicht* und der detaillierten *Komponentenebene*. Sie zeigt klar, welche *Container* miteinander kommunizieren und über welche Schnittstellen dies geschieht, ohne bereits auf interne Module oder Implementierungsdetails einzugehen. Der Schwerpunkt liegt dabei auf den Kommunikationsbeziehungen nach dem Prinzip „wer spricht mit wem“. Die Container-Ansicht ist in *Abbildung 13* zu sehen.

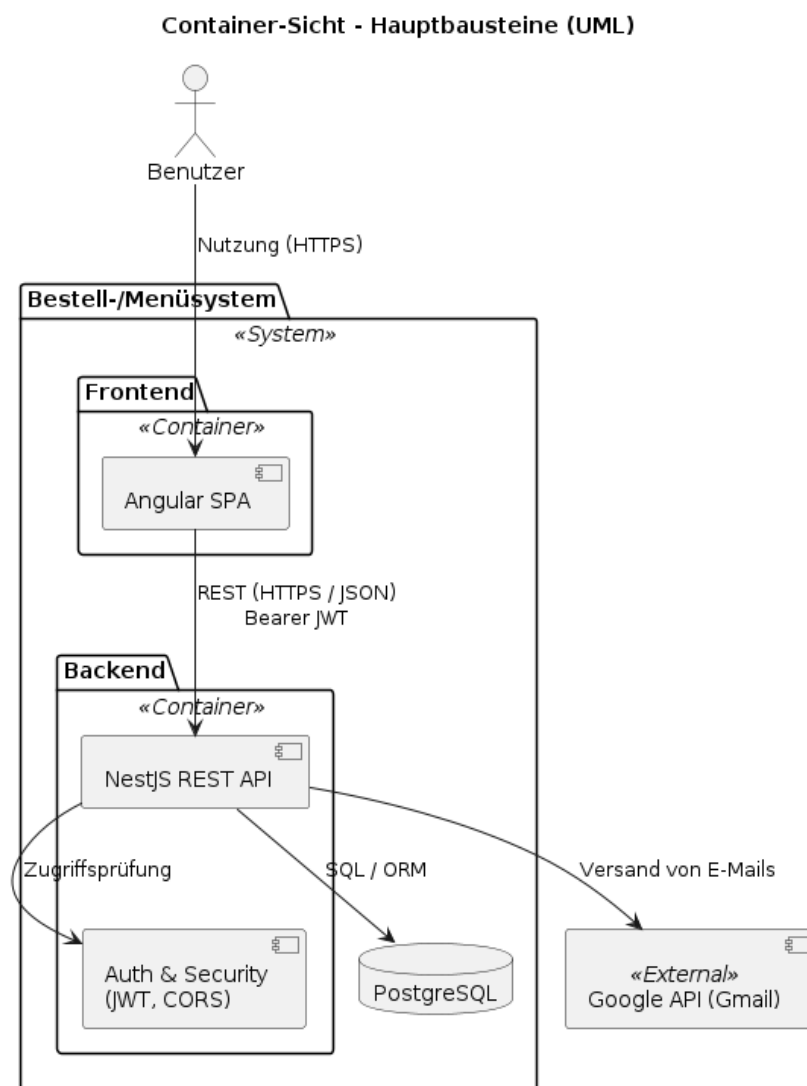


Abbildung 13: Containerdiagramm - C4-Diagramm

4.1.4 Interne Struktur des Backend

Das Backend stellt den zentralen Knotenpunkt des Systems dar. Es stellt die *REST-API* bereit, erzwingt Sicherheitsregeln, enthält die Geschäftslogik, greift auf die Datenbank zu und bindet externe Services wie den E-Mail-Versand an (siehe *Abbildung 14*).

Zur Verbesserung der Wartbarkeit und zur klaren Trennung von Verantwortlichkeiten ist das Backend in mehrere logisch abgegrenzten Komponenten gegliedert. Dazu zählen die *Controller-Schicht*, *Authentifizierungs- und Sicherheitskomponenten*, *fachliche Services (Domain Services)*, eine *Datenzugriffsschicht (Data Access)* sowie ein separater Mail-Service.

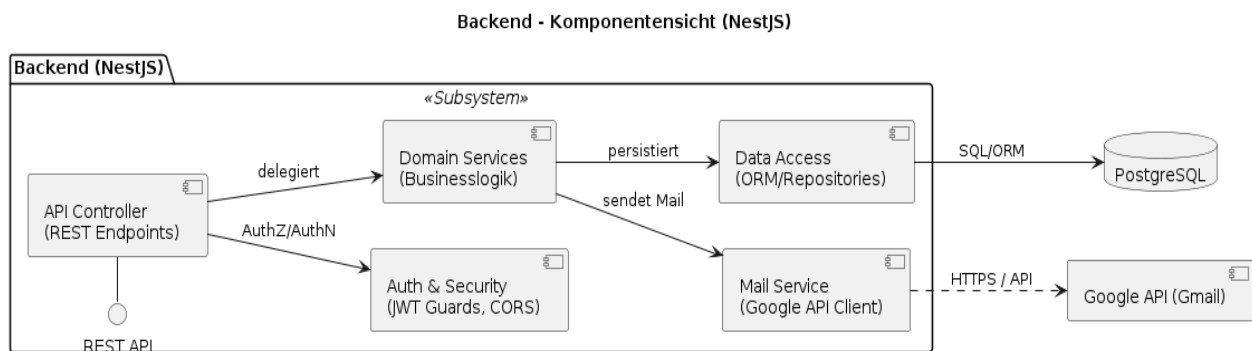


Abbildung 14: Komponentenansicht - Backend

4.1.5 Sicherheit und Authentifizierung

Ein zentraler Ablauf der Anwendung ist der Login sowie der anschließende Zugriff auf geschützte Funktionen. Nach der Anmeldung prüft das Backend die Zugangsdaten und stellt bei erfolgreichem Login ein *JSON Web Token (JWT)* aus. Dieses Token dient als Nachweis der Authentifizierung.

Bei allen weiteren *API-Aufrufen* sendet das Frontend das *JWT im Authorization-Header* mit. Das Backend kann dadurch jede Anfrage eindeutig einem Benutzer zuordnen, die Gültigkeit des *Tokens* prüfen und anhand der enthaltenen Informationen entscheiden, ob der Zugriff auf geschützte *Endpunkte* erlaubt ist. Der Login-Ablauf ist als Sequenzdiagramm in *Abbildung 15* dargestellt. Auf diese Weise wird sichergestellt, dass sensible Funktionen und Daten nur autorisierten Benutzern zur Verfügung stehen.

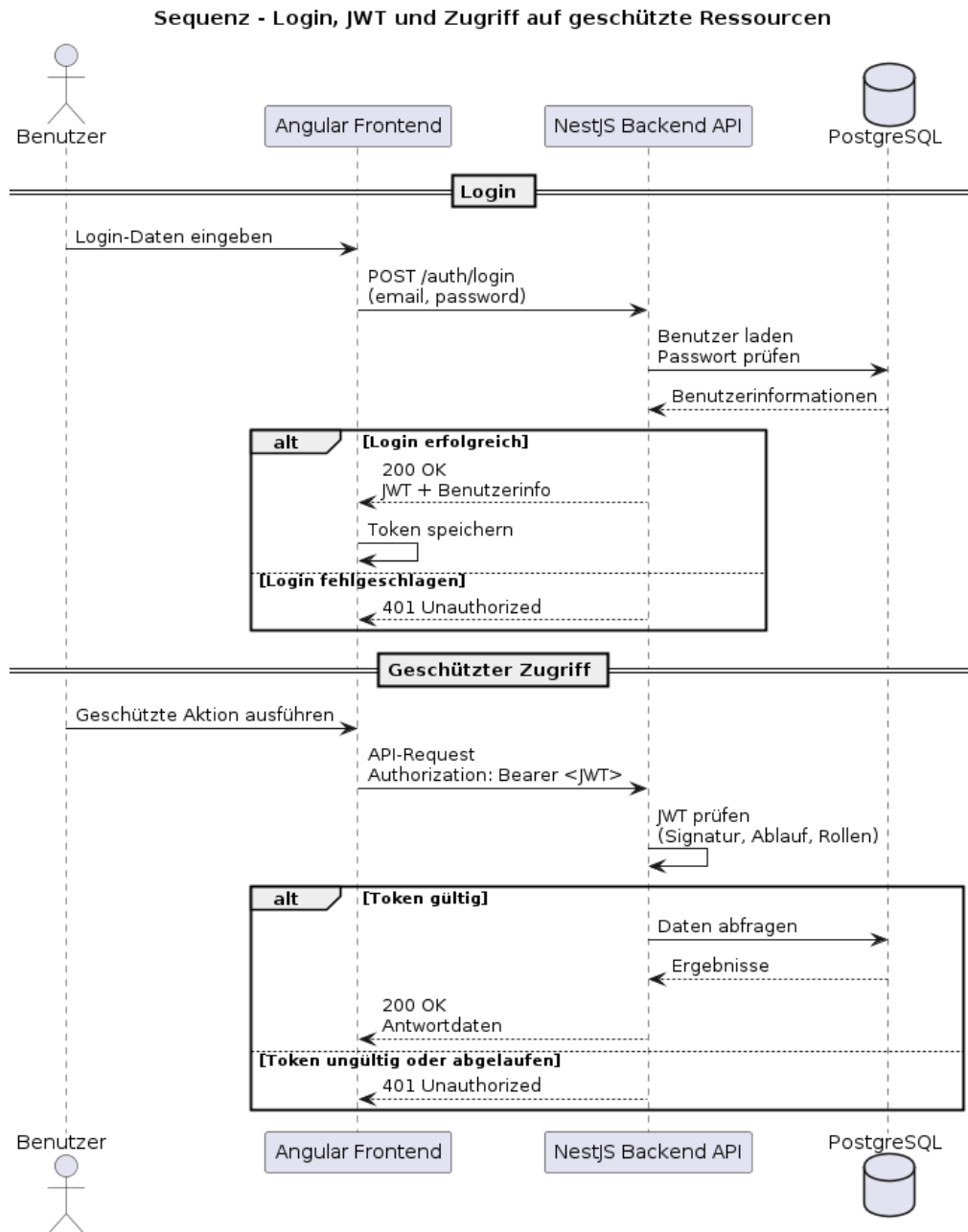


Abbildung 15: Sequenzdiagramm: Login

4.1.6 Funktionale Ansicht – Use-Cases

Das Use-Case-Diagramm gibt einen fachlichen Überblick über die wichtigsten Funktionen des Systems und zeigt, welche Rollen diese Funktionen nutzen können. Es macht auf einen Blick deutlich, welche Anwendungsfälle für normale Benutzer und welche für Administratoren vorgesehen sind. Eine Übersicht aller Use-Cases bietet *Abbildung 16*. Diese Sicht hilft dabei, die fachlichen Möglichkeiten des Systems zu verstehen, bevor man sich mit der technischen Umsetzung beschäftigt. Die dargestellten Use-Cases beschreiben typische Interaktionen wie

Registrierung, Anmeldung, das Durchsuchen des Menüs, das Aufgeben von Bestellungen sowie administrative Verwaltungsaufgaben.

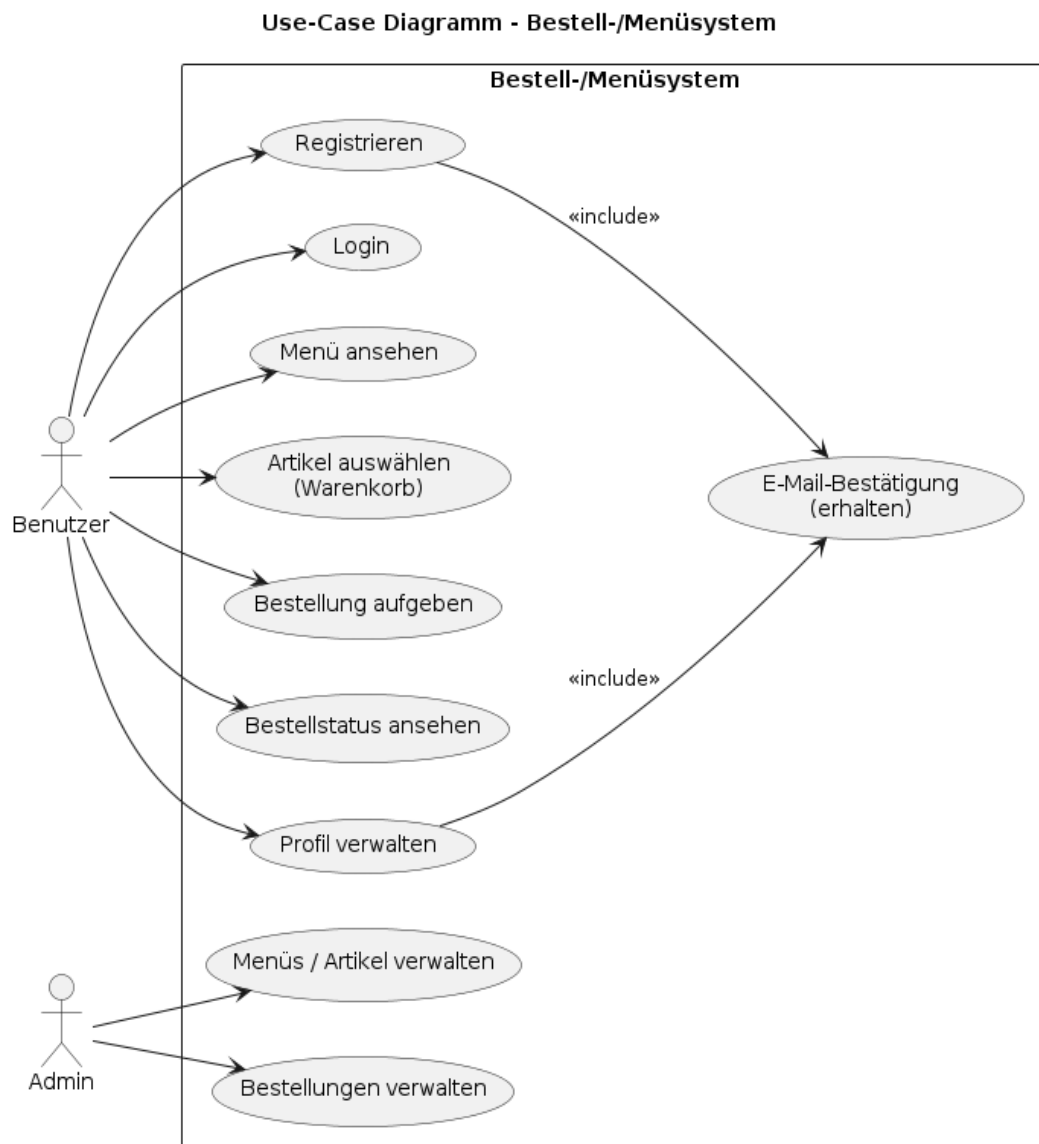


Abbildung 16: Use-Case Diagramm

4.1.7 Systemabläufe – Aktivitätsdiagramme

Bestellprozess (User)

Das *Aktivitätsdiagramm* (Abbildung 17) beschreibt den vollständigen Ablauf einer Bestellung aus Sicht eines Benutzers. Nach dem Öffnen der Anwendung wird der Benutzer zur Anmeldung weitergeleitet, sofern noch keine gültige Sitzung besteht. Erst nach erfolgreicher Authentifizierung stehen die Funktionen der Anwendung uneingeschränkt zur Verfügung.

Im nächsten Schritt kann der Benutzer das verfügbare Menü durchsuchen und einzelne Artikel auswählen. Für jeden Artikel besteht die Möglichkeit, optionale Angaben wie Hinweise oder Zusatzinformationen zu ergänzen. Die ausgewählten Artikel werden anschließend in den Warenkorb übernommen. Dieser Vorgang kann beliebig oft wiederholt werden, bis der Warenkorb vollständig ist.

Vor dem Absenden der Bestellung erhält der Benutzer eine Übersicht über alle ausgewählten Artikel sowie die Möglichkeit, den Warenkorb zu überprüfen und gegebenenfalls anzupassen. Nach der Bestätigung wird die Bestellung über eine *REST-Schnittstelle* an das Backend übermittelt. Dort erfolgt eine Validierung der übermittelten Daten, unter anderem hinsichtlich Vollständigkeit, Gültigkeit und Berechtigungen. Bei erfolgreicher Prüfung wird die Bestellung persistent in der Datenbank gespeichert und erhält einen entsprechenden Status.

Der finale Abschluss der Bestellung erfolgt zeitversetzt über einen *QR-Code-Scanner*. Dieser Schritt dient der eindeutigen Zuordnung und Bestätigung der Bestellung und markiert den Abschluss des Bestellprozesses. Durch diese Trennung zwischen Bestellerafassung und Abschluss wird ein kontrollierter und nachvollziehbarer Ablauf sichergestellt.

Aktivität - Bestellprozess

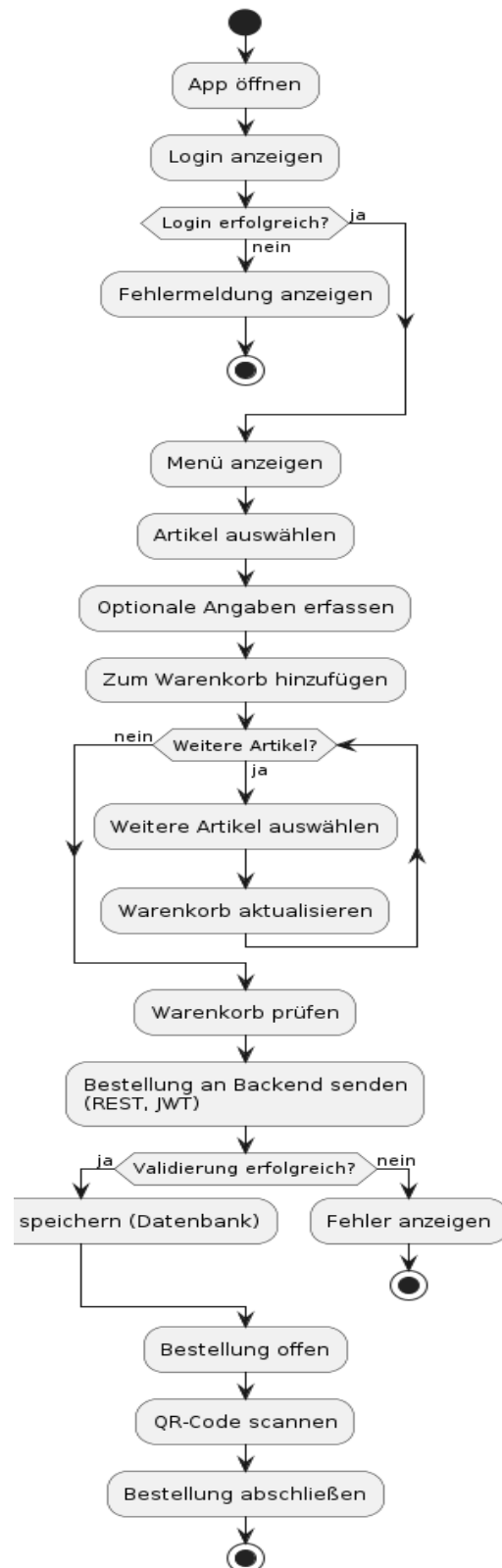


Abbildung 17: Aktivitätsdiagramm - Bestellprozess

Verwaltungsprozess (Admin)

Für den Inhaber stehen zusätzliche Funktionen zur Pflege von Menüs sowie zur Verwaltung von Bestellungen zur Verfügung. Der Verwaltungsprozess beginnt mit der Anmeldung und dem Zugriff auf den Administrationsbereich der Anwendung. Dabei wird geprüft, ob ein gültiges Token mit entsprechender Rolle vorliegt (siehe *Abbildung 18*).

Nach erfolgreicher Authentifizierung erhält der Administrator eine Übersicht über vorhandene Menüs und Artikel. Abhängig von der gewählten Aktion können neue Einträge erstellt, bestehende Inhalte bearbeitet oder deaktiviert werden. Die vorgenommenen Änderungen werden über die API an das Backend übermittelt, dort validiert und anschließend in der Datenbank gespeichert. Zusätzlich besteht die Möglichkeit, Bestellungen einzusehen und zu verwalten.

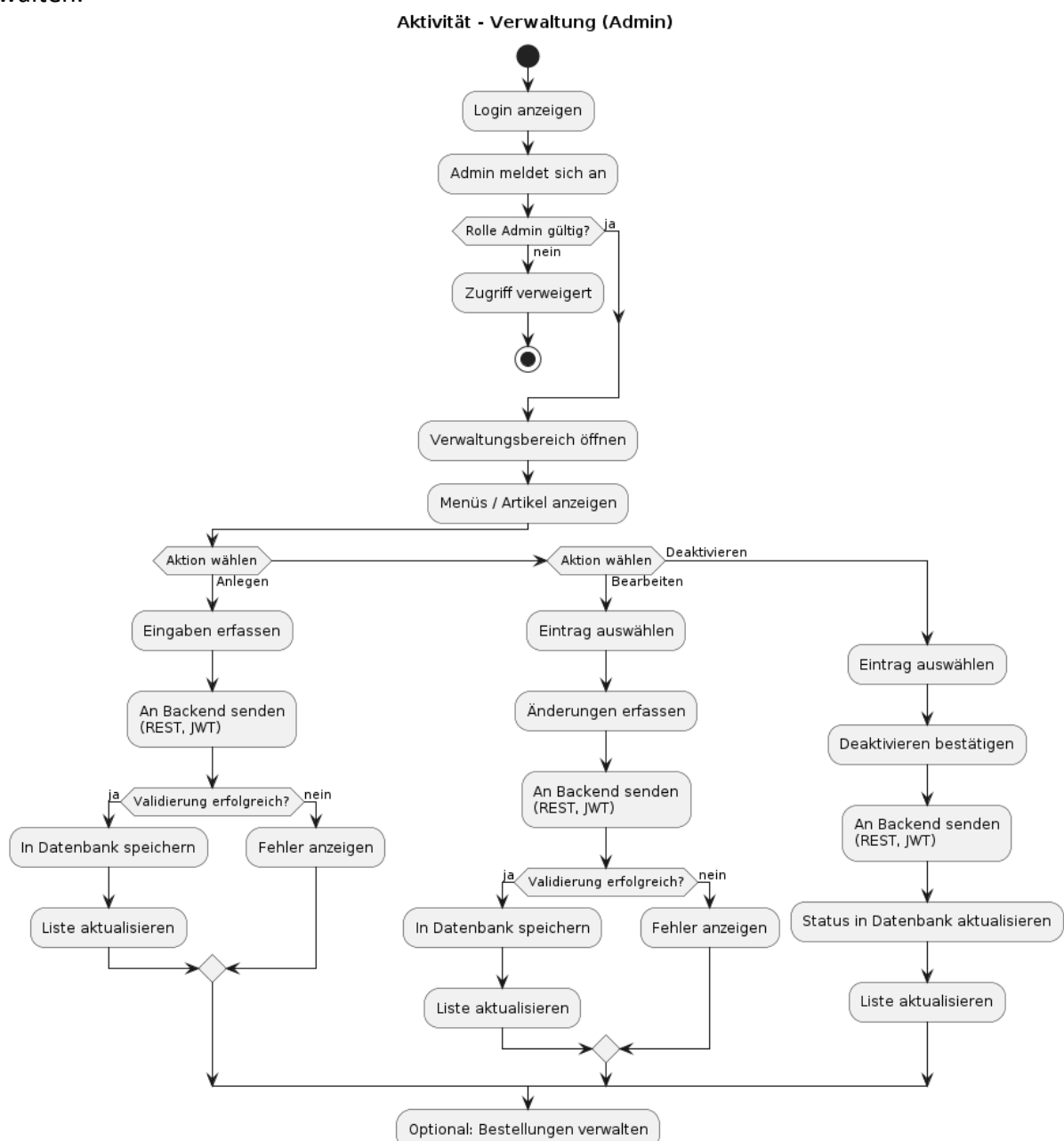


Abbildung 18: Aktivitätsdiagramm: Verwaltungsprozess

4.1.8 C4-Diagramme

Am Ende dieses Kapitels werden die zentralen Architektur Aspekte noch einmal im *C4-Format* dargestellt. Diese Darstellungsform eignet sich besonders gut, um die Systemarchitektur schrittweise und nachvollziehbar von einer übergeordneten Gesamtsicht bis hin zu internen Strukturen zu beschreiben. Durch die klare Trennung in *Kontext*-, *Container*- und *Komponentenebene* wird ein strukturiertes Verständnis des Systems ermöglicht.

Im Folgenden wird das *C4-Diagrammset* vorgestellt. Es zeigt das System zunächst im Kontext seiner Umgebung, anschließend auf Containerebene und abschließend in der Detailansicht der internen Backend-Komponenten. Die Diagramme basieren auf den definierten *Use-Cases* und stellen den aktuellen fachlichen und technischen Stand des Systems dar.

Kontextdiagramm

Im *Kontextdiagramm* wird das System in seiner Gesamtheit dargestellt und in die umgebende Systemlandschaft eingeordnet. Es verdeutlicht, welche *Akteure (Benutzer und Admin)* mit der Anwendung interagieren und welche externen Systeme angebunden sind. In diesem Projekt zählen dazu insbesondere die *PostgreSQL-Datenbank* zur persistenten Datenhaltung sowie die *Gmail API*, die für den Versand von E-Mails im Rahmen der Registrierung und Profilverwaltung genutzt wird (siehe *Abbildung 19*).

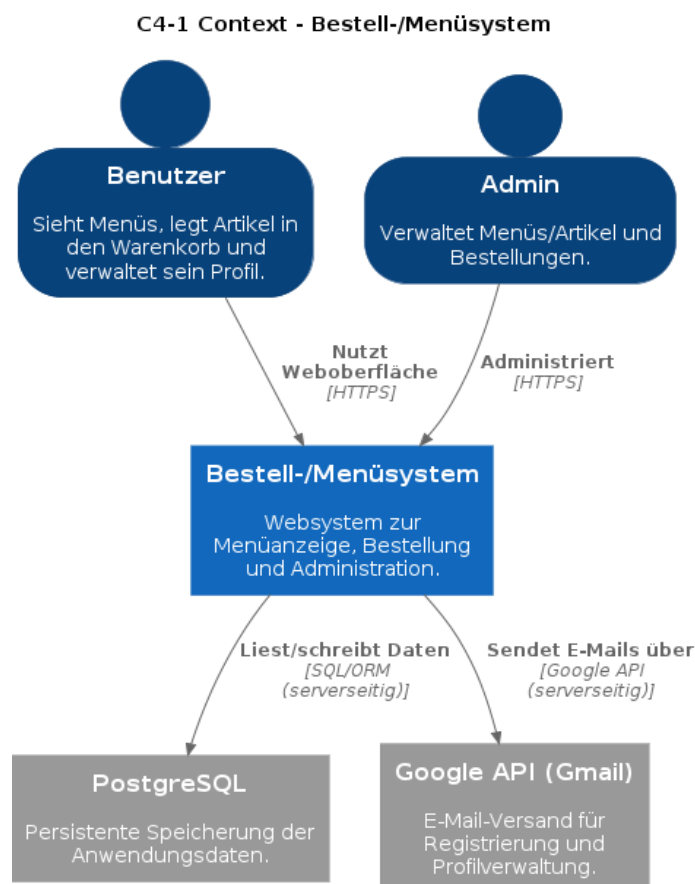


Abbildung 19: Kontextdiagramm - C4

Containerdiagramm

Aufbauend darauf veranschaulicht das *Containerdiagramm* die zentralen technischen Bausteine der Anwendung, sowie deren Kommunikationsbeziehungen. Das *Angular-Frontend* wird im *Browser* ausgeführt und stellt die Benutzeroberfläche einschließlich *clientseitiger Abläufe* (z. B. *QR-Scan-Flow*) bereit. Die Geschäftslogik und Zugriffskontrolle sind im *NestJS-Backend* angesiedelt. Diese stellt eine *REST-API*, *JWT-basierte Authentifizierung* und *Autorisierung* bereit und reguliert den Zugriff mittels *CORS*. Die persistente Speicherung erfolgt in *PostgreSQL*, während der E-Mail-Versand über die *Gmail API* realisiert wird (siehe Abbildung 20).

C4-2 Container - Bestell-/Menüsystem

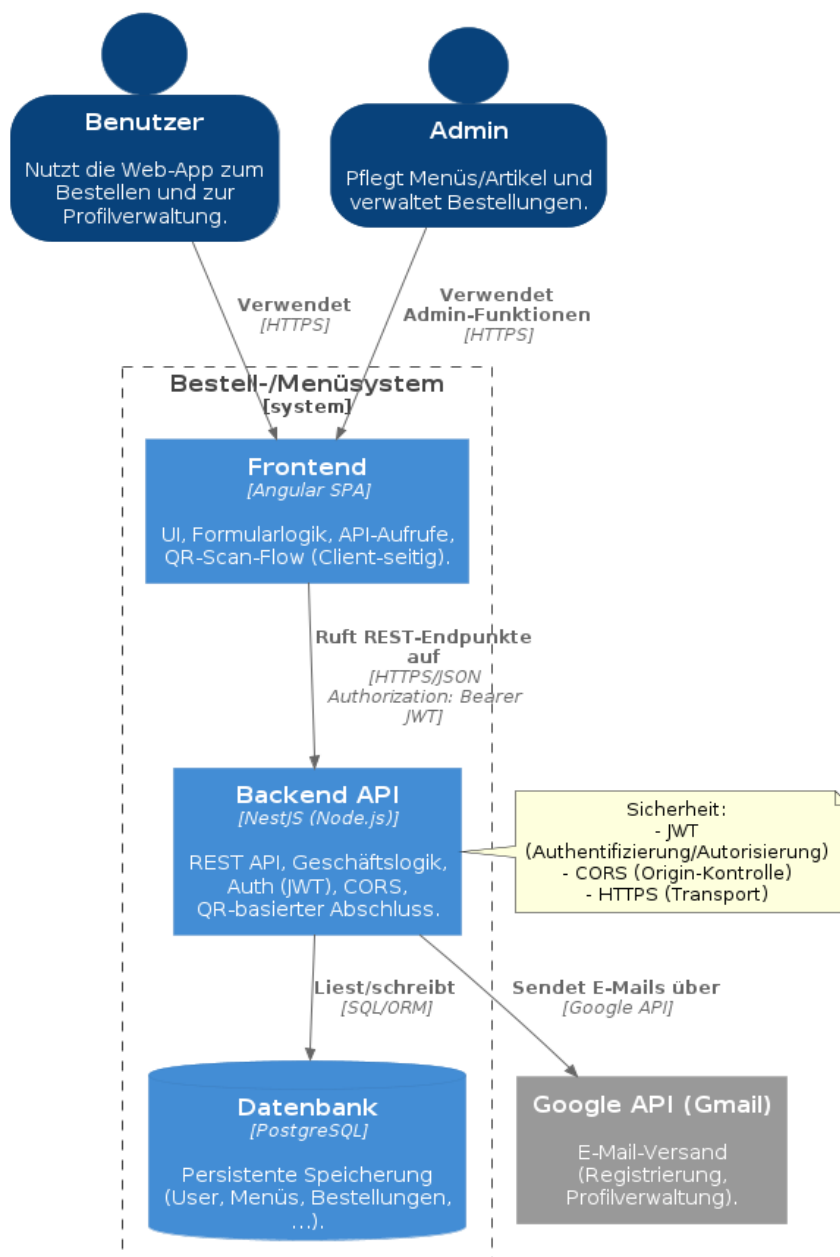


Abbildung 20: Containerdiagramm – C4

Komponentendiagramm

Eine detailliertere Sicht auf die interne Struktur liefert das *Komponentendiagramm* (Abbildung 21), das hier den Fokus auf das Backend legt. Es zeigt die Aufteilung in zentrale Komponenten, wobei Controller die *REST-Endpunkte* für Benutzer- und Admin-Funktionen bereitstellen. Sicherheitsrelevante Aspekte wie *JWT*, *Guards* und *CORS* sind im Modul *Auth & Security* gebündelt. Die fachliche Logik ist auf spezialisierte Services verteilt (Menü/Artikel, Bestellungen inklusive QR-Abschluss sowie Registrierung/Profil), während der Datenzugriff über *Object Relational Mapping (ORM)* auf die *PostgreSQL-Datenbank* erfolgt. Ergänzend ist ein Mail-Service integriert, der über die Google API den E-Mail-Versand für Registrierung und Profilverwaltung übernimmt.

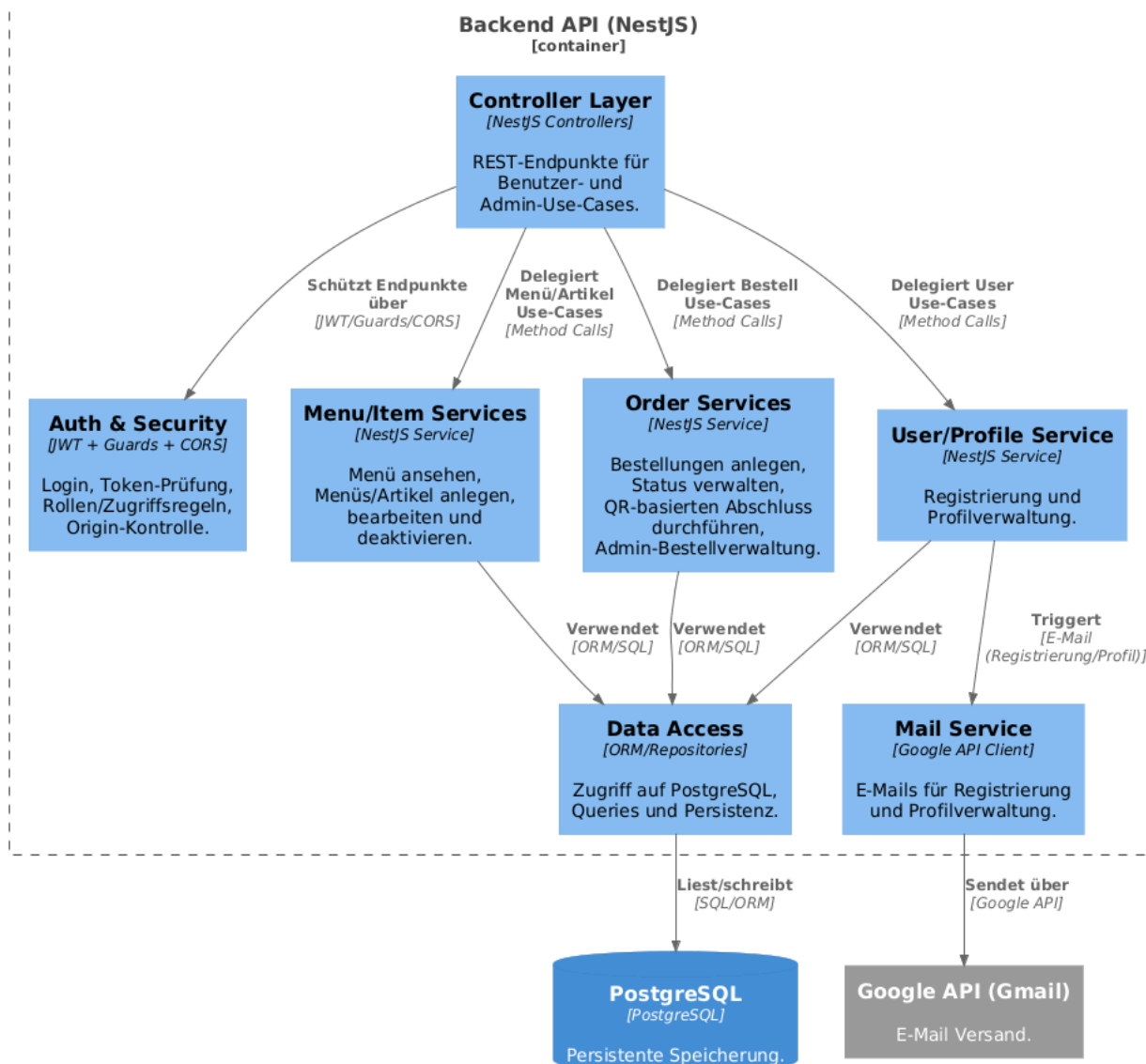


Abbildung 21: Komponentendiagramm – Backendservices – C4

4.2 Frontend-Entwicklung

In diesem Abschnitt werden die zentralen Bestandteile des *Frontends* sowie deren technische Umsetzung beschrieben. Der Fokus liegt auf dem strukturellen Aufbau der Benutzeroberfläche, der Aufteilung in *Komponenten* und *Services* sowie der Umsetzung der zentralen Anwendungslogik im Browser. Ziel ist eine klare Trennung von Darstellung, Benutzerinteraktion und *Backend-Kommunikation*, um Wartbarkeit und Erweiterbarkeit sicherzustellen.

4.2.1 Menüverwaltung (Magos)

Die Menüverwaltung ist ein zentraler Bestandteil des Bestellsystems. Die Seite bietet dem Inhaber eine zentrale Übersicht von allen erstellten Speiseplänen. Dabei können bestehende Speisepläne bearbeitet und neue hinzugefügt werden. Der Speiseplan der Woche wird auch hier ausgewählt. Weiters kann ein beliebiger Speiseplan als *PDF-Datei* exportiert werden. Ziel der Menüverwaltung ist es, eine benutzerfreundliche Übersicht der Speisepläne zu schaffen und alle damit gebundene Funktionalitäten zentral zu ermöglichen.

Laden und Anzeigen der Speisepläne

Beim Aufruf der Menüverwaltungsseite werden die bestehenden Speisepläne vom Backend geladen. Diese *Initialisierung* erfolgt über die *Angular-Lifecycle-Methode* *ngOnInit()*, die beim Erstellen der Komponente aufgerufen wird.

Der eigentliche Abruf der Speisepläne erfolgt innerhalb der *loadMenus()* Methode, wie in *Listing 1* zu sehen. Dabei wird mit dem *menuService* eine Anfrage an das Backend gesendet. Die empfangenen Speisepläne werden anschließend im *localStorage* gespeichert und im *Template* dargestellt.

```
1. private loadMenus(): void {
2.   this.loading = true;
3.   this.loadError = null;
4.
5.   this.menuService.getMenus().subscribe({
6.     next: (menus) => {
7.       this.Menus = menus;
8.       this.loading = false;
9.     },
10.    error: (err) => {
11.      this.loading = false;
12.      this.loadError = 'Menüs konnten nicht geladen werden.';
13.    }
14.  });
15. }
```

Listing 1: Frontend - Laden der Menus

Navigation zum Menüplaner

Um einen bestehenden Speiseplan zu bearbeiten, oder einen neuen hinzuzufügen, wird der Inhaber zum Menüplaner weitergeleitet. Hier können die gewünschten *Gerichte* dem Speiseplan hinzugefügt werden. Dabei wird das *Menüobjekt* über den *Router-State* an die Zielseite übergeben (siehe *Listing 2*).

```
1. goToMenuPlanner(menu: MealPlan) {  
2.   this.router.navigate(['menuplaner'], { state: { menu } });  
3. }
```

Listing 2: Frontend - Navigation zum Menüplaner

Mit dieser Vorgehensweise kann der *Menüplaner* die vorhandenen Daten bearbeiten, ohne einen zusätzlichen Abruf ans Backend zu machen.

Auswahl eines Speiseplans

Der Inhaber kann einen beliebigen Speiseplan anklicken, um diesen auszuwählen. Das heißt, dass die ausgewählten Gerichte und Menüs des Speiseplans im Frontend angezeigt werden und vom Kunden einsehbar sind. Der ausgewählte Speiseplan wird visuell hervorgehoben, um die Orientierung für den Inhaber zu erleichtern. Die Auswahl erfolgt über eine einfache Vergleichslogik (siehe *Listing 3*).

```
1. selectMenu(menu: MealPlan): void {  
2.   if (menu !== this.selectedMenu) {  
3.     this.selectedMenu = menu;  
4.   } else {  
5.     this.selectedMenu = null;  
6.   }  
7. }
```

Listing 3: Frontend - Speiseplan auswählen

Diese Logik dient ausschließlich der Benutzerinteraktion und beeinflusst keine gespeicherten Daten im Backend. Das ausgewählte Menü wird zunächst nur im *localStorage* gespeichert.

Löschen eines Speiseplans

Um eine Löschung durchzuführen, wird der Speiseplan aus der Liste der Speisepläne gelöscht. Dies findet zunächst nur im Frontend statt, wo sogleich eine Aktualisierung der Benutzeroberfläche ausgeführt wird. Danach wird die Löschanfrage an das Backend gesendet und der Speiseplan auch aus dem Backend gelöscht. Tritt bei der Anfrage ein Fehler auf, wird der vorherige Zustand der Speiseplanliste wiederhergestellt (siehe *Listing 4*).

```
1. removeMenu(menu: MealPlan): void {  
2.   const previousMenus = [...this.Menus];  
3.   this.Menus = this.Menus.filter((m) => m !== menu);  
4.  
5.   this.menuService.deleteMenu(menu.id).subscribe({  
6.     error: () => {  
7.       this.Menus = previousMenus;  
8.     }  
9.   });  
10. }
```

Listing 4: Frontend - Speiseplan löschen

Erstellung einer Druckbaren Speisekarte

Eine weitere Funktion der Menüverwaltungsseite ist die Erstellung einer druckbaren Speisekarte im *PDF-Format*. Diese Funktion wird in der Methode *PrintMenu()* realisiert. Wie bereits im Recherchekapitel erläutert, wird dafür die Bibliothek *jsPDF* verwendet (siehe *Listing 5*).

```
1. async printMenu(menu: MealPlan): Promise<void> {
```



```
2.   const doc = new jsPDF({
3.     orientation: 'portrait',
4.     unit: 'mm',
5.     format: 'a4'
6.   });
7.   ...
8.   doc.save(`${fileName}.pdf`);
9. }
```

Listing 5: Frontend - Kurzsicht der Erstellung einer PDF mit jsPDF

Die Generierung des *PDFs* erfolgt ausschließlich im Frontend. Das erstellte *PDF* enthält dabei die Gerichte des aktivierten Speiseplans, die Preise, sowie optionale Beschreibungen und Allergeninformationen. Zusätzlich kann ein Logo geladen und in das Dokument eingefügt werden. Dafür wird eine eigene Hilfsmethode verwendet, die das Bild *asynchron* lädt (siehe Listing 6).

```
1. private async loadLogo(): Promise<HTMLImageElement | null> {
2.   return new Promise((resolve) => {
3.     const img = new Image();
4.     img.onload = () => {
5.       resolve(img);
6.     };
7.     img.onerror = () => {
8.       resolve(null);
9.     };
10.    img.src = 'assets/logo.png';
11.  });
12.}
```

Listing 6: Frontend - Logo zur Anzeige laden

Durch diese Funktion kann der Inhaber Speisekarten direkt aus dem System heraus erstellen und weiterverwenden.

Zusammenfassung

Die Menüverwaltungsseite stellt eine zentrale Verwaltungsoberfläche im *Frontend* des Projekts „HungerSatt“ dar. Sie ermöglicht eine übersichtliche Verwaltung aller Menüs und bietet essenzielle Funktionen wie bearbeiten, löschen oder als Speiseplan exportieren. Durch die klare Struktur der Angular-Komponente, die Nutzung von Services für die *Backend-Kommunikation* sowie die gezielte Einbindung externer Bibliotheken, bleibt die Umsetzung wartbar, erweiterbar und gut nachvollziehbar. Die Menüverwaltung bildet damit eine wichtige Verbindung zwischen Menüplanung und dem operativen Einsatz des Bestellsystems.

4.2.2 Menüplanung (Magos)

Der Menüplaner ist eine weitere wichtige Komponente der Inhaberansicht. Er erlaubt dem Inhaber, die in der Menüverwaltung erstellten Speisepläne zu bearbeiten, bzw. mit Gerichten zu befüllen. Dabei können, von einer Liste aller verfügbaren Gerichte, beliebige Gerichte dem aktuellen Speiseplan hinzugefügt oder auch wieder entfernt werden. Diese Funktion ermöglicht die beliebige Gestaltung und Anpassung der Speisepläne. Der Fokus dabei liegt auf einer intuitiven und effizienten Bedienung für den Inhaber.

Initialisierung und Laden der Daten

Beim Öffnen des Menüplaners wird als Erstes geprüft, ob ein bestehender Speiseplan bearbeitet oder ob ein neuer Speiseplan erstellt werden soll. Diese Feststellung erfolgt in der *ngOnInit()*-Methode anhand des *Router-States* (siehe *Listing 7*).

```
1. ngOnInit(): void {
2.   const stateMenu = history.state.menu as MealPlan | undefined;
3.   if (stateMenu?.id) {
4.     this.menu = {
5.       id: String(stateMenu.id),
6.       title: stateMenu.title ?? '',
7.       menuItems: rawItems
8.     };
9.
10.    this.menuTitle = this.menu.title ?? '';
11.    this.selectedDishes = [...(this.menu.menuItems ?? [])];
12.  } else {
13.    this.menu = { id: 'new', title: '', menuItems: [] };
14.    this.selectedDishes = [];
15.  }
16.  this.loadAllMenuItemsAndSplit();
17. }
```

Listing 7: Frontend – Ausgewählten Speiseplan laden

Nach der Initialisierung werden alle verfügbaren Gerichte über einen *API-Service* geladen und anschließend in zwei Listen eingeteilt: Gerichte, die bereits im Menü enthalten sind, und Gerichte, die noch hinzugefügt werden können.

Laden und Aufteilen der Daten

Die Methode *loadAllMenuItemsAndSplit()* wird in der *Initialisierungsmethode* aufgerufen und ist zuständig für das Laden von vorhandenen Gerichten vom Backend und die Teilung dieser Gerichte in zwei Listen. Die besagte Trennung in zwei Listen stellt sicher, dass jedes Gericht entweder im Menü oder in der Liste der verfügbaren Gerichte angezeigt wird, aber nie in beiden gleichzeitig (siehe *Listing 8*).

```
1. this.menuItems.getAll().subscribe({
2.   next: (all: MenuItem[]) => {
3.     const selectedIds = new Set(this.selectedDishes.map(x => x.id));
4.     this.unselectedDishes = all.filter(mi => !selectedIds.has(mi.id));
5.   }
6. });
```

Listing 8: Frontend - Gerichte des gewählten Speiseplan laden

Drag and Drop

Um die Verschiebung der Gerichte zwischen den beiden Listen, sowie die Sortierung der Gerichte innerhalb einer Liste benutzerfreundlich und intuitiv zu machen, wurde für diesen Zweck ein *Drag-and-Drop System* implementiert. Wie in der Recherche erwähnt, wurde dazu das *CDK Drag-Drop Modul* verwendet. Dieses stellt Direktiven zur Verfügung, die direkt im *HTML-Template* verwendet werden und die *Drag-&-Drop-Logik kapselt*.

Im Template werden beide Listen als sogenannte Drop-Listen definiert (siehe *Listing 9*). Diese Listen ermöglichen das Verschieben der Elemente, da diese mit dem *cdk-Drag Attribut* versehen sind. (siehe *Listing 10*).

```

1. <div
2.   cdkDropList
3.   [cdkDropListData]="selectedDishes"
4.   [cdkDropListConnectedTo]="['unselectedDishesList']"
5.   (cdkDropListDropped)="drop($event)"
6.   id="selectedDishesList"
7. >

```

Listing 9: Frontend - Gerichte in Drop Listen

```

1. @for (dish of selectedDishes; track dish.id) {
2.   <div class="dish-container"
3.     cdkDrag
4.     ...
5.   >
6.   </div>
7. }

```

Listing 10: Frontend - Liste der gewählten Gerichte anzeigen

Jede Liste übergibt dabei ihr bestehendes Array an die *Drag-&Drop-Direktive*. Beim Ablegen eines Elements wird automatisch ein *Drop-Event* ausgelöst, das im *Typescript* Code verarbeitet wird.

Die zentrale Logik dazu befindet sich in der Methode *drop()*. Diese stellt fest, ob ein *Element* innerhalb der eigenen Liste verschoben wurde oder zwischen den Listen gewechselt hat (siehe *Listing 11*).

```

1. drop(event: CdkDragDrop<MenuItem[]>): void {
2.   if (event.previousContainer === event.container) {
3.     moveItemInArray(event.container.data, event.previousIndex, event.currentIndex);
4.     return;
5.   }
6.
7.   transferArrayItem(
8.     event.previousContainer.data,
9.     event.container.data,
10.    event.previousIndex,
11.    event.currentIndex
12.  );
13. }

```

Listing 11: Frontend - Drag and drop Funktion

Wenn ein Element innerhalb der gleichen Liste verschoben wird, wird nur die Reihenfolge im Array angepasst. Wenn ein Element von der einen Liste zur anderen wechselt, wird es vom Quellarray entfernt und ins Zielarray eingefügt. Da *Angular* die Listen direkt an die Datenstrukturen bindet, aktualisiert die Anzeige damit automatisch.

Klickbasierte Interaktion

Als Alternative zu der *Drag and Drop Funktion* können Menüs zwischen den Listen auch durch einfaches Anklicken verschoben werden.

Zur Verbesserung der Benutzerfreundlichkeit wird verhindert, dass während eines aktiven *Drag*-Prozesses, Klicks ausgelöst werden, damit unbeabsichtigte Aktionen vermeiden werden können.

Aktivieren und Deaktivieren von Gerichten

Gerichte können innerhalb eines Menüs temporär deaktiviert werden, ohne entfernt zu werden (siehe *Listing 12*). Diese Information wird sowohl im Frontend als auch im Backend gespeichert.

```
1. toggleAvailable(item: MenuItem; ev?: Event): void {  
2.   ...  
3.   const nextAvailable = !(item.available === false);  
4.   ...  
5.   this.selectedDishes = this.selectedDishes.map(x =>  
6.     x.id === item.id ? { ...x, available: nextAvailable } : x  
7.   );  
8.   ...  
9. }
```

Listing 12: Frontend - Aktivieren /Deaktivieren von Gerichten

Deaktivierte Gerichte werden im Benutzerinterface entsprechend gekennzeichnet und bei der Anzeige von Menüs berücksichtigt.

Bearbeiten und Löschen von Gerichten

Gerichte können im Menüplaner bearbeitet sowie gelöscht werden. Für die Bearbeitung eines Gerichts wird der Benutzer zur Gerichtsverwaltung weitergeleitet. Dabei wird das ausgewählte Gericht und der aktuelle Menüplan übergeben.

Das Löschen eines Gerichts erfolgt über ein Bestätigungsdialogfenster. Beim Löschen wird das Gericht sofort aus der Oberfläche entfernt und erst danach aus dem Backend gelöscht. Sollte es bei dem Löschvorgang zu einem Fehler kommen, wird der vorherige Zustand wiederhergestellt.

Speichern des Speiseplans

Beim Speichern wird zwischen dem Erstellen eines neuen Menüs und dem Aktualisieren eines bestehenden Menüs unterschieden. Vor dem Speichern wird der *Menü Titel* validiert, um leere Eingaben zu verhindern. Abhängig vom Zustand des Menüs wird anschließend entweder eine *Create-* oder *Update-Anfrage* an das Backend gesendet. Nach erfolgreichem Speichern wird der Benutzer zurück zur Menüverwaltung navigiert.

Zusammenfassung

Der Menüplaner ermöglicht eine detaillierte und flexible Verwaltung der Speisepläne, innerhalb des Frontend. Durch die Kombination der *Drag and Drop Funktion* und der klickbasierten Interaktion ist der Bearbeitungsprozess eines Speiseplans benutzerfreundlich und effizient. Die klare Trennung zwischen ausgewählten und verfügbaren Gerichten, sowie die Möglichkeit einzelne Gerichte zu deaktivieren erhöht die Flexibilität bei der Menüplanung. Der Menüplaner bildet damit eine zentrale Komponente für die Nutzung des Systems durch den Inhaber.

4.2.3 Verwaltung von Gerichten (*Magos*)

Die Verwaltung von Gerichten ist die notwendige Komponente für die Erstellung und Bearbeitung einzelner Gerichte. Es können damit sowohl neue Gerichte erstellt und gespeichert, als auch bestehende Gerichte bearbeitet werden. Die Komponente stellt ein Formular zur Verfügung, in dem alle relevanten Eigenschaften eines Gerichts definiert werden können, darunter fallen Name, Beschreibung, Preis, Kategorie, Allergene sowie zusätzliche Angaben für Menüs. Zusätzlich wird eine Vorschau des aktuellen Gerichts angezeigt, sodass Änderungen unmittelbar sichtbar sind. Damit kann der Inhaber sofort sehen, wie das Gericht später den Kunden angezeigt wird.

Initialisierung und Übernahme vorhandener Daten

Beim Öffnen der *Verwaltung von Gerichten* wird zuerst geprüft, ob ein bestehendes Gericht bearbeitet werden soll. Wenn das der Fall ist, werden die mittels Router-State übertragenen Daten in das Formular übernommen (siehe *Listing 13*).

```
1. ngOnInit(): void {
2.   const st = history.state ?? {};
3.   const prefill = st.dish as MenuItem | undefined;
4.
5.   if (prefill?.id) {
6.     this.dish = {
7.       ...this.dish,
8.       ...prefill,
9.       id: String(prefill.id),
10.      price: Number(prefill.price ?? 0),
11.      ...
12.    };
13.  }
14. }
```

Listing 13: Frontend - Laden von bereits bestehenden Gerichten

Optional wird das Gericht anschließend nochmals vollständig vom Backend geladen. Dadurch wird sichergestellt, dass auch bei älteren oder unvollständigen Übergabedaten immer der aktuelle Stand verwendet wird.

Dynamische Formularlogik

Da Menüs, zusätzlich zum Hauptgericht, ein Getränk und eine Nachspeise beinhalten, müssen bei der Bearbeitung oder Erstellung eines Menüs diese zusätzlichen Eigenschaften definiert werden können. Um Verwirrung bei der Nutzung zu vermeiden, werden die Eingabefelder für diese Eigenschaften nur im Fall eines Menüs angezeigt. Die Anzeige wird über das Auswählen der Kategorie *Menü* gesteuert (siehe *Listing 14*).

```
1. get isMenuCategory(): boolean {
2.   return this.normalizeCategory(this.dish.category) === 'Menü';
3. }
```

Listing 14: Frontend - dynamische Veränderung der Formularlogik

Im *Template* werden diese Eingabefelder also nur dann angezeigt, wenn die notwendige Bedingung dazu erfüllt wird. Wenn die Kategorie gewechselt wird, werden nicht mehr relevante Felder automatisch geleert, damit keine ungültigen Daten gespeichert werden.

Allergenverwaltung

Allergene können einzeln über ein Eingabefeld hinzugefügt werden. Dabei wird geprüft, ob der Wert leer ist und ob dieses Allergen schon vorhanden ist, um doppelte Werte zu vermeiden. Die Allergene werden anschließend in einer Liste von Textwerten gespeichert (siehe *Listing 15*).

```
1. addAllergen(): void {
2.   const allergen = this.allergenTemp?.trim();
3.   if (!allergen) return;
4.
5.   if (!this.dish.allergens.includes(allergen)) {
6.     this.dish = {
7.       ...this.dish,
8.       allergens: [...this.dish.allergens, allergen]
9.     };
10.  }
11. }
```

Listing 15: Frontend - Allergene Logik

Aus dieser Liste können bereits vorhandene Allergene auch wieder entfernt werden. Die Darstellung erfolgt als Liste einzelner Einträge mit der Option, jedes einzelne Element zu entfernen.

Vorschau des Gerichts

Parallel zur Eingabe, ist eine *Vorschau-Komponente* eingebunden, die das aktuelle Gericht visuell darstellt. Diese Komponente greift direkt auf das Datenobjekt zu und aktualisiert sich bei Änderungen automatisch. Durch diese Vorschau bekommt der Inhaber sofort Rückmeldung über die Eingaben.

Validierung der Eingaben

Vor dem Speichern des Gerichts werden die Eingaben *validiert*. Dabei wird geprüft, ob die Pflichtfelder Name, Beschreibung und Kategorie, leer sind und ob der Preis 0 ist. Nur wenn das nicht der Fall ist, kann ein Gericht gespeichert werden. Diese Validierung erfolgt über die *validateDish()-Methode* (siehe *Listing 16*).

```
1. private validateDish(): string[] {
2.   const errors: string[] = [];
3.
4.   if (!this.dish.name.trim()) errors.push('Name fehlt.');
```

Listing 16: Frontend - Validierung der Eingaben beim Erstellen von Gerichten

Speichern eines Gerichts

Beim Speichern wird als Erstes die vorher erwähnte *validateDish()*-Methode aufgerufen. Wenn keine Fehler auftreten, werden die Eingabewerte nochmal bereinigt und in ein standardisiertes Objekt überführt (siehe *Listing 17*).

```
1. const itemToSave: MenuItem = {
2.   ...this.dish,
3.   name: this.dish.name.trim(),
4.   description: this.dish.description.trim(),
5.   category,
6.   price: Number(this.dish.price),
7.   drink: category === 'Menü' ? this.dish.drink?.trim() : undefined,
8.   dessert: category === 'Menü' ? this.dish.dessert?.trim() : undefined,
9. };
```

Listing 17: Frontend - Speichern des neuen Gerichtes als MenuItem

Nach der Erstellung eines *Gericht-Objekts*, wird es über den *API-Service* im Backend gespeichert. Dieser entscheidet intern auch, ob ein neues Gericht angelegt wird, oder ob ein bestehendes aktualisiert wird. Nach erfolgreichem Speichern wird zur vorherigen Ansicht zurücknavigiert. Das aktualisierte Gericht wird dabei an den *Router-State* übergeben. Dadurch können andere Komponenten ihre Ansicht sofort aktualisieren (siehe *Listing 18*).

```
1. this.menuItemsApi.save(itemToSave).subscribe({
2.   next: (saved) => {
3.     this.saving = false;
4.     this.dish = {
5.       ...saved,
6.       category: this.normalizeCategory((saved as any).category),
7.       drink: String((saved as any).drink ?? ''),
8.       dessert: String((saved as any).dessert ?? ''),
9.     };
10.
11.     this.router.navigate([this.returnTo], {
12.       state: { ...(this.returnState ?? {}), menuItem: saved },
13.     });
14.   },
```

Listing 18: Frontend - Speichern des erstellten Gerichtes

Zusammenfassung

In der Ansicht zur Verwaltung von Gerichten wird die Erstellung und Bearbeitung von Gerichten ermöglicht. Durch die Kombination aus Formularlogik, Normalisierung, Validierung und direkter Backend-Anbindung können Gerichte *konsistent* erstellt und geändert werden. Die dynamische Anzeige abhängiger Felder und die *Live-Vorschau* verbessern die Benutzerfreundlichkeit zusätzlich. Die Komponente bildet damit die Grundlage für die Verwaltung aller Gerichte im System und unterstützt die Integration im Menüplaner und in der Menüverwaltung.

4.2.4 Auswahl von Gerichten (Kunden-Ansicht) (Ellmer)

Die Artikelauswahl ist die zentrale Benutzeroberfläche für reguläre Benutzer und bildet den Einstieg in den Bestellprozess. Auf dieser Seite wird der aktuell aktive Speiseplan geladen und dessen Artikel werden übersichtlich dargestellt. Nutzer können Gerichte suchen, nach Kategorien filtern und Artikel inklusive optionaler Notiz dem Warenkorb hinzufügen.

Technisch wird diese Funktionalität durch die Komponenten *MenuPlanComponent* (Daten laden, Filtern, Warenkorb-Integration) und *MenuItemComponent* (Darstellung und Interaktion pro Artikel) umgesetzt. Dadurch bleibt die Logik klar getrennt: Die Menüseite steuert den Ablauf, während die Artikelkomponente wiederverwendbar und schlank bleibt.

Laden des aktiven Speiseplans

Beim Initialisieren der Menüansicht wird der aktive Speiseplan regelmäßig aktualisiert. Dies geschieht sowohl periodisch (z. B. alle 10 Minuten) als auch beim Zurücknavigieren auf die Startseite über die *Angular-Lifecycle-Methode* *ngOnInit()*. Damit wird sichergestellt, dass Nutzer Änderungen am Wochenmenü zeitnah sehen, ohne die Seite manuell neu laden zu müssen (siehe *Listing 19*).

```
1. ngOnInit(): void {  
2.   this.fetchSelectedMenu();  
3.   this.refreshInterval = setInterval(() => {  
4.     this.fetchSelectedMenu();  
5.   }, 600000);  
6. }
```

Listing 19: Frontend – Regelmäßiges prüfen, geänderter Daten

Der eigentliche Abruf erfolgt in *fetchSelectedMenu()*. Die Methode prüft zuerst, ob überhaupt ein Benutzer eingeloggt ist. Danach wird das aktive Menü vom Backend geladen und in einen frontendtauglichen Zustand überführt. Außerdem wird die *Menü-ID* im Warenkorb gespeichert, um spätere Validierungen gegen Menüwechsel zu ermöglichen (siehe *Listing 20*).

```
1. fetchSelectedMenu(): void {  
2.   if (!this.authService.isLoggedIn()) return;  
3.  
4.   this.menuService.getActiveMenu().subscribe(menu => {  
5.     this.menu = menu;  
6.     this.cartService.setMenuId(menu.id);  
7.   });  
8. }
```

Listing 20: Frontend - regelmäßiges Laden eines Menüs

Filter- und Suchfunktionen

Für eine schnelle Orientierung in der Menüauswahl werden Kategorien, ein vegetarischer Filter sowie eine *Freitextsuche* bereitgestellt. Die Filterlogik wird clientseitig angewendet, damit die Interaktion ohne zusätzliche *Backend-Anfragen* flüssig bleibt (siehe *Listing 21*).

```
1. get filteredItems(): MenuItem[] {  
2.   return this.menu.items  
3.     .filter(i => this.selectedCategory ? i.category === this.selectedCategory : true)  
4.     .filter(i => !this.vegetarianOnly || i.vegetarian)  
5.     .filter(i => i.title.toLowerCase().includes(this.searchTerm.toLowerCase())); // schnelle  
6.   }  
7. }
```

Listing 21: Frontend - Filteroptionen

Darstellung und Bestellbarkeit einzelner Artikel

Die Darstellung eines einzelnen Artikels erfolgt über die *MenuItemComponent*. Diese Komponente übernimmt nicht nur die visuelle Darstellung, sondern entscheidet auch, ob ein Artikel aktuell bestellbar ist.

Ein Artikel ist beispielsweise nicht bestellbar, wenn er als reines Extra gekennzeichnet ist oder wenn er vom Inhaber deaktiviert wurde (siehe *Listing 22*).

```
1. get isExtraItem(): boolean {  
2.   return String(this.item?.id ?? '').startsWith('extra:'); // Extras nicht direkt bestellbar  
3. }  
4.  
5. get canOrder(): boolean {  
6.   if (this.isExtraItem) return false;  
7.   return this.item.available === true; // zentrale Bestellbarkeitsregel für UI  
8. }
```

Listing 22: Frontend - Holen der Daten von einem Gericht bzw. einem Menü

Hinzufügen von Artikeln zum Warenkorb

Beim Hinzufügen eines Artikels zum Warenkorb kann der Benutzer optional eine Notiz hinterlegen. Zusätzlich werden automatisch Menüinformationen ergänzt, um die Bestellung eindeutig zu machen (z. B. bei Menüsets) (siehe *Listing 23*).

Existiert bereits ein identischer Artikel mit gleicher Notiz im Warenkorb, wird lediglich die Menge erhöht. Andernfalls wird eine neue Position angelegt.

```
1. addToCart(item: MenuItem, note: string): void {  
2.   this.cartService.addItem({  
3.     menuItemId: item.id,  
4.     note,  
5.     quantity: 1  
6.   });  
7. }
```

Listing 23: Frontend - Auswahl eines Gerichtes für den Warenkorb

Diese Vorgehensweise verhindert doppelte Einträge und sorgt für einen übersichtlichen Warenkorb.

Die Artikelauswahl ermöglicht eine übersichtliche Darstellung und Auswahl der verfügbaren Gerichte und Menüsets. Durch clientseitige Filterung, regelmäßiges Nachladen des aktiven Speiseplans und die klare Trennung der Komponenten wird eine reaktionsschnelle und konsistente Benutzerführung gewährleistet.

4.2.5 Warenkorb *(Ellmer)*

Der Warenkorb stellt die zentrale Übersicht aller ausgewählten Artikel dar und erlaubt dem Benutzer, diese vor dem Absenden der Bestellung zu überprüfen und anzupassen. Die Umsetzung erfolgt über die *CartPageComponent*, während einzelne Positionen durch die *CartItemComponent* repräsentiert werden.

Mengensteuerung und Entfernen von Artikeln

Die Anpassung der Artikelmenge sowie das Entfernen von Positionen erfolgen über klar definierte Methoden, die an den *CartService* delegiert werden (siehe *Listing 24*). Dadurch bleibt die Seitenkomponente übersichtlich, während die Geschäftslogik zentral gebündelt ist.

```
1. increase(itemId: string): void {
2.   this.cartService.increment(itemId); // Logik zentral im Service
3. }
4. decrease(itemId: string): void {
5.   this.cartService.decrement(itemId);
6. }
7. remove(itemId: string): void {
8.   this.cartService.remove(itemId);
9. }
```

Listing 24: Frontend – Mengensteuerung und Aufruf von Servicefunktionen für den Warenkorb

Diese Methode *increaseQuantity* aus dem *CartService* ist für die eigentliche Erhöhung der Menge zuständig. Sie sucht den entsprechenden Warenkorbbeitrag anhand der *menuItemid* und erhöht dessen *quantity*. Existiert kein passender Eintrag, wird die Operation abgebrochen (siehe *Listing 25*).

```
1. increaseQuantity(menuItemId: string): void {
2.   const item = this.items.find(i => i.menuItemId === menuItemId);
3.   if (!item) return;
4.
5.   item.quantity += 1; // Erhöhen der Bestellmenge um eins
6. }
```

Listing 25: Frontend – Ausprogrammierung increment Methode

Validierung des Warenkorbs

Bevor eine Bestellung abgesendet wird, wird geprüft, ob die im Warenkorb enthaltenen Artikel noch zum aktuell aktiven Speiseplan gehören. Diese Validierung verhindert, dass Bestellungen mit veralteten oder nicht mehr verfügbaren Menüs abgeschickt werden (z. B. nach einem Menüwechsel durch den Inhaber).

Die Seitenkomponente ruft dafür ausschließlich die Methode *isValidForMenu()* auf, während die eigentliche Logik im *CartService* gekapselt ist.

Die Seitenkomponente kennt dabei keine Details zur Validierung, sondern reagiert lediglich auf das Ergebnis. Die eigentliche Validierung findet im Service statt. Dabei wird überprüft, ob ein *Menüplan* aktiviert wurde und das angeklickte *Gericht* darin ist bzw. verfügbar ist (siehe *Listing 26*).

```
1. isValidForMenu(): boolean {
2.   // Falls kein Menü gesetzt ist, kann keine gültige Bestellung erfolgen
3.   if (!this.activeMenuId) {
4.     return false;
5.   }
6.
7.   // Prüfen, ob alle Warenkorbpositionen zum aktuell aktiven Menü gehören
8.   return this.items.every(item => item.menuId === this.activeMenuId);
9. }
```

Listing 26: Frontend - Überprüfung verfügbarer Gerichte

Absenden der Bestellung

Vor dem Absenden der Bestellung werden alle relevanten Bedingungen geprüft. Anschließend wird ein reduziertes Datenobjekt erstellt und an das Backend übermittelt. Nach erfolgreicher Bestellung wird der Warenkorb geleert und der Benutzer zur Bestellübersicht weitergeleitet (siehe *Listing 27*).

```
1. submitOrder(): void {
2.   if (!this.cartService.isValidForMenu()) return; // verhindert Bestellung mit veraltetem Menü
3.
4.   const payload = this.cartService.buildOrderPayload(); // nur relevante Felder fürs Backend
5.   this.orderService.createOrder(payload).subscribe(() => {
6.     this.cartService.clear(); // Zustand nach Erfolg zurücksetzen
7.     this.router.navigate(['/orders']); // Weiterleitung zur Bestellübersicht
8.   });
9. }
```

Listing 27: Frontend - Absenden der Bestellung

Der Warenkorb bietet eine zentrale Übersicht über alle ausgewählten Artikel und erlaubt deren Anpassung vor dem Bestellabschluss. Durch Validierungen und klare Zuständigkeiten in den Komponenten wird ein stabiler und nachvollziehbarer Bestellprozess unterstützt.

4.2.6 Bestellübersicht – Kunden (Ellmer)

Die Bestellübersicht dient dem Benutzer als zentrale Anlaufstelle zur Einsicht seiner bisherigen und aktuellen Bestellungen. Sie ermöglicht eine strukturierte Darstellung aller Bestellungen sowie eine klare Trennung zwischen offenen und abgeschlossenen Bestellvorgängen. Neben allgemeinen Informationen wie Bestelldatum, Inhalt und Gesamtpreis stellt diese Ansicht auch die Grundlage für den *QR-basierten* Abschluss des Bestellprozesses dar.

Die Umsetzung erfolgt über die *OrderOverviewComponent* in Kombination mit dem *MyOrderService*, welcher für den Abruf, die Aufbereitung und die Anreicherung der Bestelldaten verantwortlich ist.

Laden der Bestellungen

Beim Initialisieren der Bestellübersicht werden die dem aktuell angemeldeten Benutzer zugeordneten Bestellungen vom Backend geladen (siehe *Listing 28*). Während dieses Vorgangs wird ein Ladezustand angezeigt, um dem Benutzer entsprechendes Feedback zu geben und eine transparente Benutzerführung zu gewährleisten.

```
1. loadOrders(): void {
2.   this.loading = true; // UI-Feedback während Request
3.   this.myOrderService.getMyOrders().subscribe({
4.     next: (orders) => {
5.       this.orders = orders;
6.       this.loading = false;
7.     },
8.     error: () => {
9.       this.loading = false;
10.    }
11.  });
12. }
```

Listing 28: Frontend - Alle Bestellungen des jeweiligen Kunden werden geladen

Durch diese Aufbereitung wird sichergestellt, dass jede Bestellung einen konsistenten Gesamtpreis besitzt und die Daten korrekt dargestellt werden können.

Trennung nach Bestellstatus

Die Bestellungen werden im Frontend nach ihrem Status gefiltert. Dadurch können offene und abgeschlossene Bestellungen übersichtlich getrennt dargestellt werden (siehe *Listing 29*).

```
1. get openOrders(): Order[] {
2.   return this.orders.filter(o => o.status === 'open'); // aktive Bestellungen
3. }
4. get closedOrders(): Order[] {
5.   return this.orders.filter(o => o.status === 'closed'); // abgeschlossene Bestellungen
6. }
```

Listing 29: Frontend - Trennung zwischen aktiven und abgeschlossenen Bestellungen

Diese Trennung verbessert die Übersichtlichkeit und unterstützt den Benutzer dabei, aktive Bestellungen schnell zu finden und auch einen Überblick über vergangene Bestellungen zu erhalten.

QR-Code für offene Bestellungen

Ein zentrales Merkmal der Bestellübersicht ist die Generierung eines *QR-Codes* für offene Bestellungen. Dieser *QR-Code* enthält eine eindeutige Referenz auf die jeweilige Bestellung und wird später für den Abschluss des Bestellvorgangs verwendet. Dabei wird in dieser Ansicht der *QR-Code* mithilfe des externen *API-Dienstes* (<https://api.qrserver.com/>) erstellt. Wie dieser Dienst funktioniert wurde bereits im [Rechercheteil, Kapitel 2.9.2](#) genauer beschrieben. Der *QR-Code* enthält die *Id* der Bestellung. Dieser *QR-Code* wird später vom Inhaber in der *Bestellungen-Übersicht* bzw. dem *Admin-Scanner* gescannt, um den Bestellabschluss eindeutig und nachvollziehbar auszulösen. Die Generierung des *QR-Codes* erfolgt clientseitig, wobei lediglich eine *URL* mit der *Bestell-ID* kodiert wird (siehe *Listing 30*). Die eigentliche Verarbeitung und Validierung der Bestellung findet jedoch serverseitig statt.

```
1. getQrUrl(orderId: string): string {
2.   const data = `order-${orderId}`; // bewusst nur ID/Referenz, keine sensiblen Daten
3.   return `https://api.qrserver.com/v1/create-qr-code/?data=${encodeURIComponent(data)}`;
4. }
```

Listing 30: Frontend - Erstellung und Laden des QR-Codes zur Bestellbestätigung

Anzeige von Bestelldetails

Um die Übersichtlichkeit der Bestellübersicht zu gewährleisten, werden Bestellungen standardmäßig in einer kompakten Darstellung angezeigt. Der Benutzer kann bei Bedarf zusätzliche Details ein- oder ausblenden, ohne die Seite zu wechseln. Zusätzlich wird für jede Bestellung ein kurzer, zusammenfassender Titel erzeugt, der die wichtigsten Artikel enthält (siehe *Listing 31*).

```
1. getOrderTitle(order: Order): string {
2.   const parts = order.items.map(i => `${i.quantity}x ${i.menuItem.name}`);
3.   if (parts.length <= 2) return parts.join(', ');
4.   return `${parts[0]}, ${parts[1]} (+${parts.length - 2} weitere)`;
5. }
```

Listing 31: Frontend - dynamische Erstellung des Namens der Bestellung

Diese Darstellung erleichtert dem Benutzer die schnelle Orientierung, ohne alle Detailinformationen sofort anzeigen zu müssen. Die Bestellübersicht ermöglicht dem Benutzer eine übersichtliche Einsicht in seine Bestellungen. Durch die klare Trennung zwischen offenen und abgeschlossenen Bestellungen sowie den Einsatz von *QR-Codes* für aktive Bestellungen wird ein einfacher und nachvollziehbarer Bestellabschluss unterstützt. Die Umsetzung greift die im Rechercheteil beschriebenen Konzepte auf und integriert sie in eine klar strukturierte und wartbare *Frontend-Lösung*.

4.2.7 QR-Scan und Bestellabschluss - Inhaber

Der *QR-Scan* bildet den abschließenden Schritt im Bestellprozess und ist ausschließlich für den Inhaber vorgesehen. Während Benutzer Bestellungen lediglich auslösen, erfolgt der eigentliche Abschluss kontrolliert über diese administrative Oberfläche. Dadurch wird eine klare Trennung der Verantwortlichkeiten erreicht und sichergestellt, dass Bestellungen nur in einem definierten organisatorischen Kontext abgeschlossen werden können. Der *QR-Code* wird dabei nicht als sicherheitskritisches Element verstanden, sondern – wie im Rechercheteil beschrieben – ausschließlich als Transportmedium für eine eindeutige Referenz. Der Code enthält keine sensiblen Daten, sondern lediglich eine Kennung (z. B. *Order-<id>* oder *BalanceReq-<uuid>*). Sämtliche fachlichen und sicherheitsrelevanten Prüfungen (Existenz, Status, Berechtigung) erfolgen ausschließlich im Backend.

Initialisierung und Kameraverwaltung

Beim Öffnen der Scan-Seite werden alle verfügbaren Kameras des Endgeräts ermittelt. Falls bereits eine Kameraauswahl aus früheren Sitzungen existiert, wird diese automatisch wiederhergestellt. Dadurch wird der Scanvorgang beschleunigt und der Konfigurationsaufwand für den Inhaber reduziert (siehe *Listing 32*).

```
1. async initCameras(): Promise<void> {  
2.   this.cameras = await Html5Qrcode.getCameras(); // Geräte-Kameras ermitteln  
3.   this.selectedCameraId = localStorage.getItem('cameraId') ?? this.cameras[0]?.id; // gespeicherte Auswahl wiederverwenden  
4. }
```

Listing 32: Frontend Kamerazugriff für QR-Code Scan

Wenn keine gespeicherte Kamera vorhanden ist, wird eine geeignete Standardkamera (typischerweise die Rückkamera am Smartphone) gewählt und gespeichert.

Starten des QR-Scans

Der Scanvorgang wird bewusst manuell gestartet. Dadurch bleibt die Kontrolle beim Inhaber, und die Kamera wird nur bei Bedarf aktiviert. Beim Start werden Statusmeldungen gesetzt und der *QR-Reader* initialisiert (siehe *Listing 33*).

```
1. startScan(): void {  
2.   this.scanning = true;  
3.   this.qr.start(  
4.     this.selectedCameraId!,  
5.     { fps: 10, qrbox: 250 },  
6.     (text) => this.onScanSuccess(text)
```

```
7.   );  
8. }
```

Listing 33: Frontend - Definiere den Scan-Bereich und starte den Scan

Der *Scanner* wird über die Bibliothek *html5-qrcode* gestartet. Als *Callback* wird bei erfolgreicher Erkennung direkt *onScanSuccess()* aufgerufen.

Erkennen und Klassifizieren des QR-Codes

Nach der Erkennung wird der *QR-Code* zunächst bereinigt und anhand eines definierten Schemas klassifiziert. Dadurch können unterschiedliche *QR-Code-Typen* über dieselbe *Scan-Oberfläche* verarbeitet werden. Der Scanner wird dabei sowohl für Bestellungen als auch für Guthabenänderungen verwendet (siehe *Listing 34*).

```
1. private classifyCode(raw: string): 'order' | 'balance' | null {  
2.   if (raw.startsWith('Order-')) return 'order';  
3.   if (raw.startsWith('BalanceReq-')) return 'balance';  
4.   return null;  
5. }
```

Listing 34: Frontend - Unterscheidung zwischen Guthaben- und Bestellungscode

Schutz vor Mehrfach-Scans

Da *QR-Codes* bei aktiver Kamera häufig mehrfach hintereinander erkannt werden, wird ein Schutzmechanismus eingesetzt. Während der Verarbeitung eines Codes wird der Scanner temporär pausiert, um Mehrfachauslösungen zu verhindern.

Abschluss einer Bestellung per QR-Code

Nach dem erfolgreichen Scannen wird der erkannte *QR-Code* im Frontend zunächst validiert und klassifiziert. Anschließend wird die enthaltene Referenz an das Backend übermittelt. Um Mehrfachauslösungen durch wiederholte Kameraerkennungen zu verhindern, wird der *Scanner* während der Verarbeitung temporär pausiert. Erst nach Abschluss der *serverseitigen* Verarbeitung und Rückmeldung wird der Scanvorgang wieder freigegeben und der *Scanner* fortgesetzt (siehe *Listing 35*).

```
1. completeOrder(orderId: string): void {  
2.   this.pauseScanner();  
3.   this.orderService.completeOrder(orderId).subscribe(() => {  
4.     this.qr.resume();  
5.   });  
6. }
```

Listing 35: Frontend - Abschluss der Bestellung

Bestätigung einer Guthabenänderung per QR-Code

Neben dem Bestellabschluss unterstützt die *Scan-Oberfläche* auch die Bestätigung von Guthabenänderungen. Diese Funktion wird beispielsweise verwendet, um zuvor erzeugte Guthabenanforderungen eindeutig zu bestätigen. Auch hier dient der *QR-Code* ausschließlich als Referenz auf einen serverseitig gespeicherten Vorgang. Nach dem Scannen eines *Guthaben-QR-Codes* wird dieser an den entsprechenden *Admin-Service* weitergeleitet. (siehe *Listing 36*).

```

1. confirmBalance(requestId: string): void {
2.   this.pauseScanner();
3.   this.balanceService.confirm(requestId).subscribe(() => {
4.     this.qr.resume();
5.   });
6. }

```

Listing 36: Frontend - Bestätigung der Guthabenänderung

Der *QR-Scan* ermöglicht dem Inhaber sowohl den Abschluss von Bestellungen als auch die Bestätigung von Guthabenänderungen über eine einheitliche Oberfläche. *QR-Codes* werden dabei konsequent als reine Referenzen verwendet, während alle fachlichen und sicherheitsrelevanten Prüfungen im *Backend* erfolgen. Durch die klare Trennung von Benutzer- und Inhaberfunktionen sowie den kontrollierten *Scan-Ablauf* entsteht ein robuster, nachvollziehbarer und praxistauglicher Abschluss des Bestellprozesses (siehe Listing 37).

```

1. await this.html5.start(
2.   { deviceId: { exact: this.selectedCameraId! } },
3.   { fps: 12 },
4.   (decodedText) => this.onScanSuccess(decodedText, false),
5.   () => {}
6. );
7. private onScanSuccess(decodedText: string, fromManual: boolean) {
8.   const code = (decodedText || '').trim();
9.   if (!code || !/^Order-.\+$/i.test(code)) return;
10.  const h: any = this.html5 as any;
11.  h?.pause?.(true); // Mehrfach-Scans vermeiden
12.  this.adminOrders.completeByQrCode(code)
13.    .pipe(finally(() => h?.resume?.()))
14.    .subscribe();
15. }
16. }

```

Listing 37: Frontend - Gesamtdefinition und Ablauf der Bestellbestätigung

4.2.8 Auswertungen und Benutzerverwaltung (Ellmer)

Neben dem operativen Bestellablauf stellt das Frontend auch Funktionen zur administrativen Auswertung und Verwaltung bereit. Diese Bereiche richten sich ausschließlich an den Inhaber und unterstützen sowohl organisatorische als auch wirtschaftliche Entscheidungen. Ziel ist es, relevante Informationen übersichtlich darzustellen und administrative Eingriffe kontrolliert zu ermöglichen.

Das Kapitel gliedert sich in die Benutzerverwaltung sowie die statistische Auswertung von Bestellungen und Umsätzen.

Benutzerverwaltung

Die Benutzerverwaltung bietet dem Inhaber eine zentrale Übersicht aller registrierten Benutzer. Neben der Anzeige grundlegender Informationen können administrative Aktionen wie Sperren, Löschen oder das Zurücksetzen von Passwörtern durchgeführt werden. Dabei wird bewusst auf einfache, nachvollziehbare Interaktionen gesetzt.

Beim Laden der Seite werden alle Benutzer vom Backend abgerufen und im lokalen Zustand gespeichert.

Zur besseren Übersichtlichkeit unterstützt die Benutzerverwaltung eine *clientseitige* Suche sowie eine *Pagination*. Dadurch bleibt die Oberfläche auch bei größeren Benutzerzahlen übersichtlich und performant (siehe *Listing 38*).

```
1. updatePagination(): void {  
2.   const start = (this.currentPage - 1) * this.usersPerPage;  
3.   this.paginatedUsers = this.filteredUsers.slice(start, start + this.usersPerPage);  
4. }
```

Listing 38: Frontend - Verwendung eines Paginator für bessere Übersichtlichkeit

Administrative Aktionen werden bewusst mit zusätzlichen Prüfungen abgesichert. Beispielsweise ist das Löschen eines Benutzers nur möglich, wenn dessen Guthaben den Wert null hat. Dadurch werden fachliche Regeln bereits im Frontend sichtbar und verständlich kommuniziert.

Die Darstellung einzelner Benutzer erfolgt über eine eigene Unterkomponente. Dadurch bleibt die Hauptkomponente übersichtlich und die Benutzerlogik klar gekapselt.

Statistische Auswertung

Die Statistikseite dient der Analyse von Bestell- und Umsatzdaten innerhalb eines frei wählbaren Zeitraums. Ziel ist es, dem Inhaber sowohl eine schnelle Übersicht als auch detaillierte Einblicke in das Bestellverhalten zu ermöglichen.

Beim Initialisieren der Seite wird die *Angular-Lifecycle-Methode* *ngOnInit()* verwendet (siehe *Listing 39*). Standardmäßig verwendet das System einen Zeitraum von sieben Tagen für die die Statistikdaten anschließend geladen werden.

```
1. ngOnInit(): void {  
2.   this.startDate = weekAgo;  
3.   this.endDate = today;  
4.   this.loadStatistics();  
5. }
```

Listing 39: Frontend - Grundsätzliche Definition für statistische Auswertungen

Die Kennzahlen – Anzahl der Bestellungen, Umsatz, Anzahl der Kunden sowie der durchschnittliche Warenkorbwert – werden vollständig clientseitig aus den gelieferten Rohdaten berechnet.

Zur visuellen Darstellung dieser Kennzahlen wird die JavaScript-Bibliothek *Chart.js* eingesetzt. Die Diagramme werden im Frontend dynamisch erzeugt und reagieren auf Änderungen des ausgewählten Zeitraums sowie der Darstellungsform. Abhängig von der Datenmenge entscheidet die Anwendung, ob eine Tages- oder Wochenansicht verwendet wird, um eine übersichtliche Visualisierung zu gewährleisten. Der Benutzer kann zwischen Balken- und Liniendiagrammen wechseln, wodurch sowohl absolute Umsätze als auch zeitliche Trends nachvollziehbar dargestellt werden können.

Die notwendigen Diagrammdaten werden clientseitig aus den geladenen Bestellinformationen aggregiert. Dabei werden die Datumswerte als Achsenbeschriftungen verwendet und die Umsätze pro Zeitraum berechnet. Anschließend wird das Diagramm mit der aktuell gewählten Konfiguration neu initialisiert (siehe *Listing 40*).


```
1. this.chart = new Chart(ctx, {  
2.   type: this.selectedChartType,  
3.   data: { labels, datasets: [{ data, label: 'Umsatz (€)'} ] }  
4. });  
5.
```

Listing 40: Frontend - Auswahl unterschiedlicher Statistiken

Neben der grafischen Darstellung können die Finanzdaten auch in tabellarischer Form als PDF exportiert werden. Die PDF-Erstellung erfolgt vollständig im Frontend mithilfe der Bibliotheken *jsPDF* und *autoTable* (siehe Listing 41). Genauer dazu wurde bereits im [Rechercheteil Kapitel 2.8](#) beschrieben. Dadurch ist kein zusätzlicher Serverprozess notwendig und der Inhaber kann Auswertungen direkt aus der Anwendung heraus weiterverwenden oder archivieren. Die Tabelle enthält die aggregierten Umsätze und Bestellzahlen pro Zeitraum und wird automatisch an das *DIN-A4-Format* angepasst. Es können bei *jsPDF* zahlreiche Einstellungen bezüglich des Layouts und der Struktur getätigt werden.

```
1. exportFinancePDF(): void {  
2.   const doc = new jsPDF();  
3.   autoTable(doc, { body: this.financeRows.map(r => [r.label, r.orders, r.gross]) });  
4.   doc.save('finanzuebersicht.pdf');  
5. }
```

Listing 41: Frontend - Exportieren der Statistikdaten als PDF

Ein weiterer Schwerpunkt der statistischen Auswertung liegt also auf der Identifikation von Trend-Artikeln. Dazu werden alle Bestellpositionen innerhalb des gewählten Zeitraums zusammengeführt, nach Artikel aggregiert und anschließend nach Verkaufsmenge sowie Umsatz sortiert. Die meistverkauften Gerichte werden hervorgehoben dargestellt und ermöglichen dem Inhaber, beliebte Produkte schnell zu erkennen und sein Angebot entsprechend anzupassen.

Die Bereiche Benutzerverwaltung und Statistik ergänzen das Bestellsystem um zentrale administrative Funktionen. Während die Benutzerverwaltung eine kontrollierte Pflege von Benutzerkonten ermöglicht, liefern die statistischen Auswertungen wertvolle Einblicke in Umsatzentwicklung und Bestellverhalten.

4.2.9 Authentifizierung und E-Mail-Bestätigung

Dieses Kapitel beschreibt die Umsetzung der Benutzerverwaltung von der Registrierung bis zur laufenden Nutzung des Benutzerprofils. Ziel ist es, einen sicheren Zugang zur Anwendung zu ermöglichen und dem Benutzer gleichzeitig eine klare und verständliche Bedienung zu bieten. Sicherheitskritische Entscheidungen werden dabei konsequent im Backend getroffen, während das Frontend für Eingabeprüfung, Benutzerführung und Statusrückmeldungen zuständig ist.

Login und Authentifizierung

Der Login stellt den zentralen Einstiegspunkt in die Anwendung dar. Benutzer melden sich mit ihrer E-Mail-Adresse und ihrem Passwort an. Bereits im Frontend werden die Eingaben auf

Vollständigkeit und Plausibilität geprüft, um unnötige Serveranfragen zu vermeiden. Erst danach werden die Zugangsdaten an das Backend übermittelt (siehe *Listing 42*).

```
1. onSubmit(): void {
2.   if (this.loginForm.invalid) {
3.     this.loginForm.markAllAsTouched();
4.     return;
5.   }
6.   const { email, password } = this.loginForm.value;
7.   this.auth.login(email, password).subscribe({
8.     next: () => this.router.navigate(['/']),
9.     error: (err) => this.handleLoginError(err)
10.  });
11. }
12. }
```

Listing 42: Frontend - Routing bei Authentifizierung

Nach erfolgreicher Authentifizierung erstellt das Backend ein *JSON Web Token (JWT)*, das im Frontend gespeichert wird. Dieses *Token* identifiziert den Benutzer bei allen weiteren *API-Anfragen* und ermöglicht den Zugriff auf geschützte Bereiche der Anwendung.

Typische Fehlerfälle – wie falsche Zugangsdaten, eine noch nicht bestätigte E-Mail-Adresse oder ein gesperrter Account – werden vom Backend eindeutig zurückgemeldet. Das Frontend übersetzt diese Rückmeldungen in verständliche Hinweise, sodass der Benutzer jederzeit weiß, warum ein Login nicht möglich ist.

Registrierung und E-Mail-Bestätigung

Die Registrierung erfolgt über ein eigenes Formular. Abhängig vom gewählten Schultyp wird überprüft, ob die eingegebene E-Mail-Adresse einem vorgegebenen Format entspricht. Vor- und Nachname werden dabei automatisch aus der E-Mail-Adresse abgeleitet, was die Dateneingabe vereinfacht und Eingabefehler reduziert (siehe *Listing 43*).

```
1. const match = email.match(cfg.pattern);
2. if (match) {
3.   this.registerForm.patchValue({
4.     firstName: cfg.map(match).firstName,
5.     lastName: cfg.map(match).lastName
6.   }, { emitEvent: false });
7. }
```

Listing 43: Frontend - Automatisches Ausfüllen der Namen

Nach dem Absenden der Registrierung wird der Account noch nicht sofort aktiviert. Stattdessen versendet das Backend eine Bestätigungs-E-Mail mit einem einmaligen, zeitlich begrenzten Token. Erst nach dem Öffnen dieses Links gilt die E-Mail-Adresse als verifiziert und der Account wird freigeschaltet.

Die Bestätigungsseite liest das Token aus der URL aus, übermittelt es an das Backend und zeigt abhängig vom Ergebnis einen passenden Status an (siehe *Listing 44*).

```
1. this.route.queryParamMap.pipe(
2.   map(q => q.get('token')),
3.   switchMap(token => this.auth.verifyEmail(token!))
4. ).subscribe({
5.   next: () => this.setSuccess(),
6.   error: err => this.setError(err)
7. });
```

Listing 44: Frontend - Routing, nachdem die Registrierung bestätigt wurde

Fehler wie abgelaufene oder ungültige Tokens werden dabei explizit behandelt. Bei erfolgreicher Bestätigung erfolgt automatisch eine Weiterleitung zur Login-Seite, wodurch der Registrierungsprozess klar abgeschlossen wird.

Passwort-Reset und Passwortänderung

Der Passwort-Reset ist als eigenständiger, *tokenbasierter* Prozess umgesetzt, um die Sicherheit auch bei verlorenen Zugangsdaten zu gewährleisten. Der Ablauf ist strikt zweistufig und verhindert, dass Passwörter ohne Besitz der zugehörigen E-Mail-Adresse geändert werden können.

Im ersten Schritt gibt der Benutzer seine E-Mail-Adresse in ein separates Formular ein. Das Frontend prüft lediglich, ob eine formal gültige Adresse vorliegt, und übermittelt diese anschließend an das Backend (siehe *Listing 45*).

```
1. submit(): void {
2.   if (this.form.invalid) return;
3.   const email = this.form.value.email;
4.   this.auth.forgotPassword(email).subscribe(() => {
5.     this.done = true;
6.   });
7. }
```

Listing 45: Frontend - Passwort zurücksetzen

Das Backend erzeugt daraufhin ein kryptografisch zufälliges *Reset-Token*, speichert dieses zeitlich begrenzt (z. B. mit Ablaufdatum) und versendet einen personalisierten *Reset-Link* per E-Mail. Das Frontend erhält aus Sicherheitsgründen keinerlei Information darüber, ob ein Benutzerkonto tatsächlich existiert. Dadurch wird verhindert, dass E-Mail-Adressen durch das Formular abgefragt oder erraten werden können.

Wird der *Reset-Link* geöffnet, gelangt der Benutzer auf eine eigene *Reset-Seite*. Das enthaltene Token wird aus den *URL-Parametern* gelesen und im Frontend zwischengespeichert. Bereits an dieser Stelle wird geprüft, ob ein *Token* vorhanden ist – fehlt es, wird der Vorgang sofort abgebrochen.

Nach Eingabe und Bestätigung des neuen Passworts erfolgt eine clientseitige Plausibilitätsprüfung (Mindestlänge, Übereinstimmung beider Eingaben). Das Frontend sendet anschließend das neue Passwort gemeinsam mit dem *Token* an das Backend (siehe *Listing 46*).

```
1. submit(): void {
2.   if (!this.token || this.form.invalid) return;
3.
4.   const newPw = this.form.value.pw;
5.   this.auth.resetPassword(this.token, newPw).subscribe({
6.     next: () => this.success = true,
7.     error: () => this.error = 'Token ungültig oder abgelaufen.'
8.   });
9. }
```

Listing 46: Frontend - Neues Passwort und Token senden

Die eigentliche Sicherheitslogik liegt vollständig *serverseitig*:

- Überprüfung, ob das Token existiert und noch gültig ist
- Zuordnung des Tokens zu einem Benutzerkonto

- Hashen des neuen Passworts
- Entwertung des Tokens nach erfolgreicher Verwendung

Das Frontend verarbeitet ausschließlich das Ergebnis und stellt dem Benutzer den Status verständlich dar.

Benutzerprofil und Kontoverwaltung

Das Benutzerprofil dient als zentrale Anlaufstelle für persönliche Informationen und benutzerspezifische Aktivitäten. Beim Aufruf der Profilseite werden sowohl die Stammdaten des Benutzers als auch ergänzende Informationen wie bestehende Bestellungen oder Guthabenbewegungen aus dem Backend geladen und im Frontend zusammengeführt. Innerhalb dieses Bereichs kann der Benutzer sein Guthaben erhöhen oder leeren sowie die bisherige Guthabenhistorie einsehen. Änderungen am Profil, beispielsweise das Aktualisieren der Klassenangabe, werden zunächst im Frontend erfasst und erst nach erfolgreicher Validierung durch das Backend übernommen. Auf diese Weise bleibt der Datenbestand konsistent, selbst wenn während der Bearbeitung Eingaben fehlerhaft sind oder der Vorgang abgebrochen wird.

4.2.10 Zugriffskontrollen und Sicherheit (Ellmer)

In diesem Abschnitt wird detailliert beschrieben, wie *Zugriffskontrollen*, Rollensteuerung und *sicherheitsrelevante Querschnittslogik* im Frontend umgesetzt sind. Dabei ist wichtig, den Verantwortungsbereich klar einzugrenzen: Das Frontend ist keine *Sicherheitsinstanz* im engeren Sinne, sondern fungiert als steuernde und vorbereitende Schicht. Es sorgt dafür, dass Benutzer logisch geführt werden, nur zulässige Navigationspfade sehen und typische Fehlerfälle konsistent behandelt werden. Die verbindliche Durchsetzung von *Berechtigungen*, *Rollen* und *Policies* erfolgt ausschließlich im Backend.

Das Frontend trägt somit zur Sicherheit bei, indem es Fehlbedienungen reduziert, Zustände eindeutig hält und sicherheitsrelevante Informationen (z. B. *Tokens*) korrekt transportiert.

Routenbasierte Zugriffskontrolle (*Route Guards*)

Die erste Schutzebene ist das Angular-Routing. Jede Route kann über einen *canActivate-Guard* abgesichert werden. Im Projekt wird dafür ein *parametrisierbarer Auth-Guard* verwendet, die zwei zentralen Prüfungen durchführt:

1. *Authentifizierungsprüfung*: Ist ein *JWT* im lokalen Speicher vorhanden? Falls nein, wird der Benutzer sofort auf die Login-Seite umgeleitet.
2. *Autorisierungsprüfung*: Falls Rollen definiert sind, wird geprüft, ob die Rolle des eingeloggt Benutzers erlaubt ist. Falls nicht, erfolgt eine Weiterleitung auf die Startseite.

Beide Überprüfungen sind im *Listing 47* ersichtlich.

```
1. if (!auth.isLoggedIn()) {  
2.   router.navigate(['/Login']);  
3.   return false;  
4. }  
5.
```

```
6. if (allowedRoles && !allowedRoles.includes(user.role)) {
7.   router.navigate(['//']);
8.   return false;
9. }
```

Listing 47: Frontend - Definition freier Routen

Die *Guards* werden direkt in der *Routen-Konfiguration* eingesetzt. Dadurch ist auf einen Blick ersichtlich, welche Rolle welche Seite betreten darf. Kritische Funktionen wie *Scanner*, *Statistiken* oder *Benutzerverwaltung* sind explizit auf *INHABER* und *ADMIN* beschränkt, während Standardfunktionen auch für *KUNDE* freigegeben sind. Dadurch wird der Zugriffsschutz zentral definiert und die Komponenten bleiben frei von Sicherheitslogik. Änderungen der Rollen und Berechtigungen lassen sich nur an dieser Stelle durchführen.

Token-Handling bei API-Aufrufen (*Auth Interceptor*)

Damit das *Backend API-Anfragen* eindeutig einem Benutzer zuordnen kann, muss bei jedem geschützten *Request* das *JWT* mitgesendet werden. Dies geschieht über einen *HTTP-Interceptor*, der sich automatisch vor jeden ausgehenden *Request* schaltet (siehe Listing 48).

```
1. const token = this.auth.getToken();
2. if (!token) return next.handle(req);
3.
4. return next.handle(
5.   req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
6. );
```

Listing 48: Frontend - Erstellung der Tokens

Der *Interceptor* liest das *Token* aus dem *AuthService* und ergänzt den *HTTP-Header*. Existiert kein *Token*, wird der *Request* unverändert weitergeleitet.

Ein *Interceptor* ist sinnvoll, weil *Authentifizierung* ein Querschnittsthema ist. Dadurch werden fachliche Services übersichtlicher und testbar, während Änderungen (z. B. ein anderes *Auth-Verfahren*) zentral umgesetzt werden können.

Automatisches Logout bei ungültigem oder abgelaufenem Token

JWTs besitzen eine begrenzte Gültigkeit oder können serverseitig invalidiert werden. In diesen Fällen antwortet das Backend mit einem *HTTP-Status 401 (Unauthorized)*. Um inkonsistente Zustände zu vermeiden, reagiert das Frontend darauf automatisch (siehe Listing 49).

```
1. @Injectable()
2. export class AutoLogoutInterceptor implements HttpInterceptor {
3.   constructor(private auth: AuthService) {}
4.
5.   intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
6.     return next.handle(req).pipe(
7.       catchError((err: HttpResponse) => {
8.
9.         // 401 → Token ungültig oder abgelaufen
10.        if (err.status === 401) {
11.          this.auth.logoutIfNeeded();
12.        }
13.
14.        return throwError(() => err);
15.      })
16.    );
17.  }
18. }
```

Listing 49: Frontend - Automatischer Logout bei falschen Token

Der *Interceptor* erkennt den *Statuscode*, löscht *Token* und Benutzerdaten aus dem lokalen Speicher und leitet – sofern nötig – auf die Login-Seite weiter. Dabei wird bewusst vermieden, mehrfach zu navigieren, falls der Benutzer sich bereits auf der Login-Seite befindet.

Der automatische Logout verhindert so inkonsistente „halb eingeloggte“ Zustände und ermöglicht eine zentrale und einheitliche Fehlerbehandlung.

Policy-basierte Weiterleitungen (fachliche Zugriffsbeschränkungen)

Neben Authentifizierungsfehlern existieren fachliche Zugriffsbeschränkungen. Ein typisches Beispiel ist ein Benutzer, der zwar erfolgreich eingeloggt ist, dessen Profil jedoch unvollständig ist (z. B. fehlende Klassenangabe). In solchen Fällen entscheidet bewusst das Backend über die Zugriffsbeschränkung und antwortet mit dem *HTTP-Status 403 (Forbidden)* sowie einem eindeutigen Fehlercode wie *USER_PROFILE_UPDATE_REQUIRED* (siehe Listing 50).

```

1. @Injectable()
2. export class PolicyRedirectInterceptor implements HttpInterceptor {
3.   constructor(private router: Router) {}
4.   intercept(req: HttpRequest<any>, next: HttpHandler) {
5.     return next.handle(req).pipe(
6.       catchError((err: HttpResponse) => {
7.         const code = (err as any)?.error?.error;
8.
9.         // 403 + fachlicher Fehlercode → gezielte Weiterleitung
10.        if (err.status === 403 && code === 'USER_PROFILE_UPDATE_REQUIRED') {
11.          this.router.navigate(['/user-profile']);
12.        }
13.        return throwError(() => err);
14.      })
15.    );
16.  }
17. }
18. }
19.

```

Listing 50: Frontend - Einsatz des HTTP-Interceptors

Der *PolicyRedirectInterceptor* erkennt fachliche Sperren anhand des *Statuscodes* und des *Fehlercodes* und leitet den Benutzer gezielt auf die Profilseite weiter. Dadurch wird verhindert, dass der Benutzer in einer nicht lösbaren Situation verbleibt oder unverständliche Fehlermeldungen erhält.

Ergänzend dazu existiert ein *Guard*, der lokal prüft, ob ein Benutzer als *blocked* markiert ist. Ist dies der Fall, sind ausschließlich Profilfunktionen erreichbar (siehe Listing 51).

```

1. @Injectable({ providedIn: 'root' })
2. export class ClassPolicyGuard implements CanActivate {
3.   constructor(private auth: AuthService, private router: Router) {}
4.
5.   canActivate(): boolean | UrlTree {
6.     const user = this.auth.getCurrentUser();
7.
8.     if (!this.auth.isLoggedIn() || !user) return true;
9.
10.    // blockierte Benutzer → nur Profilseite erlauben
11.    if ((user as any).blocked) {
12.      return this.router.createUrlTree(['/user-profile']);
13.    }
14.
15.    return true;

```

```
16.   }  
17. }
```

Listing 51: Frontend - Verwendung von Route Guard zur Sicherung geschützter Routen

Der *Guard* sorgt auf diese Weise dafür, dass blockierte Benutzer ausschließlich auf Seiten zugreifen können, auf denen sie den blockierenden Zustand selbst auflösen können.

Rollensteuerung in Navigation und UI

Zusätzlich zur Routenabsicherung wird die Benutzeroberfläche dynamisch an die Rolle des eingeloggten Benutzers angepasst. Dafür existiert eine zentrale *NAV_CONFIG*, in der pro Navigationseintrag festgelegt ist, welche Rollen diesen sehen dürfen (siehe *Listing 52*).

```
1. export const NAV_CONFIG = [  
2.   { label: 'Start', route: '/', roles: ['KUNDE', 'INHABER', 'ADMIN'] },  
3.   { label: 'Scanner', route: '/scanner', roles: ['INHABER', 'ADMIN'] },  
4.   { label: 'Benutzer', route: '/users', roles: ['ADMIN'] }  
5. //usw.  
6. ];
```

Listing 52: Frontend - Routen und Rollen Definition

Die Navigation filtert diese Konfiguration anhand der effektiven Rolle. Für Administratoren existiert dabei ein Sonderfall: Sie können in der *UI* zwischen einer User- und einer Management-Sicht wechseln. Diese effektive Rolle beeinflusst ausschließlich Darstellung und Navigation - nicht die tatsächlichen Berechtigungen, die weiterhin serverseitig geprüft werden.

Dadurch bleibt die Benutzeroberfläche übersichtlich, Fehlbedienungen werden reduziert und UI-Komfort wird klar von sicherheitsrelevanter Logik getrennt.

Zentrale Registrierung der Sicherheitsmechanismen

Alle sicherheitsrelevanten *Interceptors* werden zentral in der App-Konfiguration registriert. Dadurch ist garantiert, dass sie für sämtliche *HTTP-Kommunikation* aktiv sind und in einer definierten Reihenfolge ausgeführt werden (siehe *Listing 53*).

```
1. // appConfig  
2. providers: [  
3.   { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },  
4.   { provide: HTTP_INTERCEPTORS, useClass: PolicyRedirectInterceptor, multi: true },  
5.   { provide: HTTP_INTERCEPTORS, useClass: AutoLogoutInterceptor, multi: true }  
6. ]
```

Listing 53: Frontend - Implementierung des HTTP-Interceptors

Die Reihenfolge ist dabei bewusst gewählt: Zuerst wird das Token gesetzt, anschließend fachliche Fehler behandelt und zuletzt Authentifizierungsfehler ausgewertet.

Das Frontend setzt Zugriffskontrollen über klar getrennte Mechanismen wie *Route Guards*, *HTTP-Interceptors* und eine rollenbasierte *UI* um, während sicherheitskritische Entscheidungen im Backend verbleiben. Durch saubere Zustandsverwaltung, konsistente Fehlerbehandlung und klare Nutzerführung entsteht ein wartbares und benutzerfreundliches Sicherheitskonzept.

4.3 Backend und API-Schnittstellen

Das Backend ist als *NestJS*-Anwendung umgesetzt und stellt die gesamte Geschäftslogik sowie sicherheitskritische Entscheidungen bereit. Das Frontend übernimmt zwar Eingabeprüfung und Benutzerführung, aber *Autorisierung, Rollen, Policies, Geld-/Order-Logik und Datenintegrität* werden konsequent serverseitig erzwungen. Die Architektur folgt dabei einem klaren Muster: *Controller* definieren Endpunkte, *Services* kapseln Logik und Datenzugriffe, und *Guards/Pipes* sichern *Requests* zentral ab.

4.3.1 Einstiegspunkt und globale Konfiguration (Ellmer)

Der *Einstiegspunkt* (*entry point*) der Anwendung befindet sich in der *bootstrap()*-Funktion (siehe *Listing 54*). Dort werden alle globalen Querschnittsfunktionen initialisiert, die für den stabilen und sicheren Betrieb der API erforderlich sind. Dazu zählen unter anderem *CORS*, *Request-Parsing*, *globale Guards*, *Validierung* sowie die automatische *API-Dokumentation* mittels *Swagger*.

```
1. async function bootstrap() {
2.   const app = await NestFactory.create(AppModule);
3.
4.   app.enableCors({
5.     origin: true,
6.     credentials: true,
7.     methods: ['GET', 'POST', 'PATCH', 'PUT', 'DELETE', 'OPTIONS'],
8.     allowedHeaders: ['Content-Type', 'Authorization'],
9.   });
10.
11.   app.use(express.json({ limit: '2mb' }));
12.   app.use(express.urlencoded({ extended: true }));
13.
14.   app.setGlobalPrefix('api');
15.
16.   app.useGlobalGuards(new ClassPolicyGuard(app.get(UsersService)));
17.   app.useGlobalPipes(
18.     new ValidationPipe({
19.       whitelist: true,
20.       transform: true,
21.       forbidNonWhitelisted: true,
22.     }),
23.   );
24.
25.   await app.listen(3000);
26. }
```

Listing 54: Backend - Entry-Points und Zugriffsrechte

Durch die Verwendung eines globalen *API-Präfixes* (*api*) wird eine klare Trennung zwischen *Backend-Endpunkten* und potenziellen statischen Ressourcen erreicht. Die globale *Validation-Pipe* stellt sicher, dass nur definierte und gültige *DTO-Strukturen* verarbeitet werden.

4.3.2 Authentifizierung mit JWT

Die Authentifizierung erfolgt über *JSON Web Tokens (JWT)*. Nach erfolgreichem Login erhält der Client ein signiertes Token, das bei jeder weiteren Anfrage im *Authorization*-Header mitgeführt wird (siehe *Listing 55*). Das Backend prüft dieses Token zentral über einen eigenen Guard.

```

1. @Injectable()
2. export class JwtAuthGuard implements CanActivate {
3.   canActivate(ctx: ExecutionContext): boolean {
4.     const req = ctx.switchToHttp().getRequest();
5.
6.     if (req.method === 'OPTIONS') return true;
7.
8.     const header = req.headers?.authorization ?? '';
9.     const [type, token] = header.split(' ');
10.
11.    if (type !== 'Bearer' || !token) {
12.      throw new UnauthorizedException('AUTH_REQUIRED');
13.    }
14.    const secret = process.env.JWT_SECRET;
15.    req.user = jwt.verify(token, secret) as AuthJwtPayload;
16.    return true;
17.  }
18. }

```

Listing 55: Backend - Erstellung und Kontrolle des Tokens

Der *Guard* validiert dabei nicht nur die *Signatur des Tokens*, sondern stellt auch sicher, dass ungültige oder fehlende *Tokens* frühzeitig abgefangen werden. Das validierte *Payload-Objekt* wird anschließend unter *req.user* abgelegt und steht allen nachfolgenden Komponenten zur Verfügung.

4.3.3 Rollenbasierte Autorisierung

Zusätzlich zur reinen Authentifizierung implementiert das Backend ein rollenbasiertes Zugriffskonzept. Rollen wie *KUNDE*, *INHABER* oder *ADMIN* werden direkt im *JWT* transportiert und serverseitig ausgewertet (siehe *Listing 56*).

```

1. export const Roles = (...roles: Role[]) => SetMetadata(ROLES_KEY, roles);
2. @Injectable()
3. export class RolesGuard implements CanActivate {
4.   constructor(private readonly reflector: Reflector) {}
5.
6.   canActivate(ctx: ExecutionContext): boolean {
7.     const allowed = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [
8.       ctx.getHandler(),
9.       ctx.getClass(),
10.    ]);
11.    if (!allowed || allowed.length === 0) return true;
12.
13.    const req = ctx.switchToHttp().getRequest();
14.    const role = req.user?.role;
15.
16.    if (!role || !allowed.includes(role)) {
17.      throw new ForbiddenException('INSUFFICIENT_ROLE');
18.    }
19.
20.    return true;

```

21. }

Listing 56: Backend - Kontrolle des rollenbasierten Zugriffs

Dieses Vorgehen ermöglicht eine klare, deklarative Zugriffsbeschränkung direkt an den jeweiligen Endpunkten, ohne Geschäftslogik in die Controller selbst zu verlagern.

4.3.4 Fachliche Zugriffsbeschränkung

Neben technischen Authentifizierungs- und Autorisierungsmechanismen existiert eine fachliche Zugriffsbeschränkung: Benutzer müssen ihre *class*-Information regelmäßig aktualisieren. Ist diese Information abgelaufen, wird der Benutzer automatisch blockiert. Damit wird sichergestellt, dass mit der Zeit *verwaahrloste* Benutzerkonten zuerst deaktiviert und anschließend gelöscht werden.

Diese Policy wird zentral durch den *ClassPolicyGuard* umgesetzt und global angewendet (siehe *Listing 57*).

```

1. @Injectable()
2. export class ClassPolicyGuard implements CanActivate {
3.   constructor(private readonly users: UsersService) {}
4.
5.   async canActivate(context: ExecutionContext): Promise<boolean> {
6.     const req = context.switchToHttp().getRequest();
7.
8.     if (req.method === 'OPTIONS') return true;
9.     const jwtUser = req.user;
10.    if (!jwtUser?.id && !jwtUser?.sub) return true;
11.
12.    const userId = String(jwtUser?.id ?? jwtUser?.sub);
13.    const dbUser = await this.users.enforceClassPolicy(userId);
14.
15.    if (!dbUser || !dbUser.blocked) return true;
16.    const path = String(req.originalUrl || '');
17.    const allowed = path.startsWith('/api/users/me/class');
18.    if (allowed) return true;
19.    throw new ForbiddenException({
20.      error: 'USER_PROFILE_UPDATE_REQUIRED',
21.      message: 'Bitte Klasse aktualisieren.',
22.    });
23.  }

```

Listing 57: Backend - Prüfen nach geänderter Klasse - Guard

Die eigentliche Durchsetzung erfolgt im *UserService*. Dort wird geprüft, ob die hinterlegte Klasse abgelaufen ist. Falls ja, wird die Klasse serverseitig entfernt und der Benutzer gesperrt (siehe *Listing 58*).

```

1. async enforceClassPolicy(userId: string) {
2.   const res = await this.db.query(
3.     `SELECT id, blocked, class_updated_at FROM app.users WHERE id = ${1}`,
4.     [userId],
5.   );
6.   const expired = isClassExpired(res.rows[0].class_updated_at);

```

```

8. if (!expired) return res.rows[0];
9.
10. await this.db.query(
11.   `UPDATE app.users
12.   SET class = NULL, class_updated_at = NULL, blocked = true
13.   WHERE id = ${1}`,
14.   [userId],
15. );
17. return { blocked: true };
18. }

```

Listing 58: Backend - Prüfen nach geänderter Klasse - UserService

4.3.5 Benutzerverwaltung und Profilzugriffe

Das *Users-Modul* stellt sämtliche Endpunkte zur Verwaltung des eigenen Benutzerkontos sowie administrative Funktionen bereit. Für den eingeloggten Benutzer existieren sogenannte *me-Endpunkte*, die bewusst vom allgemeinen *CRUD*-Zugriff getrennt sind (siehe Listing 59).

```

1. @UseGuards(JwtAuthGuard)
2. @Get('me/profile')
3. getMyProfile(@Req() req: any) {
4.   const userId = String(req.user?.id ?? req.user?.sub);
5.   return this.userService.getMyProfile(userId);
6. }

```

Listing 59: Backend - Endpoint für Zugriff auf eigene Daten

4.3.6 Bestellprozess und Datenkonsistenz (Orders)

Der Bestellprozess bildet die zentrale Geschäftslogik der Anwendung. Das Backend stellt dabei sicher, dass Bestellungen nur unter gültigen Rahmenbedingungen erstellt werden können wie zum Beispiel, wenn das Bestellen aktiviert ist, die gewählten Speisen aktuell bestellbar sind und das Guthaben des Benutzers ausreicht. Kritische Schritte werden transaktional umgesetzt, damit parallele *Requests* (z. B. zwei Bestellungen gleichzeitig) nicht zu *inkonsistenten* Zuständen führen.

Eine wesentliche fachliche Steuerung ist das zentrale Feature-Flag *ordering_enabled*. Dadurch kann der Betreiber Bestellungen kurzfristig deaktivieren (z. B. nach Bestellschluss oder bei Ausfall in der Küche) (siehe Listing 60). Diese Entscheidung trifft ausschließlich das Backend.

```

1. @Injectable()
2. export class AppSettingsService {
3.   constructor(@Inject(PG_POOL) private readonly db: Pool) {}
4.
5.   async getOrderingEnabled(): Promise<boolean> {
6.     const res = await this.db.query(
7.       `SELECT value FROM app.app_settings WHERE key = 'ordering_enabled' LIMIT 1`,
8.     );
9.     if (res.rowCount === 0) return true;
10.    const v = res.rows[0]?.value;
11.    if (typeof v === 'boolean') return v;
12.    if (typeof v === 'string') return v === 'true';
13.    if (v && typeof v === 'object') {
14.      if (typeof v.enabled === 'boolean') return v.enabled;
15.    }
16.    return true;
17.  }
18.
19.  async setOrderingEnabled(enabled: boolean): Promise<{ ok: true; orderingEnabled: boolean }> {
20.    await this.db.query(

```

```

21. `INSERT INTO app.app_settings (key, value, updated_at)
22. VALUES ('ordering_enabled', $1::jsonb, now())
23. ON CONFLICT (key) DO UPDATE SET value = EXCLUDED.value, updated_at = now()`,
24. [JSON.stringify(enabled)],
25. );
26.
27. return { ok: true, orderingEnabled: enabled };
28. }
29. }

```

Listing 60: Backend - AppSettings zur Deaktivierung des Bestellfeatures

Die Order-Erstellung ist bewusst so umgesetzt, dass die Abbuchung sofort beim Anlegen der Bestellung erfolgt. Damit entfällt eine komplizierte „Reserve“-Logik auf Datenbankebene und der Kontostand bleibt eindeutig interpretierbar.

Der Ablauf in *createForUser()*:

- Prüfen, ob Bestellen aktiv ist (*settings.getOrderingEnabled()*)
- Benutzer-Datensatz sperren (*FOR UPDATE*)
- Order mit *status='open'* anlegen
- Items validieren (Existenz, Verfügbarkeit, Menge, optional: aktiver Menüplan)
- *totalPrice* berechnen
- Guthaben prüfen und anschließend abbuchen

Die Zeile *FOR UPDATE* sperrt den Benutzer-Datensatz innerhalb der Transaktion. Dadurch wird verhindert, dass zwei parallele Bestellungen denselben Guthabenstand gleichzeitig verwenden (siehe Listing 61).

```

1. await client.query('BEGIN');
2.
3. const uRes = await client.query(
4.   `SELECT id, balance
5.   FROM app.users
6.   WHERE id = $1
7.   FOR UPDATE`,
8.   [userId],
9. );
10. if (uRes.rowCount === 0) throw new NotFoundException('USER_NOT_FOUND');
11.
12. const orderRes = await client.query(
13.   `INSERT INTO app.orders (user_id, total_price, created_at, status)
14.   VALUES ($1, 0, NOW(), 'open')
15.   RETURNING id`,
16.   [userId],
17. );
18.
19. const orderId = String(orderRes.rows[0].id);

```

Listing 61: Backend - Sicherstellen von DB Prinzipien

Erst nachdem die Order vollständig und korrekt berechnet wurde, wird geprüft, ob *balance* ausreicht. Bei Erfolg erfolgt die Abbuchung *atomar* innerhalb derselben Transaktion (siehe Listing 62).

```

1. const balance = Number(uRes.rows?.[0]?.balance ?? 0);
2. if (balance < total) throw new ForbiddenException('INSUFFICIENT_FUNDS');
3.
4. await client.query(
5.   `UPDATE app.users
6.   SET balance = balance - $2
7.   WHERE id = $1`,

```

```

8.  [userId, total],
9.  );
10.
11. await client.query('COMMIT');

```

Listing 62: Backend - Sicherstellen von DB - Prinzipien - Rollback

Im Fehlerfall wird die Transaktion zurückgerollt (*ROLLBACK*). Dadurch entstehen weder „leere“ Orders noch inkonsistente Guthabenstände.

Die Methode *insertItemsAndComputeTotal()* prüft pro Position zentrale Regeln wie *menuItemId*, Menge und Verfügbarkeit. Optional wird auch geprüft, ob das Item im aktuell aktiven Menüplan enthalten ist (siehe Listing 63). Beispielauszug: *Menge + optionaler Menüplan + Verfügbarkeit*

```

1. const menuItemId = String(it.menuItemId ?? '').trim();
2. if (!menuItemId) throw new BadRequestException('MISSING_MENU_ITEM_ID');
3.
4. const qty = Number(it.quantity ?? 0);
5. if (!Number.isFinite(qty) || qty <= 0) throw new BadRequestException('INVALID_QUANTITY');
6.
7. if (activeMealPlanId) {
8.   await this.assertItemInActiveMenu(client, activeMealPlanId, menuItemId);
9. }
10.
11. const mRes = await client.query(
12.   `SELECT id, price, available
13.   FROM app.menu_items
14.   WHERE id = $1
15.   LIMIT 1`,
16.   [menuItemId],
17. );
18. if (mRes.rowCount === 0) throw new NotFoundException(`MENU_ITEM_NOT_FOUND:${menuItemId}`);
19. if (mRes.rows[0]?.available === false) throw new ForbiddenException(`MENU_ITEM_NOT_AVAILABLE:${menuItemId}`);

```

Listing 63: Backend - Ablauf und Prüfen der Bestellungen innerhalb der Transaktion

Damit gilt: Selbst wenn ein Client versucht, nicht erlaubte Items oder Mengen zu senden, verhindert das Backend zuverlässig die Erstellung ungültiger Orders.

4.3.7 QR-Abschluss und Statuswechsel

Nach dem Anlegen einer Bestellung verbleibt diese zunächst im Zustand *status='open'*. In diesem Zustand kann die Bestellung beispielsweise in der Küche vorbereitet und anschließend bei der Ausgabe abgeschlossen werden. Der Abschluss erfolgt über einen *QR-Code*, der die *Order-ID* enthält und serverseitig eindeutig validiert wird. Wichtig ist hierbei: Da die Abbuchung bereits beim Erstellen der Bestellung erfolgt (siehe *createForUser()*), führt der *QR-Scan* nicht zu einer erneuten Abbuchung, sondern ausschließlich zu einem *Statuswechsel* sowie

dem Markieren der Positionen als ausgeliefert (*delivered=true*). Dadurch bleibt der Ablauf für die Ausgabe einfach und robust: *QR scannen → Bestellung schließen → Positionen als geliefert markieren*. Die QR-Validierung erfolgt über ein striktes Muster, das sicherstellt, dass nur erwartete Codes akzeptiert werden (siehe Listing 64).

```
1. const c = String(code || '').trim();
2. const m = /^Order-([0-9a-fA-F-]{36})$/i.exec(c);
3. if (!m) return { ok: false };
4. const orderId = String(m[1]);
5.
```

Listing 64: Backend - Abschluss einer Order

Der Abschluss wird *transaktional* umgesetzt. Die Bestellung wird dabei per *FOR UPDATE* gesperrt, sodass parallele Scans (z. B. versehentlich doppelt) keinen inkonsistenten Zustand erzeugen. Ist die Bestellung bereits geschlossen, wird idempotent *ok: true* zurückgegeben (siehe Listing 65).

```
1. const orderRes = await client.query(
2.   `SELECT id, status FROM app.orders WHERE id = $1 FOR UPDATE`,
3.   [orderId],
4. );
5. if (orderRes.rowCount === 0) throw new NotFoundException('ORDER_NOT_FOUND');
6.
7. if (String(orderRes.rows[0].status) === 'closed') {
8.   await client.query('COMMIT');
9.   return { ok: true, orderId };
10. }
```

Listing 65: Backend - Transaktionaler Abschluss von Orders

Der eigentliche Abschluss setzt den Status auf *closed* und markiert alle zugehörigen *order_items* als ausgeliefert. Erst danach wird *committed* und der aktuelle Status in der Datenbank gesetzt (siehe Listing 66).

```
1. await client.query(`UPDATE app.orders SET status = 'closed' WHERE id = $1`, [orderId]);
2.
3. await client.query(
4.   `UPDATE app.order_items SET delivered = true WHERE order_id = $1`,
5.   [orderId],
6. );
7.
8. await client.query('COMMIT');
9. return { ok: true, orderId };
10.
```

Listing 66: Backend - Der Abschluss setzt die Bestellung als delivered

Durch diese Umsetzung ist gewährleistet, dass der Abschluss einer Bestellung atomar erfolgt: Entweder werden Status und Items gemeinsam aktualisiert oder bei Fehlern vollständig zurückgerollt.

4.3.8 Guthabenverwaltung und QR-Requests

Neben dem Bestellprozess stellt die Guthabenverwaltung eine weitere zentrale Geschäftslogik der Anwendung dar. Ziel dieser Komponente ist es, jede Veränderung des Guthabens kontrolliert, nachvollziehbar und revisionssicher durchzuführen. Aus diesem Grund ist es Benutzern nicht möglich, ihr Guthaben direkt zu verändern. Stattdessen erfolgt jede Änderung über einen *QR-basierten* Request, der anschließend durch berechtigte Rollen bestätigt werden muss.

Das Backend stellt dabei sicher, dass folgende fachliche Regeln strikt eingehalten werden:

- Guthaben kann nur positiv erhöht werden (*Add-Request*)
- ein vollständiges Zurücksetzen des Guthabens (*Flush*) ist nur ohne offene Bestellungen erlaubt
- jeder QR-Request kann nur einmal verwendet werden (*is_used*)
- jede Guthabenänderung wird vollständig und nachvollziehbar in *balance_logs* protokolliert

Diese Regeln werden ausschließlich serverseitig umgesetzt und können nicht durch manipulierte Client-Anfragen umgangen werden.

Erzeugen eines Add-Guthaben-Requests

Beim Erstellen eines *Add-Requests* wird zunächst die *Class-Policy* erzwungen. Dadurch wird sichergestellt, dass auch für Guthabenfunktionen nur Benutzer mit einem gültigen, nicht abgelaufenen Profil agieren können. Anschließend erfolgen Validierungen auf Betrag und Benutzerstatus (siehe *Listing 67*).

```
1. await this.enforceClassPolicy(userId);
2. const d = Number(delta);
3. if (!isFinite(d) || d <= 0) {
4.   throw new ForbiddenException('DELTA_MUST_BE_POSITIVE');
5. }
6. const uRes = await this.db.query(
7.   `SELECT id, blocked FROM app.users WHERE id = $1 LIMIT 1`,
8.   [userId],
9. );
10. if (uRes.rowCount === 0) throw new NotFoundException('USER_NOT_FOUND');
11. if (uRes.rows[0]?.blocked) throw new ForbiddenException('USER_BLOCKED');
```

Listing 67: Backend - Validierung beim Aufladen des Guthabens

Erst wenn diese Prüfungen erfolgreich sind, wird der *Request* fortgesetzt. Dabei wird kein Guthaben verändert, sondern lediglich ein *ausstehender Request* erzeugt (siehe *Listing 68*).

```
1. const res = await this.db.query(
2.   `INSERT INTO app.balance_change_requests (user_id, kind, delta)
3.   VALUES ($1, 'add', $2)
4.   RETURNING id`,
5.   [userId, d],
6. );
7. const id = String(res.rows[0].id);
8. const code = `BalanceReq-${id}`;
9. return { id, code, qrCodeUrl: this.generateQrCode(code) };
```

Listing 68: Backend - Requests werden während validierung zwischengespeichert

Der QR-Code enthält ausschließlich eine Referenz auf den *Request* und keine sensiblen Daten. Die eigentliche Geschäftslogik wird erst bei der Bestätigung ausgelöst.

Flush-Request: Guthaben zurücksetzen

Ein *Flush*-Request setzt das Guthaben eines Benutzers vollständig auf null. Diese Operation ist fachlich besonders sensibel und daher streng eingeschränkt. Sie ist nur erlaubt, wenn:

- das aktuelle Guthaben größer als *null* ist und
- keine offenen Bestellungen existieren.

Dadurch wird verhindert, dass Guthaben zurückgesetzt wird, während noch nicht abgeschlossene Bestellungen existieren (siehe *Listing 69*).

```
1. const reservedRes = await this.db.query(  
2.   `SELECT COALESCE(SUM(total_price), 0) AS reserved  
3.   FROM app.orders  
4.   WHERE user_id = $1 AND status = 'open'`,  
5.   [userId],  
6. );  
7. const reserved = Number(reservedRes.rows?.[0]?.reserved ?? 0);  
8. if (reserved > 0) {  
9.   throw new ForbiddenException('CANNOT_FLUSH_WITH_OPEN_ORDERS');  
10. }
```

Listing 69: Backend - Guthaben leeren

Bestätigung eines Guthaben-Requests (QR-Scan)

Die Bestätigung eines *Guthaben-Requests* stellt den kritischsten Schritt in der Guthabenverwaltung dar, da hier tatsächlich Geldstände verändert werden. Deshalb ist dieser Ablauf bewusst *transaktional* implementiert.

Zunächst wird der *Request*-Datensatz mit *FOR UPDATE* gesperrt. Dadurch ist garantiert, dass ein QR-Code nicht mehrfach oder parallel verarbeitet werden kann (siehe *Listing 70*).

```
1. await client.query('BEGIN');  
2. const rRes = await client.query(  
3.   `SELECT id, user_id, kind, delta, is_used  
4.   FROM app.balance_change_requests  
5.   WHERE id = $1  
6.   FOR UPDATE`,  
7.   [reqId],  
8. );  
9. if (rRes.rowCount === 0) {  
10.   throw new NotFoundException('REQUEST_NOT_FOUND');  
11. }  
12. if (rRes.rows[0].is_used) {  
13.   await client.query('COMMIT');  
14.   return { ok: true, alreadyUsed: true };  
15. }  
17. const newBalance =  
18.   rRes.rows[0].kind === 'flush'  
19.     ? 0  
20.     : currentBalance + Number(rRes.rows[0].delta);  
21.  
22. await client.query(  
23.   `UPDATE app.users SET balance = $2 WHERE id = $1`,  
24.   [rRes.rows[0].user_id, newBalance],  
25. );
```

Listing 70: Backend - Guthaben ändern

Audit-Logging und Abschluss der Transaktion

Nach der erfolgreichen Guthabenänderung wird ein *Log-Eintrag* erzeugt (siehe *Listing 71*). Dieser stellt sicher, dass jede finanzielle Veränderung langfristig nachvollziehbar bleibt.

```
1. await client.query(  
2.   `INSERT INTO app.balance_logs  
3.   (user_id, delta, balance_after, reason, ref_type, ref_id, actor)  
4.   VALUES ($1, $2, $3, $4, 'balance_request', $5, $6)`,  
5.   [  
6.     rRes.rows[0].user_id,  
7.     rRes.rows[0].kind === 'flush'  
8.     ? -currentBalance  
9.     : rRes.rows[0].delta,  
10.    newBalance,  
11.    rRes.rows[0].kind,  
12.    String(reqId),  
13.    `admin:${actorId}`,  
14.  ],  
15. );
```

Listing 71: Backend - Guthabenänderungen werden mitgeloggt

Abschließend wird der *Request* als verwendet markiert und die *Transaktion* abgeschlossen (siehe *Listing 72*).

```
1. await client.query(  
2.   `UPDATE app.balance_change_requests  
3.   SET is_used = true, used_at = now(), used_by = $2  
4.   WHERE id = $1`,  
5.   [reqId, String(actorId)],  
6. );  
7. await client.query('COMMIT');
```

Listing 72: Backend - Abschluss der Guthabentransaktion

Tritt während eines dieser Schritte ein Fehler auf, wird die gesamte *Transaktion* zurückgerollt. Dadurch entstehen weder inkonsistente Guthabenstände noch unvollständige *Log-Einträge*.

Rollenbasierte Absicherung

Die Bestätigung von *Guthaben-Requests* ist ausschließlich für den Inhaber bzw. den Admin vorgesehen. Der entsprechende *Endpunkt* ist daher zusätzlich rollenbasiert abgesichert (siehe *Listing 73*).

```
1. @UseGuards(JwtAuthGuard, RolesGuard)  
2. @Roles('ADMIN', 'INHABER')  
3. @Patch('balance/confirm')  
4. confirmBalance(@Req() req: any, @Body() dto: ConfirmBalanceRequestDto) {  
5.   const actor = String(req.user?.id ?? req.user?.sub);  
6.   return this.userService.confirmBalanceRequestByQr(dto.code, actor);  
7. }
```

Listing 73: Backend - Nur Inhaber und Admin kann Guthaben ändern

Durch diese Architektur entsteht eine robuste, sichere und nachvollziehbare Guthabenverwaltung, die auch unter parallelen Zugriffen konsistent bleibt und den fachlichen Anforderungen eines realen Abrechnungssystems gerecht wird.

4.3.9 Account-Lifecycle: Erstellung - Löschung

Das Löschen eines Benutzerkontos ist nur eingeschränkt möglich, da ein Konto mit verschiedenen wichtigen Daten verknüpft ist. Dazu zählen neben persönlichen Daten auch Guthaben, Transaktionen und Bestellungen. Um sicherzustellen, dass alle Daten korrekt bleiben und keine Abrechnungsfehler entstehen, kann ein Konto nur dann gelöscht werden, wenn keine offenen Verbindungen oder Abhängigkeiten mehr vorhanden sind.

Konkret ist ein Löschen nur zulässig, wenn:

- das Guthaben des Benutzers exakt 0 beträgt (*balance = 0*),
- keine offenen Bestellungen existieren (*status = 'open'*).

Diese Regeln werden serverseitig erzwungen und verhindern, dass finanzielle Daten verloren gehen oder Bestellungen ohne zugehörigen Benutzer verbleiben.

Zunächst wird geprüft, ob der Benutzer existiert und ob noch Guthaben vorhanden ist (siehe *Listing 74*). Ein Löschen bei positivem oder negativem Kontostand ist nicht erlaubt, da dies zu nicht nachvollziehbaren Abrechnungszuständen führen würde.

```
1. const uRes = await this.db.query(  
2.   `SELECT id, balance FROM app.users WHERE id = $1 LIMIT 1`,  
3.   [userId],  
4. );  
5. if (uRes.rowCount === 0) throw new NotFoundException('USER_NOT_FOUND');  
6. const balance = Number(uRes.rows[0]?.balance ?? 0);  
7. if (balance !== 0) throw new ForbiddenException('BALANCE_NOT_ZERO');  
8. const reservedRes = await this.db.query(  
9.   `SELECT COALESCE(SUM(total_price), 0) AS reserved`  
10.  `FROM app.orders WHERE user_id = $1 AND status = 'open'`,  
11.  [userId],  
12. );  
13. if (Number(reservedRes.rows?.[0]?.reserved ?? 0) > 0)  
14.   throw new ForbiddenException('HAS_OPEN_ORDERS');
```

Listing 74: Backend - Regeln zur Guthabenänderungen

Erst wenn beide Bedingungen erfüllt sind, werden abhängige Datensätze (z. B. *Guthaben-Requests, Logs, Bestellungen und Auth-Daten*) bereinigt und anschließend der Benutzer gelöscht. Diese restriktive Vorgehensweise reduziert die Komplexität im System und stellt sicher, dass der Datenbestand jederzeit konsistent bleibt.

4.3.10 E-Mail-Service (Express + Gmail API)

Registrierung, E-Mail-Verifizierung, Login und Passwort-Zurücksetzung sind in der Anwendung in einem eigenen Authentifizierungsbereich zusammengefasst, der über einen *separaten Express-Router* realisiert und im *NestJS-Gateway* unter */api/auth* eingebunden ist. Diese strukturelle Trennung wurde bewusst gewählt, da Authentifizierungsprozesse besonders sicherheitskritisch sind und sich sowohl fachlich als auch technisch von den übrigen *API-Endpunkten* unterscheiden.

Der Authentifizierungsbereich bündelt sämtliche Funktionen, die den *Lebenszyklus* eines Benutzerkontos betreffen. Dazu zählen:

- Registrierung neuer Benutzer,
- Verifikation von E-Mail-Adressen,
- Login und Session-Erneuerung,
- *Passwort-Reset*,
- Versand sicherheitsrelevanter E-Mails.

Tokenerzeugung und sichere Speicherung

Für E-Mail-Verifizierung und Passwort-Reset werden kryptografisch zufällige *Tokens* erzeugt (siehe *Listing 75*). Diese werden nicht im *Klartext* gespeichert, sondern als *Hash* in der Datenbank abgelegt. Dadurch kann selbst bei einem Datenbankleck kein gültiger *Token* missbraucht werden.

```
1. export function _createRandomToken(): string {
2.   return crypto.randomBytes(32).toString('hex');
3. }
4. export function _hashToken(token: string): string {
5.   return crypto.createHash('sha256').update(token).digest('hex');
6. }
```

Listing 75: Backend - Erzeugung von Tokens

Registrierung mit Pending-State und E-Mail-Verifizierung

Beim Aufruf von */register* wird zunächst kein aktiver Benutzeraccount erstellt. Stattdessen wird ein Eintrag in *pending_registrations* angelegt. Erst nach erfolgreicher E-Mail-Verifizierung wird ein echter Benutzeraccount erzeugt. Dieses zweistufige Verfahren verhindert, dass ungültige oder fremde E-Mail-Adressen sofort produktive Konten erzeugen (siehe *Listing 76*).

```
1. const verifyToken = createRandomToken();
2. const tokenHash = hashToken(verifyToken);
3. await pool.query(
4.   `INSERT INTO app.pending_registrations (email, payload, password_hash, token_hash, expires_at)
5.   VALUES ($1, $2, $3, $4, now() + interval '24 hours')`,
6.   [emailLower, payload, passwordHash, tokenHash],
7. );
8. const verifyLink = `${APP_BASE_URL}/verify-email?token=${verifyToken}`;
```

Listing 76: Backend - Speichern von Requests, die sich registrieren wollen

Login und Session-Erneuerung

Beim Login werden mehrere Bedingungen geprüft: Der Benutzer darf nicht blockiert sein, die E-Mail-Adresse muss verifiziert sein und das Passwort muss korrekt sein. Erst danach wird eine neue *Session-ID* erzeugt und ein *JWT* ausgestellt. Durch die Erneuerung der *Session-ID* werden alte Sitzungen automatisch ungültig (siehe *Listing 77*).

```
1. if (row.blocked) return res.status(403).json({ error: 'USER_BLOCKED' });
2. if (!row.email_verified) return res.status(403).json({ error: 'EMAIL_NOT_VERIFIED' });
3.
4. const ok = await bcrypt.compare(String(password), row.password_hash);
5. if (!ok) return res.status(401).json({ error: 'INVALID_CREDENTIALS' });
6.
7. const newSessionId = uuidv4();
8. await pool.query(
```

```
9.   `UPDATE app.auth_credentials
10.   SET auth_token = $1, last_used_at = now()
11.   WHERE user_id = $2`,
12.   [newSessionId, row.id],
13. );
```

Listing 77: Backend - Sessions laden und erneuern

Forgot/Reset Password: Schutz vor User Enumeration

Der *Endpoint* `/forgot-password` liefert aus Sicherheitsgründen immer `ok: true`, unabhängig davon, ob die E-Mail existiert. Dadurch kann ein Angreifer nicht durch Response-Unterschiede auf registrierte Accounts schließen.

Existiert der Benutzer, läuft der Prozess intern weiter: Es wird ein *Reset-Token* erzeugt, als Hash gespeichert und anschließend eine E-Mail mit *Reset-Link* versendet. Für den Client bleibt die Antwort jedoch gleich (`ok: true`), sodass der Ablauf nach außen nicht unterscheidbar ist.

Der *Reset* selbst erfolgt über `/reset-password`. Dabei wird das Token nicht im Klartext verarbeitet, sondern zuerst *gehasht* und dann gegen die Datenbank geprüft. Zusätzlich wird sichergestellt, dass das *Token* noch gültig ist und nicht bereits verwendet wurde (siehe Listing 78).

```
1. const tokenHash = hashToken(token);
3. const tok = await client.query(
4.   `SELECT id, user_id
5.   FROM app.auth_tokens
6.   WHERE token_type='PASSWORD_RESET'
7.   AND token_hash=$1
8.   AND used_at IS NULL
9.   AND expires_at > now()
10.  LIMIT 1`,
11.  [tokenHash],
12. );
13. if ((tok.rowCount ?? 0) === 0) {
14.   return res.status(400).json({ error: 'INVALID_OR_EXPIRED_TOKEN' });
15. }
```

Listing 78: Backend - Speicherung vom Token zur Authentifizierung

Ist das Token gültig, wird das neue Passwort mit *bcrypt* *gehasht* und gespeichert. Anschließend wird das *Reset-Token* als verwendet markiert, sodass derselbe Link nicht erneut eingesetzt werden kann. In Kombination mit *Token-Hashing* und einer begrenzten Ablaufzeit wird damit eine einmalige und sichere Verwendung des Reset-Links gewährleistet (siehe Listing 79).

```
1. const passwordHash = await bcrypt.hash(newPassword, 10);
2. await client.query(
3.   `UPDATE app.auth_credentials
4.   SET password_hash=$1, last_used_at=now()
5.   WHERE user_id=$2`,
6.   [passwordHash, tok.rows[0].user_id],
7. );
8. await client.query(
9.   `UPDATE app.auth_tokens SET used_at=now() WHERE id=$1`,
10.  [tok.rows[0].id],
11. );
```

Listing 79: Backend - Der Token und das Passwort wird *gehasht* gespeichert

Mailer über Gmail API (OAuth2)

Der Versand von Verifikations- und Passwort-Reset-E-Mails erfolgt über die *Gmail API* mit *OAuth2*. Dabei werden keine klassischen Mail-Passwörter gespeichert, sondern ein sicherer *API*-Zugriff verwendet. Das erhöht die Sicherheit und macht die Verwaltung einfacher, da Zugriffe jederzeit zentral erneuert oder gesperrt werden können.

Beim Start der Anwendung wird automatisch überprüft, ob der Mailversand funktioniert. Dafür wird ein sogenannter Health-Check durchgeführt. Dabei wird getestet, ob ein *OAuth2*-Client erstellt werden kann und ob ein gültiges Zugriffstoken verfügbar ist. Falls dies nicht funktioniert, steht der Mailversand nicht zur Verfügung. So werden Probleme früh erkannt und nicht erst beim ersten Versand.

Wichtig ist, dass die *Gmail API* keine einzelnen Felder wie „Empfänger“, „Betreff“ oder „Inhalt“ erwartet, sondern eine komplette E-Mail in einem standardisierten Format. Diese wird in der Funktion *buildRawEmail()* erzeugt. Die E-Mail besteht aus:

- Headern (z. B. Absender, Empfänger, Betreff)
- technischen Angaben (MIME-Header für Format und Zeichensatz)
- dem eigentlichen Inhalt (HTML-Body)

Der Aufbau der E-Mail ist in *Abbildung 22* dargestellt.

```
1 const headers = [  
2   `From : $ { from }`,  
3   `To : $ { to }`,  
4   `Subject : $ { subject }`,  
5   `MIME - Version : 1.0`,  
6   `Content - Type : text / html; charset = "UTF-8"`,  
7 ];  
8  
9 const mimeType =  
10 ` $ {  
11   headers.join('\r\n')  
12 }  
13 \r\n\r\n$ {  
14   html  
15 }  
16 `;
```

Abbildung 22: Email-Aufbau

Bevor die E-Mail gesendet wird, wird sie *Base64URL-codiert*. Dieses Format stellt sicher, dass die Nachricht ohne Probleme übertragen werden kann. Die Kodierung passiert automatisch im Hintergrund.

Der Versand selbst erfolgt dann über die *Gmail API*. Dabei wird die vorbereitete Nachricht übergeben und über den autorisierten *Google-Account* versendet (siehe *Listing 80*). Als Ergebnis erhält man unter anderem eine *Message-ID*, die für Protokolle oder Fehleranalysen genutzt werden kann.

```
1. const gmail = google.gmail({ version: 'v1', auth: oauth });  
2. const raw = buildRawEmail({ to, subject, html });
```

```
3.
4. const res = await gmail.users.messages.send({
5.   userId: 'me',
6.   requestBody: { raw },
7. });
8.
9. return res.data;
```

Listing 80: Backend - Email Erstellung und senden

Dabei gilt:

- *userId: 'me'* bedeutet, dass die E-Mail über genau den Google-Account versendet wird, der durch OAuth2 autorisiert ist.
- *requestBody.raw* enthält die vollständig aufgebaute und *Base64URL-kodierte MIME-Nachricht*.
- *res.data* enthält unter anderem eine Message-ID, die für Logging und spätere Fehleranalyse genutzt werden kann (z. B. zur Nachverfolgung fehlgeschlagener Zustellungen).

Durch diese Umsetzung ist der Mailversand klar gekapselt, technisch robust und gut wartbar. Typische Fehlerquellen wie ungültige *OAuth-Tokens* oder fehlerhaftes Encoding werden frühzeitig erkannt, und der eigentliche Versand folgt einem klar definierten, nachvollziehbaren Ablauf.

4.3.11 Zusammenfassung

Das Backend ist als *NestJS*-Anwendung umgesetzt und bildet die zentrale Instanz für Geschäftslogik, Sicherheit und *Datenintegrität*. *Authentifizierung*, *Autorisierung*, fachliche *Policies* sowie alle finanzrelevanten Prozesse werden konsequent serverseitig durchgesetzt und sind unabhängig vom Client abgesichert. Kritische Abläufe wie Bestellungen und Guthabenänderungen sind *transaktional* umgesetzt und verhindern durch Sperren und Validierungen *inkonsistente Zustände*. Ergänzend sorgen klar definierte *Auth-Flows*, *restriktive Account-Regeln* und ein robuster E-Mail-Versand über die *Gmail API* für Sicherheit, Nachvollziehbarkeit und Wartbarkeit im laufenden Betrieb.

4.4 PostgreSQL DB und DB-Zugriffe *(Ellmer)*

Neben *Frontend* und *Backend-Logik* bildet die relationale Datenbank die dritte zentrale Säule der Anwendung. Während das Frontend für Interaktion und Benutzerführung zuständig ist und das Backend die Geschäftslogik und die Kommunikation zwischen Frontend bis hin zur Datenbank regelt, übernimmt *PostgreSQL* die dauerhafte, konsistente und strukturierte Speicherung aller relevanten Informationen.

Die Datenbank ist dabei nicht lediglich ein technischer Speicherort, sondern integraler Bestandteil des Gesamtsystems. Fachliche Zusammenhänge – etwa die Zuordnung von Bestellungen zu Benutzern oder die Bindung von Gerichten an einen bestimmten Speiseplan – werden bewusst im relationalen Modell abgebildet.

Ziel dieses Kapitels ist es, das zugrunde liegende Datenmodell strukturiert darzustellen, die Organisation der Datenzugriffe im Backend zu erläutern sowie zentrale Lese- und Schreiboperationen nachvollziehbar zu dokumentieren.

4.4.1 Datenbankschema und ER-Diagramm

Das relationale Datenmodell orientiert sich direkt an den fachlichen Anforderungen der Anwendung. Im Mittelpunkt stehen drei Kernbereiche:

- Benutzerverwaltung
- Menü- und Speiseplanverwaltung
- Bestellwesen

Das vollständige Datenbankschema ist in *Abbildung 23* als ER-Diagramm ersichtlich.

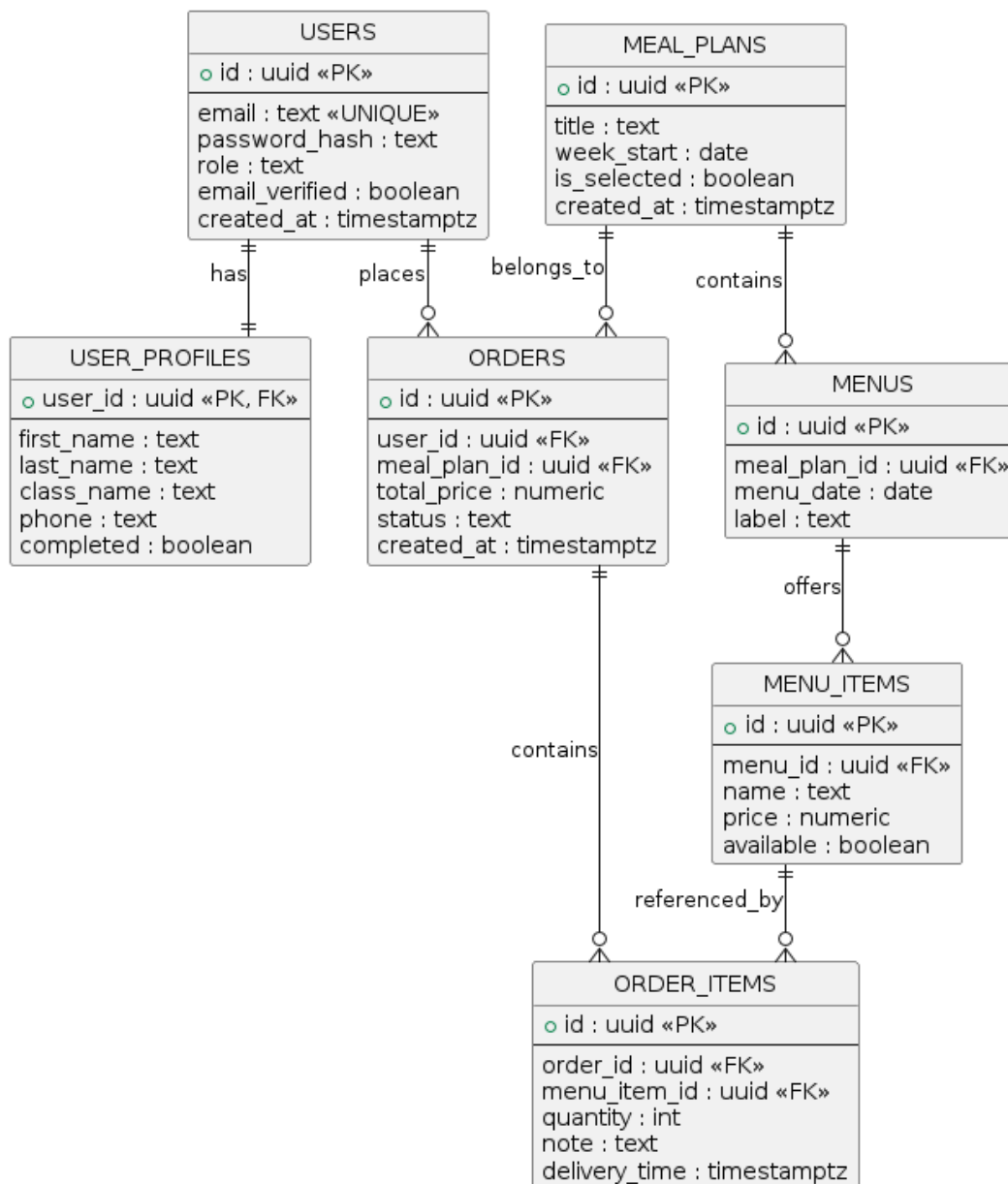


Abbildung 23: ER-Diagramm - Datenbankschema (Überblick)

Benutzer und Profildaten

In der Tabelle *USERS* werden alle sicherheitsrelevanten Informationen eines Benutzers gespeichert – vor allem *email*, *password_hash* und *role*. Diese Daten sind die Grundlage für Anmeldung und Rechteverwaltung in der Anwendung.

Persönliche Angaben wie *Vor- und Nachname*, *Klasse* oder *Telefonnummer* befinden sich bewusst in einer eigenen Tabelle, *USER_PROFILES*. Jeder Benutzer hat genau ein Profil. Technisch wird das dadurch umgesetzt, dass *user_id* gleichzeitig *Primär- und Fremdschlüssel* ist.

Diese Trennung sorgt für ein übersichtliches Datenmodell: *Sicherheitsdaten und Profildaten* sind klar voneinander abgegrenzt. Das macht das System verständlicher, wartbarer und leichter erweiterbar.

Speiseplan und Menüstruktur

Ein *MEAL_PLAN* steht für einen Speiseplan in einem bestimmten Zeitraum, meist für eine Woche.

Ein Speiseplan enthält mehrere *MENUS*, die jeweils einem Datum zugeordnet sind. Jedes Menü besteht wiederum aus mehreren *MENU_ITEMS*, also den einzelnen Gerichten.

Über das Feld *is_selected* wird festgelegt, welcher Speiseplan aktuell aktiv ist. Bestellungen sind nur für diesen aktiven Plan möglich. Dadurch wird verhindert, dass alte oder zukünftige Menüs bestellt werden.

Die Struktur - *Speiseplan* → *Menü* → *Gericht* - bildet die Realität klar ab und bleibt flexibel für spätere Erweiterungen.

Bestellungen

Eine *ORDER* gehört immer zu einem Benutzer und zu einem bestimmten Speiseplan. Sie fasst alle Gerichte zusammen, die zu einem Zeitpunkt bestellt wurden.

Die einzelnen Positionen werden in *ORDER_ITEMS* gespeichert. Jede Position verweist auf ein bestimmtes *MENU_ITEM* und enthält Angaben wie *quantity*, eine optionale *note* und die gewünschte *delivery_time*.

Gerichtsinformationen werden dabei nicht mehrfach gespeichert, sondern nur referenziert. Dadurch bleibt die Datenbank übersichtlich und konsistent, auch wenn sich Preise oder Bezeichnungen ändern.

Die Trennung zwischen Bestellung und Bestellpositionen sorgt außerdem für eine klare *1:n-Beziehung*: Eine Bestellung kann mehrere Positionen enthalten. Das erleichtert nicht nur die technische Umsetzung, sondern auch spätere Auswertungen, etwa zu Bestellmengen oder Umsätzen.

4.4.2 Organisation der Datenzugriffe

Die Datenzugriffe im Backend sind klar strukturiert und folgen dem bereits eingeführten Schichtenmodell. Die Anwendung gliedert sich in drei zentrale Ebenen:

- *Controller* - stellt die *HTTP*-Schnittstelle bereit
- *Service* - enthält die fachliche Logik
- *Repository* - übernimmt sämtliche Datenbankzugriffe

SQL-Abfragen befinden sich ausschließlich in den *Repository*-Klassen. Dadurch bleibt die Fachlogik in den *Services* vollständig von den Datenbankdetails getrennt. Änderungen an Tabellenstrukturen oder Optimierungen einzelner *Abfragen* können vorgenommen werden, ohne andere Schichten zu beeinflussen.

Diese klare Aufgabentrennung erhöht die Wartbarkeit und Testbarkeit des Systems und gleichzeitig bleibt der Code übersichtlich, da fachliche Entscheidungen nicht mit technischen Implementierungsdetails vermischt werden.

4.4.3 Integrität und Transaktionsmanagement

Ein wesentliches Ziel der Datenbankarchitektur ist die Sicherstellung konsistenter und gültiger Daten.

Die *Integrität* wird durch *Foreign-Key-Constraints* gewährleistet. So kann beispielsweise ein Eintrag in *ORDER_ITEMS* nur existieren, wenn die zugehörige Bestellung und das Menüelement tatsächlich vorhanden sind.

Zusätzlich sorgen *UNIQUE-* und *CHECK-Constraints* für fachliche Korrektheit. Die E-Mail-Adresse in *USERS* ist eindeutig, Mengenangaben müssen größer als null sein und Statuswerte sind auf definierte Optionen beschränkt.

Bei komplexeren Schreibvorgängen – insbesondere beim Anlegen einer Bestellung mit mehreren Positionen – werden *Transaktionen* eingesetzt. Dadurch wird sichergestellt, dass entweder alle Datenbankoperationen erfolgreich durchgeführt oder im Fehlerfall vollständig zurückgerollt werden. Inkonsistente Zwischenzustände können so zuverlässig vermieden werden.

4.4.4 Performance und Skalierbarkeit

Neben der Datenkonsistenz spielt auch die Performance eine wichtige Rolle. Deshalb wurden häufig verwendete Fremdschlüssel und Suchattribute mit passenden Indizes versehen.

Indizes auf Spalten wie *user_id*, *meal_plan_id* oder Datumsfeldern sorgen dafür, dass Abfragen auch bei wachsender Datenmenge schnell bleiben. So bleibt das System selbst bei steigender Anzahl von Benutzern und Bestellungen zuverlässig und reaktionsschnell.

Ergänzend dazu wird im Backend ein *Connection-Pool* eingesetzt. Dadurch werden bestehende Datenbankverbindungen wiederverwendet, anstatt für jede Anfrage eine neue

Verbindung aufzubauen. Das spart Ressourcen und trägt wesentlich zur Stabilität des Gesamtsystems bei.

4.4.5 Zusammenfassung

Die *Persistenzschicht* basiert auf einem klar strukturierten *relationalen Datenmodell*, das die Anforderungen der Anwendung verständlich und logisch abbildet.

Durch die Trennung von *Anmeldedaten* und *Profildaten*, die übersichtliche *Menüstruktur* sowie die *normalisierte Speicherung* von Bestellungen entsteht ein stabiles und gut erweiterbares System.

Die klare Aufteilung in *Controller*, *Service* und *Repository* sowie der Einsatz von Constraints und Transaktionen sorgen dafür, dass die Daten jederzeit korrekt und konsistent bleiben. Die Datenbank ist damit nicht nur ein Speicherort, sondern ein zentraler Bestandteil der Anwendung. Sie stellt sicher, dass die Geschäftslogik zuverlässig umgesetzt wird und bietet eine solide Grundlage für zukünftige Weiterentwicklungen.

4.5 Deployment des Bestellservices (Ellmer)

Nach Abschluss der Entwicklungsphase der drei Hauptkomponenten, Frontend, Backend und Datenbank, wurde die Anwendung in eine produktionsnahe Cloud-Umgebung überführt. Ziel war es, eine stabile, wartbare und kostengünstige Hosting-Architektur umzusetzen, wie sie auch in professionellen Webprojekten üblich ist.

Dabei wurde bewusst auf eine klare Trennung der Systemkomponenten gesetzt. Das Frontend wird über Netlify bereitgestellt, das Backend läuft auf Render und die Datenbank wird von Supabase verwaltet.

Diese Aufteilung folgt dem Grundprinzip der *Separation of Concerns*. Jede Komponente wird auf einer Plattform betrieben, die für ihren jeweiligen Einsatzzweck optimiert ist. Dadurch kann *Vendor Lock-in* vermieden werden, da kein Anbieter das gesamte System kontrolliert. Außerdem ist das Hosting mit dieser Variante und unseren Anforderungen am günstigsten.

Frontend – Static Web Application

Das Frontend wird als *Static Web Application* betrieben. Nach dem *Build-Prozess* entstehen ausschließlich statische Dateien wie *HTML*, *CSS* und *JavaScript*. Es wird daher kein eigener Server benötigt.

Netlify eignet sich für dieses Szenario besonders gut, da automatische *Deployments* direkt aus dem *Git-Repository* möglich sind. Zusätzlich erfolgt die Auslieferung über ein *Content Delivery Network (CDN)*. Dadurch wird eine schnelle Ladezeit sowie eine hohe Verfügbarkeit erreicht, ohne zusätzliche Infrastruktur betreiben zu müssen.

Für unser Projekt ist dies eine einfache und gleichzeitig professionelle Lösung.

Backend

Im Gegensatz dazu benötigt das Backend eine eigene *Runtime*, da hier die gesamte *Business Logic* verarbeitet wird. Dazu gehören unter anderem *Authentication*, *Authorization*, Validierung von *Requests* sowie die Kommunikation mit der Datenbank.

Render stellt dafür einen sogenannten *Web-Service* bereit, der die Anwendung dauerhaft ausführt. Ein Vorteil dieser Trennung ist, dass Frontend und Backend unabhängig voneinander *deployt* werden können. Änderungen an der Benutzeroberfläche haben somit keinen direkten Einfluss auf die Serverlogik und umgekehrt.

Datenbank

Die Datenbank wird als *Managed PostgreSQL Service* über *Supabase* betrieben. Der Vorteil eines *Managed Service* liegt darin, dass Wartung, Backups und Sicherheitskonfigurationen vom Anbieter übernommen werden.

Eine selbst verwaltete Datenbank hätte zusätzlichen administrativen Aufwand bedeutet, insbesondere im Bereich Updates und Datensicherung. Für den Umfang der Diplomarbeit wäre dies nicht verhältnismäßig gewesen. Mit *Supabase* konnte eine stabile und professionelle Lösung umgesetzt werden, ohne zusätzliche Infrastruktur betreiben zu müssen.

Herausforderungen beim Deployment

Der Übergang von der lokalen Entwicklungsumgebung in die Cloud verlief nicht vollständig ohne Schwierigkeiten.

Ein zentrales Thema war die Konfiguration von *Cross-Origin Resource Sharing (CORS)*. Da Frontend und Backend auf unterschiedlichen Domains laufen, müssen sogenannte *Cross-Origin Requests* explizit erlaubt werden. Ohne korrekte Einstellungen blockiert der Browser diese Anfragen aus Sicherheitsgründen automatisch.

Auch der Umgang mit *Environment Variables* war relevant. Lokale *.env*-Dateien können nicht direkt übernommen werden, da jede Plattform ihre Konfiguration separat verwaltet. Dabei war besonders wichtig, dass das Frontend bestimmte Variablen bereits zur *Build-Time* benötigt, während das Backend diese erst zur *Runtime* ausliest.

Zusätzlich musste die Datenbankverbindung korrekt über *Secure Sockets Layer (SSL)* eingerichtet werden. Erst nach Anpassung des *Connection Strings* und entsprechender Einstellungen konnte eine stabile Verbindung zwischen Backend und Datenbank sichergestellt werden.

Hosting-Architektur im Überblick

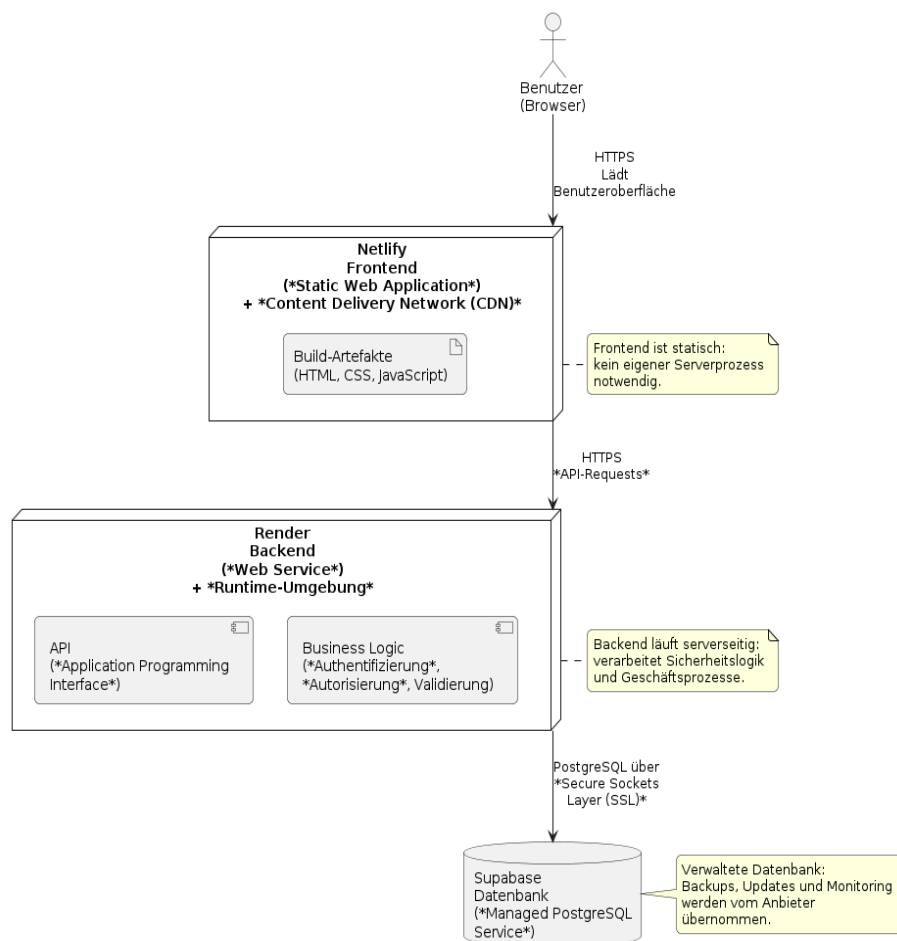


Abbildung 24: Architekturdiagramm - Hosting

Das Architekturdiagramm (Abbildung 24) zeigt die klare Aufteilung der Anwendung in Benutzer, Frontend, Backend und Datenbank.

Der Benutzer greift über den Browser auf das Frontend zu, welches als *Static Web Application* über *Netlify* bereitgestellt wird. Aktionen des Benutzers werden als *Application Programming Interface (API) Requests* an das Backend auf *Render* gesendet. Dort läuft die *Business Logic*, inklusive *Authentifizierung*, *Autorisierung* und *Validierung*.

Das Backend kommuniziert anschließend über *PostgreSQL* mit der Datenbank auf *Supabase*. Die Verbindung erfolgt verschlüsselt über *Secure Sockets Layer (SSL)*.

Ein direkter Zugriff des Frontends auf die Datenbank findet nicht statt. Dadurch bleibt die Sicherheitslogik vollständig im Backend und die Architektur entspricht dem Prinzip der *Separation of Concerns*.

Zusammenfassung

Zusammenfassend hat sich die gewählte Hosting-Strategie als sinnvoll und praxisnah erwiesen. Auch wenn die Nutzung mehrerer Plattformen anfangs etwas mehr

Konfigurationsaufwand bedeutete, entstand dadurch eine klare und gut strukturierte *Cloud-Architektur*. Durch die Umsetzung nach dem Prinzip der *Separation of Concerns* konnte eine sichere, wartbare und gleichzeitig kosteneffiziente Lösung realisiert werden, die sich an modernen Standards von Webanwendungen orientiert und realitätsnah ist. Die *Domain*, mit der das Bestellsystem dann erreichbar sein sollte, wird über *WorldForYou* bereitgestellt.

4.6 Rechtliche Absicherung

In diesem Abschnitt geht es um die rechtliche Absicherung des Bestellservices. Entsprechend der Datenschutz-Grundverordnung (*DSGVO*) muss jedem User mitgeteilt werden, wann und in welchem Umfang personenbezogene Daten (z.B. Name, Klasse, E-Mail) gespeichert und verarbeitet werden [18]. Die *DSGVO* definiert dabei grundlegende Prinzipien wie Transparenz, Zweckbindung und Datenminimierung, die bei der technischen Umsetzung zwingend zu berücksichtigen sind [19].

Der Bestellservice erhebt personenbezogene Daten ausschließlich für klar definierte Zwecke. Dazu zählen insbesondere die Erstellung und Verwaltung von Benutzerkonten, die eindeutige Zuordnung von Bestellungen sowie die organisatorische Abwicklung der Geschäftsfälle. Es werden nur jene Daten gespeichert, die für den Betrieb des Systems tatsächlich erforderlich sind. Eine darüberhinausgehende Verarbeitung oder Weitergabe findet nicht statt. Damit entspricht es dem Grundsatz der Datenminimierung gemäß *DSGVO* [18].

Die praktische Umsetzung dieser rechtlichen Anforderungen erfolgt über ein Dialogfenster, das beim ersten Start der App, beziehungsweise beim ersten Aufruf der Webanwendung automatisch angezeigt wird. Dieses Dialogfenster informiert den Nutzer transparent darüber,

- welche personenbezogenen Daten gespeichert werden,
- zu welchem Zweck die Speicherung erfolgt,
- auf welcher rechtlichen Grundlage die Verarbeitung basiert und
- welche Rechte dem Nutzer zustehen (Auskunft, Berichtigung, Löschung).

Erst nach aktiver Zustimmung kann die Anwendung vollständig genutzt werden. Die Einwilligung wird technisch dokumentiert, um die Nachweispflicht gemäß *DSGVO* zu erfüllen [18]. Dadurch wird sichergestellt, dass jeder User bewusst in die Datenverarbeitung einwilligt und umfassend informiert ist.

Zusätzlich handelt es sich beim System funktional um einen Onlineshop, weil reale Geschäftsfälle abgewickelt werden. Der Gesetzgeber verpflichtet den Betreiber daher, den Besuchern der Webseite jederzeit ein Impressum mit vollständigen Kontaktdaten, sowie die Bestimmungen der Allgemeinen Geschäftsbedingungen (*AGB*) zur Verfügung zu stellen. Diese Informationspflichten ergeben sich aus den rechtlichen Rahmenbedingungen für digitale Dienste und Online-Geschäftsmodelle [18,19].

In der Anwendung sind daher sowohl die Datenschutzerklärung als auch das Impressum und die *AGB* dauerhaft abrufbar, beispielsweise über einen eigenen Menüpunkt oder den Footer-

Bereich. Dadurch wird gewährleistet, dass alle rechtlich relevanten Informationen jederzeit zugänglich sind und nicht nur einmalig im Rahmen des Dialogfensters angezeigt werden.

Zusammenfassend wird die rechtliche Absicherung des Bestellservices durch eine Kombination aus transparenter Information, aktiver Einwilligung und dauerhaft zugänglichen Rechtstexten umgesetzt. Das Dialogfenster zu Beginn der App erfüllt die datenschutzrechtlichen Informationspflichten gemäß *DSGVO* [18,19], während Impressum und *AGB* die gesetzlichen Anforderungen an geschäftsmäßige Online-Dienste abdecken. Auf diese Weise wird sowohl die rechtliche Konformität des Systems sichergestellt als auch das Vertrauen der Nutzer in den verantwortungsvollen Umgang mit ihren personenbezogenen Daten gestärkt.

5 Softwarequalitätssicherung (Testing)

5.1 Unit-Tests *(Magos)*

Um die Funktionalität zentraler Komponenten zu prüfen, wurden im Frontend Unit-Tests implementiert. Ziel dieser Tests ist die Prüfung der internen Logik des Frontends, der Validierungsmechanismen und das Zusammenspiel mit den verschiedenen Services. Damit können Fehler in diesen Funktionalitätsbereichen frühzeitig erkannt und korrigiert werden. Da die Backendlogik mittels Integration-Tests geprüft wird, wurden Unit-Tests nur für die zentralen, funktionalen Bestandteile des Frontend eingesetzt. Die Unit-Tests leisten damit einen wesentlichen Beitrag zur Stabilität, Wartbarkeit und Weiterentwickelbarkeit der Anwendung.

5.1.1 Testframework

Für die Implementation der Unit-Tests im Frontend wurden die Testframeworks *Karma* und *Jasmine* verwendet. Diese Kombination ist der Standard innerhalb von *Angular* Projekten und wird von der *Angular CLI* unterstützt.

Jasmine ist zuständig für die Definition und Strukturierung der Testfälle. Es stellt Funktionen wie *describe*, *it*, *expect* sowie verschiedene *Matchers* zur Verfügung, mit denen das Verhalten einzelner Methoden überprüft werden kann. Außerdem ermöglicht *Jasmine* die Erstellung von sogenannten *Spies*, mit denen Abhängigkeiten wie *Services* und *HTTP-Clients* simuliert werden können. Dadurch können Komponenten und *Services* isoliert getestet werden, ohne reale Backend Anfragen auszuführen.

Karma ist für die Ausführung der Tests zuständig. Mit dem Befehl „*ng test*“ startet *Karma* alle Tests in einem Browser, führt sie aus und gibt die Ergebnisse in der Konsole aus. Die Ergebnisse der Tests werden bei der Entwicklung kontinuierlich aktualisiert, Änderungen im Code können also sofort überprüft werden.

Die Kombination von *Karma* und *Jasmine* ermöglicht also eine strukturierte, automatisierte Überprüfung der Frontend Logik und bildet die technische Grundlage für die im weiteren Verlauf beschriebenen Unit-Tests.

5.1.2 Order Service

Die Unit-Tests des *OrderService* überprüfen die korrekte Verarbeitung und Filterung von Bestellungen im Frontend. Dabei wird insbesondere getestet, ob Bestellungen auf Basis des aktuell angemeldeten Benutzers gefiltert werden, ob QR-Codes nur für offene Bestellungen generiert werden und ob Fehlerfälle bei *HTTP*-Anfragen richtig behandelt werden.

Testaufbau und Mocking

Zur Isolierung des Services werden sowohl *HttpClient* und *AuthService* mithilfe von *Jasmine Spies* simuliert (siehe *Listing 81*). Dadurch kann das Verhalten dieser Abhängigkeiten gezielt gesteuert werden.

```
1. http = jasmine.createSpyObj('HttpClient', ['get']);
2. auth = jasmine.createSpyObj('AuthService', ['getCurrentUserId']);
3.
4. TestBed.configureTestingModule({
5.   providers: [
6.     OrderService,
7.     { provide: HttpClient, useValue: http },
8.     { provide: AuthService, useValue: auth },
9.   ]
10.});
```

Listing 81: Testing - Testaufbau und Mocking

Dadurch wird sichergestellt, dass keine echten *HTTP*-Anfragen ausgeführt werden und die Benutzeridentifikation wie gewollt manipuliert werden kann. Diese Methode wird für die Definition der jeweiligen Objekte in jeder Testdatei verwendet, daher wird es bei der Beschreibung der restlichen Tests nicht weiters erwähnt.

Filterung nach Benutzer und QR Code Erzeugung

Ein zentraler Testfall überprüft die Methode *getMyOrders()* (siehe *Listing 82*). Dabei wird simuliert, dass ein Benutzer angemeldet ist und eine Liste von Bestellungen vom Backend zurückgegeben wird.

```
1. auth.getCurrentUserId.and.returnValue('user123');
2. http.get.and.returnValue(of(mockOrders));
3.
4. service.getMyOrders().subscribe(filteredWithQr => {
5.   expect(filteredWithQr.length).toBe(2);
6. });
```

Listing 82: Testing - Filtern nach Usern

Dieser Test stellt sicher, dass nur die Bestellungen des angemeldeten Benutzers zurückgegeben werden. Außerdem wird sichergestellt, dass nur offene Bestellungen das Feld *QR Code* haben.

QR Code Logik

Hier wird die Methode *addQrForOpenOrders()* getestet, es wird also sichergestellt, dass *QR-Codes* nur offenen Bestellungen hinzugefügt werden (siehe *Listing 83*).

```
1. const withQr = service.addQrForOpenOrders(orders);
2.
3. expect(withQr[0].qrCodeUrl).toContain('Order-1');
4. expect(withQr[1].qrCodeUrl).toBeUndefined();
```

Listing 83: Testing – Hinzufügung des QR-Codes zu offenen Bestellungen

In einem weiteren Test wird die Methode *generateQrCode()* isoliert getestet (siehe *Listing 84*). Hier wird also das Format der erzeugten URL überprüft.


```
1. const url = service.generateQrCode('myOrderId');
3. expect(url).toBe(
4.   'https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=Order-myOrderId'
5. );
```

Listing 84: Testing - Erzeugung QR-Code

Diese Tests stellen sicher, dass die *QR-Code Generierung* unabhängig von anderen Logikbestandteilen funktioniert.

Fehlerhandlung bei HTTP-Anfragen

Als nächstes wird in einem Test ein Fehler bei der *HTTP*-Anfrage simuliert. Dabei wird geprüft, ob der Fehler korrekt weitergegeben und gleichzeitig ein entsprechender Konsolen-Fehler ausgegeben wird (siehe *Listing 85*).

```
1. http.get.and.returnValue(throwError(() => new Error('network error')));
2. spyOn(console, 'error').and.callFake(() => {});
3.
4. service.getMyOrders().subscribe({
5.   error: (err) => {
6.     expect(err).toEqual(jasmine.any(Error));
7.     expect(console.error).toHaveBeenCalled();
8.   }
9. });
```

Listing 85: Testing - HTTP-Fehlerbehandlung OrderService

Damit wird sichergestellt, dass die im Service implementierte *catchError*-Logik richtig funktioniert und keine unerwarteten Fehler auftreten.

Zusammenfassung

Die Unit-Tests des *OrderService* decken sowohl die Kernlogik zur Benutzerfilterung als auch die *QR-Code*-Erzeugung und die Fehlerbehandlung ab. Die Tests stellen sicher, dass nur relevante Bestellungen angezeigt werden, offene Bestellungen korrekt mit *QR-Codes* versehen werden und Fehlerfälle nachvollziehbar behandelt werden.

5.1.3 Order Overview Component

Die Unit-Tests der *OrderOverviewComponent* überprüfen die zentrale Logik der Bestellübersicht im Frontend. Dabei wird getestet, ob Bestellungen korrekt geladen, verarbeitet und dargestellt werden. Zusätzlich wird überprüft, ob Bestellungen richtig nach ihrem Status gefiltert werden, ob der Gesamtpreis korrekt berechnet wird und ob Benutzerinteraktionen, wie das Anzeigen von Details oder die Navigation, funktionieren.

Laden und Verarbeiten der Bestellungen

Als Erstes wird überprüft, ob beim Initialisieren der Komponente die Bestellungen automatisch geladen werden. Dabei wird kontrolliert, ob der Service für das Laden der Daten aufgerufen wird und ob die geladenen Daten korrekt gespeichert werden (siehe *Listing 86*).

```
1. it('should load orders on init', () => {
```

```
2.   fixture.detectChanges();
3.
4.   expect(mockOrderService.getMyOrders).toHaveBeenCalled();
5.   expect(mockOrderService.addQrForOpenOrders).toHaveBeenCalled();
6.   expect(component.orders.length).toBe(2);
7.   expect(component.loading).toBeFalse();
8.   expect(component.error).toBeNull();
9. });
```

Listing 86: Testing - Laden der Bestellungen im OrderOverviewComponent

Zusätzlich erfolgt eine Überprüfung, ob die Bestellungen korrekt nach ihrem *Status* korrekt sortiert werden. Dabei werden die Getter *openOrders* und *closedOrders* verwendet, welche die vorhandenen Bestellungen entsprechend aufteilen (siehe Listing 87).

```
1. it('should filter open and closed orders correctly', () => {
2.   fixture.detectChanges();
3.
4.   expect(component.openOrders.length).toBe(1);
5.   expect(component.closedOrders.length).toBe(1);
6. });
```

Listing 87: Testing - Filtern der Bestellungen nach Status

Berechnung des Gesamtpreises

Ein weiterer Test überprüft die Berechnung des Gesamtpreises einer Bestellung (siehe Listing 88). Falls dieser Wert vom Backend nicht bereitgestellt wird, berechnet die Komponente den Preis anhand der enthaltenen Bestellpositionen.

```
1. expect(component.orders[0].totalPrice).toBe(3);
```

Listing 88: Testing - Berechnung des Gesamtpreises einer Bestellung

Dadurch wird sichergestellt, dass auch vom Backend unvollständige Daten korrekt verarbeitet werden.

Fehlerbehandlung

Beim Laden der Bestellung auftretende Fehler werden ebenfalls getestet. (siehe Listing 89). Dabei muss die Komponente eine entsprechende Fehlermeldung werfen und den Ladevorgang beenden.

```
1. expect(component.error).toBe('Bestellungen konnten nicht geladen werden.');
```

```
2. expect(component.loading).toBeFalse();
```

Listing 89: Testing - Fehlerbehandlung beim Laden der Bestellungen

Dieser Abschnitt stellt sicher, dass Fehler nicht unbemerkt bleiben und dass der Benutzer entsprechend über Probleme informiert wird.

Benutzerinteraktion und Hilfsfunktionen

Die Komponente enthält auch Funktionen für Benutzerinteraktionen, die getestet werden sollen. Eine davon ist das Ein- und Ausblenden von Bestelldetails (siehe *Listing 90*). Die Methode *toggleDetails* verändert die *showDetails* Eigenschaft der Bestellung.

```
1. component.toggleDetails(order);  
2. expect((order as any).showDetails).toBeTrue();
```

Listing 90: Testing - Ein- und Ausblenden von Bestelldetails

Auch die Navigationsfunktion wird getestet (siehe *Listing 91*). Beim Aufruf der Methode *navigateBack* soll der Router zur Startseite navigieren.

```
1. component.navigateBack();  
2. expect(mockRouter.navigate).toHaveBeenCalledWith(['/']);
```

Listing 91: Testing - Navigation vom OrderOverviewComponent zur Startseite

Weiters wird die Hilfsmethode *getOrderTitle* überprüft. Diese erzeugt eine kurze textuelle Darstellung einer Bestellung, basierend auf den enthaltenen Menüelementen. Die Tests stellen sicher, dass sowohl leere Bestellungen als auch Bestellungen mit mehreren Positionen korrekt dargestellt werden.

Zusammenfassung

Die Unit-Tests der *OrderOverviewComponent* decken die wichtigsten Funktionen der Bestellübersicht ab. Dazu gehören das Laden der Bestellungen, die Verarbeitung der Daten, die Fehlerbehandlung, sowie verschiedene Benutzerinteraktionen. Durch den Einsatz von *Mock*-Objekten werden externe Abhängigkeiten isoliert, sodass ausschließlich die Logik der Komponente getestet wird. Dadurch wird sichergestellt, dass die Bestellübersicht zuverlässig funktioniert und mögliche Fehler frühzeitig erkannt werden können.

5.1.4 Cart Page Component

Die Unit-Tests der *CartPageComponent* überprüfen die zentrale Logik des Warenkorbs im Frontend. Dabei wird getestet, ob die im Warenkorb enthaltenen Artikel korrekt geladen und verwaltet werden, ob Änderungen an Mengen und Notizen funktionieren und ob der Bestellprozess ordnungsgemäß durchgeführt wird. Zusätzlich werden verschiedene Fehlerfälle überprüft, beispielsweise ein leerer Warenkorb oder ein unzureichendes Guthaben des Benutzers.

Initialisierung der Komponente

Auch für dieser Komponente wird wieder die Initialisierung geprüft, es wird also sichergestellt, dass beim Start der Komponente die *Warenkorb-Daten* und das Benutzerprofil korrekt geladen werden. Dabei wird kontrolliert, ob die entsprechenden *Service*-Methoden aufgerufen werden und ob die zurückgegebenen Werte korrekt in der Komponente gespeichert werden (siehe *Listing 92*).

```
1. fixture.detectChanges();  
2. tick();  
3.
```

```
4. expect(mockCartService.getCartItems).toHaveBeenCalled();
5. expect(mockUserProfile.getProfile).toHaveBeenCalled();
6. expect(component.userId).toBe('user1');
7. expect(component.balance).toBe(100);
8. expect(component.orderingEnabled).toBeTrue();
```

Listing 92: Testing - Initialisierung des CartPageComponents

Verwaltung der Warenkorb Elemente

Die Komponente bietet mehrere Funktionen zur Bearbeitung der Artikel im Warenkorb. Zum Beispiel kann die Artikelmenge erhöht oder verringert werden und die Notizen zu einzelnen Artikeln bearbeitet werden. Hier wird die Bearbeitung der Artikelmenge getestet (siehe *Listing 93*).

```
1. component.increaseQuantity(0);
2. expect(component.cartItems[0].quantity).toBe(3);
3.
4. component.decreaseQuantity(0);
5. expect(component.cartItems[0].quantity).toBe(2);
```

Listing 93: Testing - Erhöhung und Verringerung von Artikel im Warenkorb

Das Aktualisieren der Notiz und das Entfernen eines Artikels aus dem Warenkorb werden auch getestet (siehe *Listing 94*).

```
1. component.updateNote(1, 'some note');
2. expect(component.cartItems[1].note).toBe('some note');
3.
4. component.removeItem(0);
5. expect(component.cartItems.length).toBe(1);
```

Listing 94: Testing - Aktualisierung der Notiz und Entfernung eines Artikels aus dem Warenkorb

Diese Tests stellen sicher, dass Veränderungen im Warenkorb richtig verarbeitet werden.

Berechnung des Gesamtpreises

Die Methode *getTotal()* berechnet den Gesamtpreis aller Artikel im Warenkorb. Dabei wird auch berücksichtigt, dass Preise sowohl als Zahl als auch als String mit Komma eingegeben werden können (siehe *Listing 95*).

```
1. component.updateNote(1, 'some note');
2. expect(component.cartItems[1].note).toBe('some note');
3.
4. component.removeItem(0);
5. expect(component.cartItems.length).toBe(1);
```

Listing 95: Testing - Berechnung des Gesamtpreises aller Artikel im Warenkorb

Dieser Test stellt sicher, dass unterschiedliche Preisformate korrekt verarbeitet werden.

Bestellprozess

Ein wichtiger Bestandteil der Tests ist die Überprüfung des Bestellvorgangs. Dabei werden verschiedene Bedingungen überprüft, die erfüllt werden müssen, bevor der Benutzer eine Bestellung erstellen kann.

Ein Test überprüft zum Beispiel ob eine Fehlermeldung angezeigt wird, wenn beim Abschluss der Bestellung der Warenkorb leer ist (siehe *Listing 96*).

```
1. component.cartItems = [];  
2. component.onOrder();  
3.  
4. expect(component.timeError).toContain('leer');
```

Listing 96: Testing - Fehlerbehandlung bei leerem Warenkorb

Ein weiterer Test simuliert den Fall, dass das Guthaben des Nutzers für die Kosten der Bestellung nicht ausreicht und überprüft, ob dabei der richtige Fehler ausgegeben wird (siehe *Listing 97*).

```
1. component.balance = 1;  
2. component.onOrder();  
3.  
4. expect(component.timeError).toContain('Nicht genug Guthaben');
```

Listing 97: Testing - Fehlerbehandlung unzureichendes Guthaben

Wenn alle Bedingungen erfüllt sind, wird überprüft, ob die Bestellung erfolgreich abgeschickt werden kann. Dabei wird kontrolliert, ob der *CartService* die Bestellung übermittelt, der Warenkorb geleert und anschließend zur Bestellübersicht navigiert wird (siehe *Listing 98*).

```
1. component.onOrder();  
2.  
3. expect(mockCartService.submitOrder).toHaveBeenCalled();  
4. expect(mockCartService.clearCart).toHaveBeenCalled();  
5. expect(mockRouter.navigate).toHaveBeenCalledWith(['/my-orders']);
```

Listing 98: Testing - Abschluss der Bestellung

Diese Tests stellen sicher, dass sowohl erfolgreiche Bestellungen als auch Fehlerfälle korrekt behandelt werden.

Navigation

Auch die Navigation der Komponente wird überprüft (siehe *Listing 99*). Die Methode *navigateBack()* soll den Benutzer zur Startseite der Anwendung zurückführen.

```
1. component.navigateBack();  
2. expect(mockRouter.navigate).toHaveBeenCalledWith(['/']);
```

Listing 99: Testing – Navigation vom CartPageComponent zur Startseite

Zusammenfassung

Die Unit-Tests der *CartPageComponent* überprüfen die wichtigsten Funktionen der Warenkorb-Seite. Dazu gehören das Laden der Warenkorbdaten, die Bearbeitung von Artikeln, die Berechnung des Gesamtpreises sowie der komplette Bestellprozess. Dadurch kann sichergestellt werden, dass der Warenkorb zuverlässig funktioniert und typische Fehlerfälle wie ein leerer Warenkorb oder ein zu geringes Guthaben korrekt behandelt werden.

5.2 Integration-Tests *(Ellmer)*

Zur Sicherstellung der funktionalen Korrektheit sowie der Einhaltung der Rollen- und Berechtigungslogik wurden für das Backend umfassende *Integration-Tests* implementiert. Diese Tests prüfen nicht einzelne isolierte Methoden, sondern vollständige *Request-Response*

Zyklen über die *HTTP*-Schnittstellen hinweg. Ziel ist es, das reale Laufzeitverhalten der Anwendung möglichst praxisnah abzubilden und Fehlerquellen frühzeitig zu identifizieren.

Die Tests werden ausschließlich im Backend ausgeführt. Das Frontend ist nicht Bestandteil der Testausführung, da hier gezielt die serverseitige Geschäftslogik, Validierung, Autorisierung sowie Datenpersistenz überprüft werden.

5.2.1 Verwendetes Testframework

Die Integration-Tests basieren auf folgenden Technologien:

- *NestJS Testing Utilities (@nestjs/testing)* zur Initialisierung einer vollständigen Testanwendung
- *Jest* als *Test Runner* und *Assertion Framework*
- *Supertest* zur Simulation von *HTTP-Requests* gegen den laufenden Server

Über das *NestJS-Testing-Modul* wird das vollständige *AppModule* geladen und eine echte *INestApplication* instanziiert. Dadurch werden *Controller*, *Services*, *Guards*, *Pipes* und *Provider* wie im regulären Betrieb ausgeführt. Es handelt sich somit nicht um isolierte *Unit-Tests*, sondern um echte *End-to-End-Integration-Tests* innerhalb des Backends.

Die Anwendung wird im Testkontext inklusive globaler *ValidationPipe* gestartet. Dadurch greifen *DTO-Validierung*, *Transformation* sowie *Whitelisting* auch während der Testausführung. Fehlerhafte Eingaben oder inkonsistente Datenmodelle werden somit direkt erkannt.

Die *HTTP-Requests* werden mittels *Supertest* gegen *app.getHttpServer()* ausgeführt. Dadurch wird die *REST-API* unter realistischen Bedingungen geprüft, einschließlich *Header-Verarbeitung*, *Payload-Validierung* und *Statuscode-Auswertung*.

5.2.2 Testarchitektur und Rollenmodell

Die Integration-Tests sind entsprechend dem fachlichen Rollenmodell strukturiert und in zwei Hauptbereiche gegliedert:

- *User-Szenarien*
- *Admin-Szenarien*

Diese Trennung ist notwendig, da sich Endpunkte in der Anwendung sowohl hinsichtlich ihrer fachlichen Funktion als auch hinsichtlich der erlaubten Zugriffsrechte unterscheiden. Während *User* primär eigene Daten abrufen bzw. Aktionen im eigenen Kontext ausführen, können *Admins* administrative Ressourcen verwalten (z. B. Katalogdaten) oder systemweite Übersichten abrufen.

Für beide Rollen werden im Testkontext gültige *JWT-Tokens* erzeugt. Die Token-Generierung erfolgt bewusst direkt über den *JwtService*, damit die Tests unabhängig von einem separaten Login-Endpoint sind und sich auf die eigentlichen Fachendpunkte konzentrieren. Die Tokens werden anschließend bei den *HTTP-Requests* im Header *Authorization: Bearer <token>*

übergeben. Dadurch wird die vollständige *Authentifizierungs-* und *Autorisierungskette* (z. B. *JWT-Guard* und *Roles-Guard*) realitätsnah ausgeführt.

Generierung eines JWT-Tokens im Testkontext

Der folgende Ausschnitt zeigt, wie ein Token innerhalb der Tests signiert wird (siehe *Listing 100*). Wesentlich ist hier der Payload-Aufbau: Über den *sub*-Claim wird die Benutzer-ID transportiert, während über den *roles*-Claim die Rolle (z. B. *user* oder *admin*) gesetzt wird. Dadurch entspricht der Token strukturell dem produktiven Token und kann von den Guards identisch verarbeitet werden.

```
1. function signAccessToken(jwt: JwtService, userId: string, role: string): string {
2.   const payload = {
3.     sub: userId,
4.     roles: [role],
5.   };
6.
7.   return jwt.sign(payload, {
8.     secret: process.env.JWT_SECRET,
9.   });
10. }
```

Listing 100: Testing - Initialisierung: Erstellung eines JWT

Im Testablauf wird diese Funktion typischerweise nach dem programmgesteuerten Anlegen eines Testbenutzers aufgerufen. Der erzeugte *Access Token* wird anschließend in Variablen wie *tokenUser* bzw *tokenAdmin* gespeichert und für alle nachfolgenden *Requests* wiederverwendet. Dadurch bleiben die Tests kompakt und vermeiden redundante Setup-Logik in jedem einzelnen Testfall.

Autorisierter Zugriff eines Users

Der folgende Test demonstriert einen typischen *User-Use-Case*: Ein authentifizierter Benutzer ruft seine Profildaten ab. Der Test setzt dafür den *Authorization*-Header mit einem gültigen *User-Token*. Die erwartete Antwort ist *200*, wodurch bestätigt wird, dass der Endpoint erreichbar ist und die *Guards* den Zugriff korrekt erlauben. Zusätzlich wird die Response-Struktur geprüft, um sicherzustellen, dass tatsächlich ein Objekt zurückgegeben wird (siehe *Listing 101*).

```
1. it('GET /api/users/me/profile (User)', async () => {
2.   const res = await request(app.getHttpServer())
3.     .get('/api/users/me/profile')
4.     .set('Authorization', `Bearer ${tokenUser}`)
5.     .expect(200);
6.   expect(res.body).toEqual(expect.any(Object));
7. });
```

Listing 101: Testing - Authorization Header

Dieses Snippet ist besonders aussagekräftig, da hier mehrere Komponenten zusammenwirken: *Supertest* führt einen realen *HTTP-Request* aus, der *JWT-Guard* validiert den Token, und die Anwendung liefert im Erfolgsfall die Daten des authentifizierten Benutzers zurück.

Administrativer Zugriff auf geschützte Ressourcen

Im Gegensatz zum vorherigen Beispiel wird hier ein *Admin-Szenario* gezeigt. Der Test erstellt ein neues *MenuItem* über einen geschützten *Endpoint*. Entscheidend ist, dass der Request mit

`tokenAdmin` durchgeführt wird. Die Erwartung eines *201-Statuscodes* bestätigt, dass die Resource erfolgreich erstellt wurde und die Rolle ausreichend berechtigt ist. Zusätzlich wird geprüft, dass die Response ein Objekt enthält (z. B. inklusive erzeugter ID) (siehe *Listing 102*).

```
1. it('POST /api/menu-items (Admin)', async () => {
2.   const res = await request(app.getHttpServer())
3.     .post('/api/menu-items')
4.     .set('Authorization', `Bearer ${tokenAdmin}`)
5.     .send({ name: 'Test Item', price: 2.5 })
6.     .expect(201);
7.   expect(res.body).toEqual(expect.any(Object));
8. });
```

Listing 102: Testing - Zugriff Endpoints - Admin

Dieses Snippet prüft damit nicht nur die fachliche Funktion “Erstellen”, sondern indirekt auch die *rollenbasierte Zugriffskontrolle*: Ein User-Token würde hier in der Regel zu *403* führen, während der *Admin-Token* den Zugriff erlaubt.

Verhinderung unautorisierter Zugriffe

Ein zentraler Bestandteil der Tests ist auch die Prüfung von Sicherheitsanforderungen. Der folgende Test simuliert den Zugriff auf einen geschützten *Admin-Endpoint* ohne *Authorization-Header* (siehe *Listing 103*). Erwartet wird ein *401 (Unauthorized)*. Damit ist sichergestellt, dass die Anwendung keine geschützten Ressourcen ohne gültige *Authentifizierung* freigibt.

```
1. it('GET /api/admin/orders (Unauthorized)', async () => {
2.   await request(app.getHttpServer())
3.     .get('/api/admin/orders')
4.     .expect(401);
5. });
```

Listing 103: Testing - Unautorisierter Zugriff auf Bestellungen

Dieser Test ist bewusst kurz gehalten, da hier ausschließlich das Verhalten der *Authentifizierungslogik* im Fokus steht. Er dient als Sicherheitsnetz gegen Konfigurationsfehler (z. B. fehlender *Guard* auf einem *Endpoint*).

Prüfung von Validierungsfehlern

Neben Autorisierung und Authentifizierung wird auch die serverseitige Eingabevalidierung getestet (siehe *Listing 104*). Im folgenden Beispiel wird absichtlich eine fehlerhafte Anfrage gesendet (*items: null*). Durch die aktive globale *ValidationPipe* und definierte *DTO-Regeln* muss die API mit *400 (Bad Request)* reagieren. Damit wird abgesichert, dass ungültige Nutzereingaben nicht in die Geschäftslogik bzw. *Persistenz* gelangen.

```
1. it('POST /api/orders (Validation Error)', async () => {
2.   await request(app.getHttpServer())
3.     .post('/api/orders')
4.     .set('Authorization', `Bearer ${tokenUser}`)
5.     .send({ items: null })
6.     .expect(400);
7. });
```

Listing 104: Testing - Validierungsfehler bei Orders

Dieser Test ist wichtig, da Validierungsfehler häufig zu Folgeproblemen führen können (z. B. fehlerhafte Datenbankeinträge oder unerwartete *Exceptions*). Die Prüfung stellt sicher, dass fehlerhafte *Requests* bereits am *API-Rand* abgefangen werden.

Durch diese Testarchitektur wird die Anwendung entlang der wichtigsten technischen und fachlichen Achsen abgesichert: *JWT-basierte Authentifizierung*, *rollenbasierte Autorisierung*, *DTO-Validierung* sowie realistische *HTTP-Kommunikation*. Die dargestellten *Code-Snippets* bilden dabei typische und aussagekräftige Kernfälle ab, welche im Testkonzept systematisch für User- und Admin-Funktionalitäten erweitert wurden.

5.2.3 Aufbau und Struktur der Tests

Die Integration-Tests folgen einem klar strukturierten Ablauf:

1. Initialisierung der Testumgebung (*beforeAll*)
2. Durchführung der HTTP-Requests
3. Validierung von *Statuscodes* und *Response-Strukturen*
4. Aufräumen erzeugter Testdaten (*afterAll*)

Neben erfolgreichen Fällen (*200*, *201*) werden auch erwartete *Fehlerfälle* (*400*, *404*, *204*) berücksichtigt. Dadurch wird sichergestellt, dass das System sowohl im Erfolgs- als auch im Fehlerfall korrekt reagiert.

Abgedeckt werden unter anderem folgende Funktionsbereiche:

- Verwaltung von *MenuItems*
- Verwaltung von *Menus*
- Erstellung und Auswahl von *MealPlans*
- Benutzerprofil- und Aktivitätsdaten
- Balance-Requests und deren administrative Bestätigung
- Erstellung und Abruf von Bestellungen
- Administrative Order-Übersichten und Statusänderungen
- Vollständige Benutzerverwaltung (*CRUD-Operationen*)

Besonderes Augenmerk liegt auf der Überprüfung der *Autorisierungslogik*. Administrative Endpunkte dürfen ausschließlich von Benutzern mit entsprechender Rolle aufgerufen werden, während reguläre Benutzer nur eingeschränkte Zugriffsrechte besitzen.

5.2.4 Datenkonsistenz und Testisolierung

Da Integration-Tests reale Daten erzeugen, werden Testdaten eindeutig generiert (z. B. durch Verwendung eines Zeitstempels in der E-Mail-Adresse). Dadurch wird sichergestellt, dass mehrere Testläufe nicht miteinander kollidieren.

Erzeugte Entitäten werden nach Abschluss der Tests wieder gelöscht. Löschoperationen sind dabei fehlertolerant implementiert (z. B. Akzeptanz eines *404-Statuscodes*), um wiederholte Testausführungen stabil zu ermöglichen.

5.2.5 Zielsetzung und Mehrwert

Die implementierten Integration-Tests dienen der systematischen Qualitätssicherung auf Systemebene. Sie gewährleisten, dass:

- die REST-Schnittstellen korrekt implementiert sind,
- die *DTO-Validierung* zuverlässig greift,
- die *Authentifizierungs- und Autorisierungsmechanismen* ordnungsgemäß funktionieren,
- Daten konsistent persistiert und abgerufen werden,
- sowie zentrale Geschäftsprozesse vollständig durchlaufen werden können.

Durch die Backend-fokussierte Teststrategie wird insbesondere die geschäftskritische Logik abgesichert. Fehler in Validierung, Rollenprüfung oder Datenfluss würden im Rahmen dieser Tests unmittelbar erkannt.

Die Integration-Tests stellen somit einen wesentlichen Bestandteil der Qualitätssicherung dar und tragen maßgeblich zur Stabilität, Wartbarkeit und langfristigen Robustheit des Systems bei.

5.3 Akzeptanz-Tests

Nach Abschluss der Implementierung sowie der technischen *Integration und Unit-Tests* folgt die Phase der *Akzeptanz-Tests*. Ziel dieser Teststufe ist es, das System unter realen Bedingungen durch ausgewählte Endnutzer prüfen zu lassen und dabei insbesondere die Praxistauglichkeit, Benutzerfreundlichkeit sowie funktionale Vollständigkeit zu evaluieren.

Im Gegensatz zu automatisierten Tests stehen hier nicht einzelne technische Komponenten im Fokus, sondern das Gesamtsystem aus Sicht der Anwender. Die Anwendung ist zu diesem Zeitpunkt bereits produktiv gehostet und im regulären Betrieb erreichbar. Dadurch kann das reale Laufzeitverhalten unter produktionsnahen Bedingungen beobachtet werden (z. B. Performance, Ladezeiten, Stabilität, Authentifizierungsprozesse, Rollen- und Berechtigungskonzepte).

Für die Akzeptanz-Tests wird eine öffentliche, jedoch eingeschränkte Testphase durchgeführt. Eine definierte Gruppe von Testnutzerinnen und Testnutzern erhält Zugriff auf das System und wird gebeten, sämtliche zentralen Funktionalitäten strukturiert zu durchlaufen. Dazu zählen insbesondere:

- Registrierung und Anmeldung
- Rollenbasierte Nutzung (z. B. Benutzer, Inhaber, Administrator)
- Erstellung und Verwaltung von Bestellungen
- QR-Code-Scan und Abschlussprozesse
- Verwaltung von Speisen und Menüs
- Profil- und Kontoverwaltung

Die Testpersonen erhalten klare Testaufgaben, um typische Nutzungsszenarien realitätsnah abzubilden. Dabei sollen sowohl Standardabläufe als auch mögliche Fehlersituationen bewusst ausprobiert werden.

Ziel dieser Phase ist es, funktionale Schwachstellen, logische Inkonsistenzen, Usability-Probleme sowie potenzielle Engpässe frühzeitig zu identifizieren. Besonders relevant sind hierbei:

- Unklare oder missverständliche Benutzerführung
- Fehlerhafte Validierungen oder unerwartetes Systemverhalten
- Performance-Probleme bei paralleler Nutzung
- Sicherheitsrelevante Auffälligkeiten im Authentifizierungs- oder Autorisierungsprozess

Alle Rückmeldungen werden strukturiert dokumentiert und kategorisiert (z. B. Fehler, Verbesserungsvorschlag, Usability-Thema). Kritische Fehler werden priorisiert behandelt und vor Abschluss der Testphase behoben.

Die Akzeptanz-Tests dienen somit als letzte qualitative Validierung vor der vollständigen Freigabe des Systems. Sie stellen sicher, dass die Anwendung nicht nur technisch korrekt funktioniert, sondern auch den fachlichen Anforderungen sowie den Erwartungen der zukünftigen Nutzerinnen und Nutzer entspricht.

Die geplante Testphase musste leider aufgrund mangelnder Zeit des Inhabers mehrfach verschoben werden.

5.4 Ergebnisse Testphase und Evaluation

Im Rahmen der ersten Testphase wurde das System von mehreren Testern intensiv geprüft. Bereits in der ersten Woche konnten wichtige Erkenntnisse gewonnen werden, die zur Verbesserung der Anwendung beigetragen haben. Ein besonderer Fokus lag dabei auf der Sicherheit sowie auf der Benutzerfreundlichkeit.

Einige der Tester führten gezielt *Penetrationstests* durch, um mögliche Sicherheitslücken zu identifizieren. Dabei wurde festgestellt, dass bestimmte *Controller*-Pfade nicht ausreichend abgesichert waren. Obwohl die zugehörigen Methoden bereits geschützt waren, bestand dennoch die Möglichkeit, unerlaubt auf Endpunkte zuzugreifen. Diese Schwachstellen konnten im Anschluss erfolgreich behoben werden, wodurch die Sicherheit des Systems deutlich verbessert wurde.

Neben den sicherheitsrelevanten Aspekten zeigte sich auch, dass die Funktionsweise einzelner Bereiche für die Benutzer anfangs nicht immer sofort verständlich war. Insbesondere bei bestimmten Fenstern und Abläufen kam es zu Unklarheiten. Auf Basis des Feedbacks wurden diese Bereiche überarbeitet und benutzerfreundlicher gestaltet, sodass die Bedienung insgesamt intuitiver wurde.

Ein weiteres Problem trat im Zusammenhang mit den Rollen „User“ und „Kunde“ auf. Hier kam es zu Unstimmigkeiten in der Rechteverteilung, wodurch Funktionen teilweise nicht wie

vorgesehen genutzt werden konnten. Auch dieses Problem wurde analysiert und entsprechend angepasst.

Insgesamt zeigte die Testphase, dass das System bereits stabil funktioniert, jedoch durch gezielte Tests und Rückmeldungen weiter optimiert werden konnte. Die gewonnenen Erkenntnisse trugen maßgeblich dazu bei, sowohl die Sicherheit als auch die Benutzerfreundlichkeit nachhaltig zu verbessern.

6 Projektabschluss

6.1 Ergebnisse

Im Rahmen dieser Diplomarbeit wurde ein vollständiges, digitales Bestell- und Verwaltungssystem für ein Schulbuffet entwickelt und erfolgreich umgesetzt. Ziel des Projektes war es, den bisherigen Bestellprozess, der stark von manuellen Abläufen geprägt war, durch eine moderne digitale Lösung zu ersetzen. Das fertige System unterstützt heute den gesamten Ablauf von der Planung der Speisen über die Bestellung bis hin zur Ausgabe der Gerichte. Dadurch entsteht sowohl für die Kundinnen und Kunden als auch für den Betreiber eine deutlich übersichtlichere und effizientere Arbeitsweise.

Das System besteht aus mehreren aufeinander abgestimmten Komponenten. Im Zentrum steht das Backend, das mit dem Framework *NestJS* umgesetzt wurde. Es enthält die gesamte Geschäftslogik der Anwendung und stellt eine *REST-API* bereit, über die sämtliche Funktionen des Systems angesprochen werden. Dazu gehören unter anderem die Benutzerverwaltung, die Verwaltung von Gerichten und Speiseplänen, die Verarbeitung von Bestellungen, die Verwaltung des Guthabensystems sowie die Funktionen für die Essensausgabe per QR-Code. Das Backend bildet damit die zentrale Schnittstelle zwischen Frontend und Datenbank und sorgt dafür, dass alle Prozesse korrekt, sicher und nachvollziehbar ablaufen.

Als Datenbanksystem wird *PostgreSQL* verwendet. Die Datenbank wird über *Supabase* gehostet und speichert alle persistenten Daten der Anwendung. Dazu zählen Benutzerkonten, Rollen, Gerichte, Menüs, Bestellungen, Guthabenstände sowie Guthabenprotokolle. Die Datenbankstruktur wurde so aufgebaut, dass die einzelnen Bereiche logisch miteinander verbunden sind. Eine Bestellung kann dadurch eindeutig einem Benutzer, einem bestimmten Gericht sowie einem konkreten Zeitpunkt zugeordnet werden. Diese klare Struktur sorgt für eine konsistente Datenspeicherung und bildet eine solide Grundlage für spätere Erweiterungen des Systems.

Das Frontend wurde als Webanwendung mit *Angular* umgesetzt. Es stellt die sichtbare Benutzeroberfläche des Systems dar und ermöglicht die Interaktion mit allen verfügbaren Funktionen. Bei der Entwicklung wurde besonders darauf geachtet, eine möglichst klare, einfache und benutzerfreundliche Oberfläche zu schaffen. Gleichzeitig wurde das System so gestaltet, dass es sowohl auf Desktop-Geräten als auch auf mobilen Endgeräten zuverlässig nutzbar ist. Das fertige Bestellsystem ist nicht nur als klassische Webseite erreichbar, sondern auch als *Progressive Web App (PWA)* nutzbar. Dadurch kann die Anwendung auf unterstützten Geräten direkt installiert und wie eine normale App verwendet werden.

Für Kunden wurde eine übersichtliche Oberfläche zur Auswahl und Bestellung von Speisen umgesetzt. Nach der Anmeldung wird der **aktuelle Speiseplan** angezeigt, der die verfügbaren Gerichte der jeweiligen Woche enthält. Diese Ansicht ist in *Abbildung 25* dargestellt. Dort ist auch zu erkennen, dass die Gerichte übersichtlich als Karten dargestellt werden und zusätzlich nach Kategorien wie Menü, Hauptgericht oder Dessert gefiltert werden können. Darüber hinaus können Benutzer nach Gerichten suchen. Diese Oberfläche bildet den zentralen Einstiegspunkt für den Bestellvorgang und ermöglicht eine einfache Auswahl der gewünschten Speisen.

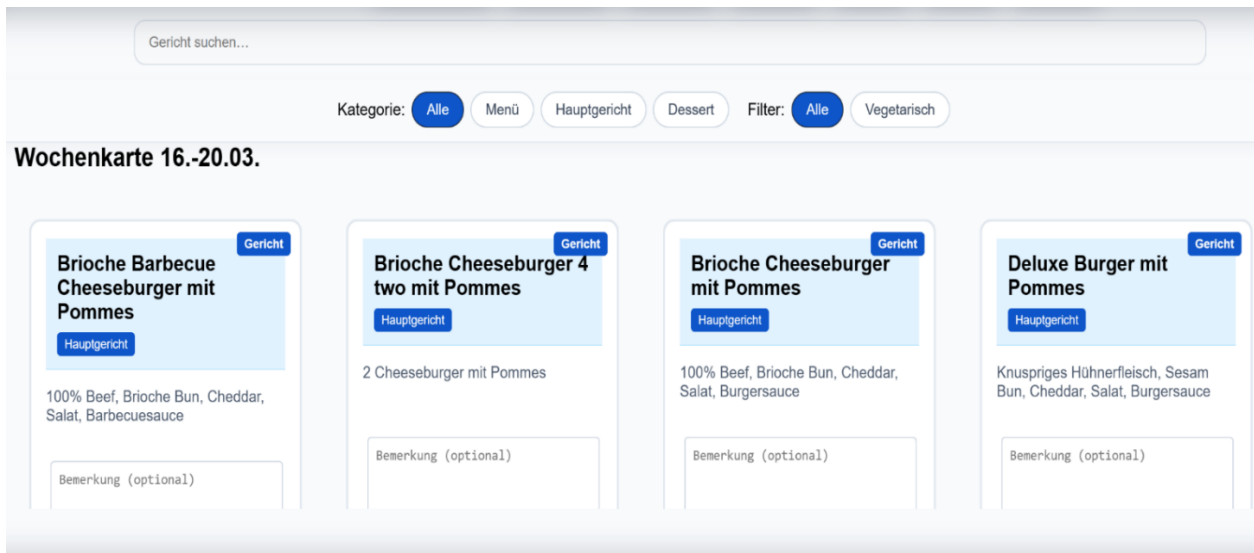


Abbildung 25: Ergebnisse - Menüübersicht Kunden

Wählt ein Benutzer ein oder mehrere Gerichte aus, werden diese im **Warenkorb** gesammelt. Die Warenkorbseite ist in *Abbildung 26* dargestellt. Hier können Mengen angepasst, Anmerkungen hinzugefügt und die Bestellung nochmals kontrolliert werden. Auf der rechten Seite wird zusätzlich eine Bestellübersicht angezeigt, in der die ausgewählten Gerichte, die Gesamtsumme und das verfügbare Guthaben dargestellt werden. Dadurch erhalten Benutzer vor dem Absenden der Bestellung einen klaren Überblick über ihre Auswahl. Zusätzlich kann in diesem Bereich eine gewünschte Abholzeit ausgewählt werden, was die Organisation der Ausgabe weiter verbessert.

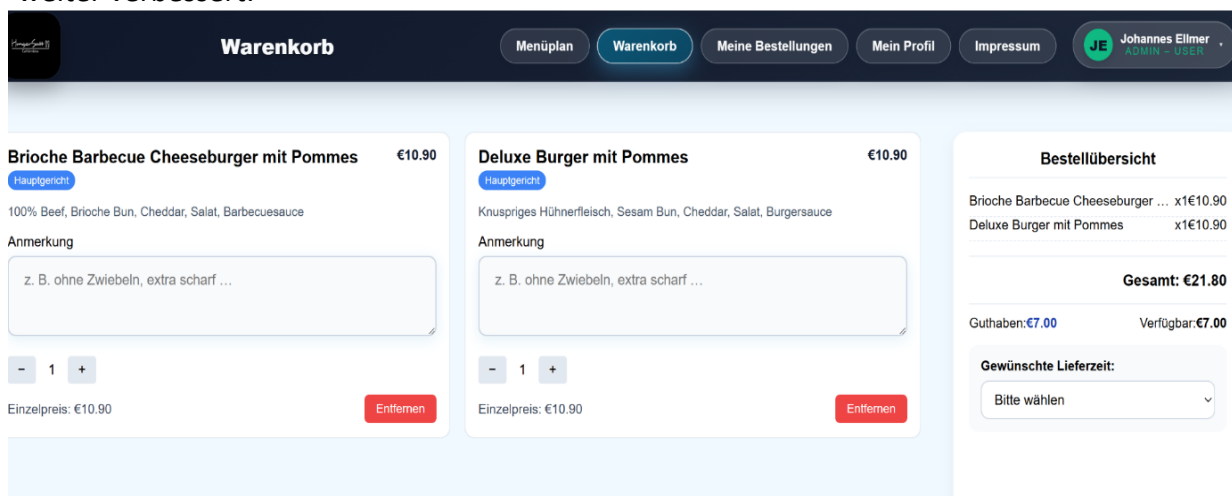


Abbildung 26: Ergebnisse - Warenkorb

Ein zentraler Bestandteil des Systems ist das integrierte **Guthabensystem**. Bestellungen können nur durchgeführt werden, wenn ausreichend Guthaben vorhanden ist. Das Guthaben wird bei einer Bestellung automatisch reduziert, wodurch die Zahlungsabwicklung stark vereinfacht wird. Gleichzeitig wird der organisatorische Aufwand für das Buffet reduziert, da keine direkte Barzahlung im Bestellprozess notwendig ist. Die Verwaltung dieses Bereichs erfolgt über die Kontoseite, die in *Abbildung 27* dargestellt ist. Dort können Benutzer ihre persönlichen Daten einsehen, ihre Klasse ändern, den aktuellen Guthabenstand prüfen und den Verlauf ihrer bisherigen Bestellungen nachvollziehen. Zusätzlich ist ersichtlich, dass auch der Guthabenverlauf dokumentiert wird. Diese Transparenz ist für die Benutzer besonders wichtig, da sie jederzeit nachvollziehen können, welche Bestellungen bereits durchgeführt wurden und wie sich ihr Guthaben verändert hat.

Mein Konto

johannes.ellmer@htl-saalfelden.at

KONTO
Aktiv

PROFIL

Deine Kontodaten (teilweise schreibgeschützt)

Name

E-Mail

Klasse

Ändern ist möglich – dadurch wird dein Account ggf. wieder freigeschaltet.

Klasse ändern

Bestellungen gesamt: 11
Guthaben: 7,00 €
Status: Aktiv

Passwort ändern

GUTHABEN

Aufladen / Ausleeren erfolgt über QR-Code Scan durch den Inhaber

GUTHABEN

7,00 €

Änderungen werden erst nach QR-Scan durch den Inhaber wirksam.

Aufladen starten **Guthaben ausleeren**

BESTELLUNGEN

11 Einträge

- Bestellung #66e8b070-b869-4... Abgeschlossen
19.03.2026, 12:09
- Bestellung #b61b104e-0339-4... Abgeschlossen
19.03.2026, 12:08
- Bestellung #bb1d0254-8ca5-4... Abgeschlossen
05.03.2026, 10:40
- Bestellung #d851e7ff-06f1-44... Abgeschlossen
19.02.2026, 11:13
- Bestellung #e6c5fb06-b3d1-4... Abgeschlossen
19.02.2026, 10:54
- Bestellung #4225ecdd-c2db-4... Abgeschlossen
16.02.2026, 11:26
- Bestellung #a19dd00a-e583-4... Abgeschlossen
13.02.2026, 10:22

GUTHABEN-VERLAUF

15 Einträge

Abbildung 27: Ergebnisse - Mein Konto und Guthaben aufladen

Neben den kundenbezogenen Funktionen wurde auch eine umfassende Verwaltungsoberfläche für den Betreiber umgesetzt. Ein besonders wichtiger Bereich ist dabei die **Menüverwaltung**, die in *Abbildung 28* zu sehen ist. Über diese Oberfläche können Menüs erstellt, bearbeitet und gespeichert werden. Der Betreiber kann Gerichte in ein aktuelles Menü übernehmen, bestehende Menüs anpassen und festlegen, welche Speisen aktiv angeboten werden. Dadurch ist es möglich, Wochenkarten flexibel zu planen und kurzfristig auf Änderungen zu

reagieren. Diese Funktion bildet die Grundlage für den digitalen Speiseplan und ist für den täglichen Betrieb des Schulbuffets von zentraler Bedeutung.

The screenshot displays a web-based menu management interface. At the top, there is a 'MENÜ-TITEL' field containing 'KW14'. Below this, the interface is split into two main sections: 'AKTUELLES MENÜ' and 'VERFÜGBARE GERICHTE'. The 'AKTUELLES MENÜ' section shows two items already added to the menu: 'Brioche Barbecue Cheeseburger mit Pommes' and 'Brioche Cheeseburger 4 two mit Pommes', both marked as 'Im Menü'. The 'VERFÜGBARE GERICHTE' section lists six available items: 'Brioche Cheeseburger mit Pommes', 'Deluxe Burger mit Pommes', 'Double Barbecue Cheeseburger mit Pommes', and 'Double Cheeseburger mit Pommes', all marked as 'Nicht im Menü'. Each item has a red trash icon, a blue edit icon, and a grey left arrow icon. At the bottom of each section are buttons: 'Menü speichern' for the current menu and 'Gericht hinzufügen' for adding items.

Abbildung 28: Ergebnisse - Menüverwaltung: Menüs erstellen

Ein weiterer wesentlicher Bestandteil des Systems ist die digitale Unterstützung der **Essensausgabe**. Dafür wurde ein QR-Code-basiertes Verfahren entwickelt. Der entsprechende Scanner-Bereich ist in *Abbildung 29* dargestellt. In dieser Ansicht kann der Betreiber oder eine berechtigte Person den Scanner starten, eine Kamera auswählen und anschließend Guthaben- oder Bestellcodes prüfen. Wird ein gültiger Code erkannt, kann die zugehörige Aktion verarbeitet werden. Bei Bestellungen bedeutet dies, dass die Bestellung eindeutig identifiziert und als abgeschlossen markiert wird. Dadurch wird verhindert, dass dieselbe Bestellung mehrfach eingelöst werden kann. Gleichzeitig wird die Essensausgabe beschleunigt, da Bestellungen schnell und eindeutig überprüft werden können.

The screenshot shows a mobile application interface for a QR scanner. The title is 'Guthaben · QR-Scanner' with a subtitle 'BalanceReq-Codes prüfen, bestätigen und Bestellungen per QR abschließen.' Below the title are two buttons: 'Scanner starten' (highlighted in blue) and 'Stoppen'. Under the 'Kamera' section, a dropdown menu shows 'Acer FHD User Facing (0408:4036)'. There are three checkboxes: 'Auto-Stop bei Erfolg' (checked), 'Sound' (checked), and 'Vibration' (checked). At the bottom, there is a 'LIVE SCAN' section with a 'QR-Erkennung' label and a 'Gestoppt' status. A large dark rectangle represents the camera view, with the text 'Scanner gestoppt' and 'Starte den Scanner, um Guthaben- oder Bestellcodes zu verarbeiten.' Below this is a 'Jetzt starten' button.

Abbildung 29: Ergebnisse - QR-Code Scanner - Handyansicht

Neben den Kernfunktionen wurden auch rechtliche Anforderungen in die Anwendung integriert. Dazu gehört insbesondere die **Impressums- und Datenschutzseite**, die in *Abbildung 30* dargestellt ist. Diese Seite enthält die wichtigsten rechtlichen Informationen zum Betreiber sowie Angaben zur Verarbeitung personenbezogener Daten. Die Integration dieser Inhalte ist notwendig, um die Anwendung vollständig und rechtlich konform bereitzustellen. Zusätzlich wurde darauf geachtet, dass diese Informationen auch ohne Login erreichbar sind, damit sie jederzeit eingesehen werden können.

Impressum	
Kontaktinformationen und grundlegende Informationen zum Betreiber	
Webseitenbetreiber	Markus Gruber
Projektname	HungerSatt Schulbistro
Adresse	Alte Bundesstraße 11, 5600 St. Johann
E-Mail	mahlzeit.hungersatt@gmail.com
UID	68016602

Datenschutzerklärung Informationen zur Verarbeitung personenbezogener Daten	Datenschutz als PDF herunterladen
---	---

Abbildung 30: Ergebnisse - Impressumseite

Darüber hinaus wurde eine **PDF-Exportfunktion** für Speisepläne umgesetzt. Dadurch können Menüs nicht nur digital innerhalb der Webanwendung angezeigt, sondern auch als PDF-Datei exportiert und bei Bedarf ausgedruckt oder außerhalb des Systems weitergegeben werden. Diese Funktion ist besonders praktisch für Aushänge oder für Personen, die den Speiseplan nicht direkt in der Anwendung aufrufen möchten. Damit verbindet das System digitale Verwaltung mit klassischen organisatorischen Anforderungen des Schulalltags.

Ein weiterer wichtiger technischer Bestandteil des Ergebnisses ist die integrierte **Swagger-API-Dokumentation**. Diese wird automatisch auf Basis des Backend-Codes generiert und dokumentiert die vorhandenen *REST*-Endpunkte des Systems. Dadurch entsteht eine übersichtliche und jederzeit aktuelle Darstellung aller Schnittstellen. In *Swagger* sind die verfügbaren Endpunkte nach Themenbereichen gegliedert, beispielsweise für Benutzer, Bestellungen, Menüs oder Guthaben. Für jeden Endpunkt werden die benötigten Parameter, Anfrageformate und möglichen Rückgaben dargestellt. Zusätzlich ermöglicht Swagger das direkte Testen der Schnittstellen über die Benutzeroberfläche. Dies erleichtert sowohl die Entwicklung als auch die spätere Wartung des Systems erheblich, da neue Funktionen schnell überprüft und bestehende Schnittstellen einfach nachvollzogen werden können. Besonders bei einem System mit mehreren Bereichen wie Benutzerverwaltung, Bestelllogik und Rollenrechten stellt diese Dokumentation einen wichtigen Mehrwert dar. In der nebenstehenden *Abbildung 31* ist ein Ausschnitt dieser *Swagger*-Dokumentation ersichtlich.

AdminUsers	
POST	/api/admin/users/balance/preview
POST	/api/admin/users/balance/confirm
GET	/api/admin/users/pending-deletions
POST	/api/admin/users/{id}/purge/preview
DELETE	/api/admin/users/{id}/purge
MyOrders	
GET	/api/orders/my
POST	/api/orders
AdminOrders	
GET	/api/admin/orders
PATCH	/api/admin/orders/complete
GET	/api/admin/orders/{id}
PATCH	/api/admin/orders/{id}
DELETE	/api/admin/orders/{id}
MenuItems	
GET	/api/menu-items
POST	/api/menu-items
GET	/api/menu-items/{id}
PATCH	/api/menu-items/{id}
DELETE	/api/menu-items/{id}
Menus	
POST	/api/menus
GET	/api/menus
GET	/api/menus/{id}
PATCH	/api/menus/{id}
DELETE	/api/menus/{id}

Abbildung 31: Ergebnisse - Ausschnitt Swagger-Dokumentation

Zusammenfassend entstand im Rahmen dieser Diplomarbeit ein vollständig funktionsfähiges, digitales Bestellsystem für ein Schulbuffet. Das System deckt alle wesentlichen Prozesse ab, die für den täglichen Betrieb notwendig sind. Dazu zählen die Erstellung und Verwaltung von Speiseplänen, die Bestellung durch Kundinnen und Kunden, die Verwaltung von Guthaben, die Kontrolle der Essensausgabe per QR-Code, die Benutzerverwaltung sowie die technische Dokumentation der Schnittstellen mit *Swagger*. Durch die klare Trennung zwischen Frontend, Backend und Datenbank wurde eine modulare Architektur geschaffen, die eine einfache Wartung und spätere Erweiterung ermöglicht.

Das entwickelte System stellt damit eine praxisnahe und technisch saubere Lösung dar, die den Bestellprozess im Schulbuffet deutlich vereinfacht. Sowohl für Benutzer als auch für den Betreiber entsteht ein strukturierter, nachvollziehbarer und effizienter Bestellablauf und Buffetalltag.

6.2 Projektverlauf

Der Projektverlauf dieser Diplomarbeit erstreckte sich über mehrere Monate und umfasste verschiedene Phasen, von der ersten Planung bis hin zur Testphase des fertigen Systems. Zu Beginn des Projektes stand vor allem die Analyse der Anforderungen sowie die Planung der grundlegenden Systemarchitektur im Vordergrund. Ziel war es, eine klare Struktur für die Umsetzung des Bestellsystems zu schaffen und alle notwendigen Funktionen frühzeitig zu definieren.

Der erste Schritt bestand darin, die Anforderungen gemeinsam mit dem Auftraggeber zu besprechen. Dabei wurden die grundlegenden Abläufe des bestehenden Bestellprozesses analysiert und mögliche Verbesserungen durch eine digitale Lösung diskutiert. Auf Basis dieser Gespräche wurde anschließend das Pflichtenheft erstellt, welches die funktionalen und technischen Anforderungen des Systems festhält. Dieses Dokument bildete die Grundlage für die weitere Entwicklung des Projektes.

Nach der Fertigstellung des Pflichtenheftes folgte die Planungs- und Konzeptionsphase. In diesem Abschnitt wurde die grundlegende Architektur des Systems entworfen. Dabei wurde entschieden, dass das Backend mit dem Framework *NestJS* umgesetzt und das Frontend als Angular-Webanwendung entwickelt wird. Zusätzlich wurde *PostgreSQL* als Datenbanksystem ausgewählt, welches über *Supabase* gehostet wird. Diese Technologieauswahl wurde getroffen, da sie eine stabile, skalierbare und gleichzeitig gut wartbare Grundlage für das System bietet.

Ein wichtiger Bestandteil der Planungsphase war außerdem die Recherche zu geeigneten Technologien und Lösungsansätzen. Dabei wurden verschiedene Möglichkeiten zur Umsetzung der Benutzerverwaltung, der *API-Struktur* sowie des Hostings untersucht. Besonders im Bereich der Authentifizierung und Rollenverwaltung wurden unterschiedliche Ansätze betrachtet, um eine sichere und zugleich praktikable Lösung zu finden.

Nach Abschluss der Konzeptionsphase begann die eigentliche Entwicklungsarbeit. Diese unterteilte sich in die Entwicklung des Frontends sowie des Backends. Zunächst wurden grundlegende Komponenten der Benutzeroberfläche umgesetzt, darunter die Startseite, die Anzeige der Speisepläne sowie die Bestellfunktion. In weiterer Folge wurde der Warenkorb implementiert, über den Benutzer ihre ausgewählten Gerichte verwalten und Bestellungen durchführen können. Parallel dazu wurde auch das Guthabensystem entwickelt, welches sicherstellt, dass Bestellungen nur bei ausreichendem Guthaben möglich sind.

Während der Entwicklung wurde das Projekt regelmäßig überprüft und besprochen. Ein wichtiger Meilenstein war die Pflichtenheftpräsentation am **23.10.2025**, bei der das geplante Konzept vorgestellt wurde. Darauf folgte das erste Audit am **20.11.2025**, bei dem der aktuelle Fortschritt sowie die technische Umsetzung besprochen wurden. Eine weitere wichtige Präsentation fand am **20.12.2025** statt, bei der der Zwischenstand des Projektes präsentiert wurde. Insgesamt kamen dort einige Anmerkungen hervor, die im weiteren Projektverlauf berücksichtigt wurden. (z.B. QR-Code Scan)

Parallel zur Frontendentwicklung wurde auch das Backend aufgebaut. Hier wurden zunächst die grundlegenden API-Strukturen erstellt sowie die Datenbank angebunden. Anschließend folgte die Implementierung der zentralen Funktionen wie Benutzerverwaltung, Menüverwaltung, Bestelllogik sowie das Guthabensystem. Zusätzlich wurde eine klare Trennung zwischen Benutzerfunktionen und administrativen Funktionen umgesetzt, um sowohl Kunden als auch dem Betreiber eine passende Benutzeroberfläche zur Verfügung zu stellen.

Im weiteren Verlauf des Projektes wurden auch Themen wie Hosting, Datenbanksicherheit sowie Rollen- und Zugriffsverwaltung umgesetzt. Diese Bereiche waren besonders wichtig, um einen sicheren und stabilen Betrieb des Systems zu gewährleisten. Dazu gehörte unter anderem die Verwaltung von Benutzerrollen sowie die Absicherung der API-Endpunkte.

Während der Projektphase fanden mehrere Besprechungen und Audits statt, bei denen der Fortschritt dokumentiert und weitere Schritte geplant wurden. Dazu zählen unter anderem Besprechungen am **13.01.2026**, **20.02.2026** sowie **03.03.2026**. Zusätzlich wurde am **17.02.2026** eine weitere Präsentation des aktuellen Entwicklungsstandes durchgeführt.

Ein wichtiger Bestandteil der letzten Projektphase war das Hosting des Systems. Dabei wurden sowohl das Frontend als auch das Backend auf einer geeigneten Infrastruktur bereitgestellt. Gleichzeitig wurde die Datenbank in die *Hostingumgebung* integriert, sodass das System vollständig online betrieben werden kann.

Nach Abschluss der Entwicklungsarbeiten begann die Testphase des Systems. Diese startete am **09.03.2026** und dient dazu, das System unter realen Bedingungen zu überprüfen. In dieser Phase werden mögliche Fehler identifiziert und gegebenenfalls behoben. Gleichzeitig wird überprüft, ob alle Funktionen wie geplant funktionieren und ob der Bestellprozess im Schulalltag reibungslos abläuft.

Der gesamte Arbeitsaufwand für das Projekt lag letztlich deutlich über den ursprünglich geplanten Stunden. Insgesamt wurden etwa **320 Stunden von Ellmer** sowie **250 Stunden von Magos** in das Projekt investiert. Diese zusätzliche Arbeitszeit ergab sich vor allem durch Erweiterungen des Systems, zusätzliche Tests sowie Anpassungen während der Entwicklung.

Die folgende *Abbildung 32* zeigt den geplanten zeitlichen Ablauf des Projektes in Form eines Projektzeitplans.

HungerSatt Online-Bestellservice	Dauer in Stunden		Sep			Oct			Nov			Dec			Jän			Feb			Mar										
Aufgabe	Ellmer	Magos	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	1	2	3	4	5	6	7	8	9	10	11	12	13
Anforderungen mit Markus besprechen	2	2																													
Pflichtenheft erstellen und Präsentation	10	10																													
Planung und Konzeption	5	5																													
Planung Abschluss	17	17																													
Recherche zu Technologien	10	10																													
Entwicklung des Frontends	41	42																													
Implementierung Menüpage	9	0																													
Erstellung des Warenkorbs	8	0																													
Dashboard	10	7																													
Entwicklung des Guthabensystems	10	2																													
Implementierung des Verwaltungssystems	0	25																													
Aufbau der Frontendservices	4	8																													
Entwicklung des Backends	30	32																													
API Schnittstelle - Frontendbridge	0	10																													
Multiservice-Architektur	0	15																													
Hosting Planen	15	5																													
Datenbank aufsetzen	10	1																													
Datenbank Bridge	5	1																													
Rollen und Benutzerverwaltung	38	26																													
Integration von Keycloak	8	6																													
Datenschutz und Datensicherheit	15	10																													
Implementation der Zugriffsverwaltung	15	10																													
Hosting	15	11																													
Hosting des Frontend	3	4																													
Backend und DB hosten	12	7																													
Abschluss	60	65																													
Testing und Demolauf	18	23																													
Dokumentation	40	40																													
Summe	211	203																													

Abbildung 32: Projektverlauf - Planung vs. Verlauf

Der Zeitplan verdeutlicht, dass sich das Projekt über mehrere Phasen erstreckt, die teilweise parallel ablaufen. Während zu Beginn vor allem Planung und Recherche im Vordergrund standen, verlagerte sich der Schwerpunkt später zunehmend auf die Entwicklung der einzelnen Systemkomponenten sowie auf Tests und Dokumentation.

Zusammenfassend lässt sich sagen, dass das Projekt strukturiert und schrittweise umgesetzt wurde. Durch regelmäßige Präsentationen, Audits und Besprechungen konnte der Fortschritt kontinuierlich überprüft und gegebenenfalls angepasst werden. Die Kombination aus Planung, Entwicklung, Tests und Dokumentation führte letztlich zu einem stabilen und funktionsfähigen System, das nun im praktischen Einsatz getestet werden kann.

6.3 Zukunftsausblick

Mit dem erfolgreichen Abschluss des Projekts liegt ein funktionsfähiges und praxistaugliches System vor, das den definierten Anwendungsfall – die digitale Verwaltung von Bestellungen inklusive rollenbasierter Steuerung und *QR-gestütztem Ausgabeprozess* – vollständig abbildet. Dennoch ergeben sich im Hinblick auf den realen Einsatzbetrieb weitere sinnvolle Entwicklungsperspektiven.

Ein zentraler Aspekt betrifft die Weiterentwicklung im laufenden Betrieb. Mit steigender Nutzerzahl – beispielsweise bei mehreren teilnehmenden Standorten oder einer größeren Anzahl gleichzeitiger Bestellungen – gewinnt die *Skalierbarkeit* weiter an Bedeutung. Hier könnten zusätzliche Maßnahmen im Bereich *Performance-Optimierung*, *Monitoring* sowie strukturierter *Fehleranalyse* implementiert werden, um einen dauerhaft stabilen Betrieb sicherzustellen.

Auch funktional bietet der konkrete Anwendungskontext weiteres Potenzial. Denkbar wäre eine erweiterte *Auswertungs- und Reporting-Funktion*, um beispielsweise Bestellvolumen, Auslastung oder besonders häufig nachgefragte Speisen systematisch analysieren zu können. Für den Betreiber könnte dies eine wertvolle Entscheidungsgrundlage zur Optimierung des Angebots darstellen.

Darüber hinaus ließe sich das bestehende Rollen- und Berechtigungssystem künftig um zusätzliche organisatorische Strukturen erweitern, etwa zur Abbildung mehrerer Standorte oder getrennter Verwaltungsbereiche. Die modulare Architektur des Systems schafft hierfür eine stabile Grundlage.

Zusammenfassend stellt das Projekt keinen endgültig abgeschlossenen Endzustand dar. Zwar wurde ein funktionsfähiges und stabiles System realisiert, dennoch bestehen weiterhin sinnvolle Erweiterungs- und Optimierungsmöglichkeiten – sowohl im funktionalen als auch im technischen Bereich. Beispielsweise könnte ein Benachrichtigungssystem eingebaut werden, durch das der Inhaber informiert wird, wenn eine neue Bestellung eingegangen ist. Während der Entwicklung sind auch noch weitere Features wie die Bestätigung mittels QR-Codes umgesetzt worden. Darüber hinaus kann nur durch regelmäßige Wartung ein langfristiger, sicherer und leistungsfähiger Betrieb gewährleistet werden.

Abbildungsverzeichnis

Abbildung 1: Zeitplan Gantt-Chart	6
Abbildung 2: Überblick Projektablauf	Fehler! Textmarke nicht definiert.
Abbildung 3: Persona 1	12
Abbildung 4: Persona 2	12
Abbildung 5: Persona 3	13
Abbildung 6: direct vs. indirect Kommunikation; Sommerville Microservice Architecture	28
Abbildung 7: Orchestration vs. Choreography – Sommerville Microservices Architecture	28
Abbildung 8: Client-Server Kommunikation [73]	30
Abbildung 9: Aufbau Container - Cloud based Software Sommerville F. 8	31
Abbildung 10: Docker Ecosystem - Cloud-based software (Sommerville) F. 11	32
Abbildung 11: Kontextdiagramm – C4 Diagramm	42
Abbildung 12: Architekturdiagramm.....	43
Abbildung 13: Containerdiagramm - C4-Diagramm.....	44
Abbildung 14: Komponentenansicht - Backend.....	45
Abbildung 15: Sequenzdiagramm: Login.....	46
Abbildung 16: Use-Case Diagramm.....	47
Abbildung 17: Aktivitätsdiagramm - Bestellprozess	48
Abbildung 18: Aktivitätsdiagramm: Verwaltungsprozess	49
Abbildung 19: Kontextdiagramm - C4	50
Abbildung 20: Containerdiagramm – C4	51
Abbildung 21: Komponentendiagramm – Backendservices – C4	52
Abbildung 22: Email-Aufbau.....	91
Abbildung 23: ER-Diagramm - Datenbankschema (Überblick)	93
Abbildung 24: Architekturdiagramm - Hosting	98
Abbildung 25: Ergebnisse - Menüübersicht Kunden.....	116
Abbildung 26: Ergebnisse - Warenkorb.....	116
Abbildung 27: Ergebnisse - Mein Konto und Guthaben aufladen	117
Abbildung 28: Ergebnisse - Menüverwaltung: Menüs erstellen.....	118
Abbildung 29: Ergebnisse - QR-Code Scanner - Handyansicht.....	118

Abbildung 30: Ergebnisse - Impressumseite	119
Abbildung 31: Ergebnisse - Ausschnitt Swagger-Dokumentation.....	120
Abbildung 32: Projektverlauf - Planung vs Verlauf	123
Abbildung 33: Aktivitätsdiagramm User - Anleitung.....	XXIV
Abbildung 34: Aktivitätsdiagramm ADMIN - Anleitung	XXVII

Tabellenverzeichnis

Tabelle 1: Abkürzungsverzeichnis	IX
Tabelle 2: Schätzung des Arbeitsaufwandes.....	5
Tabelle 3: geplante Projektphasen und Meilensteine	6
Tabelle 4: Rollenverteilung.....	8
Tabelle 5: Stakeholder-Matrix.....	10

Listingsverzeichnis

Listing 1: Frontend - Laden der Menus	53
Listing 2: Frontend - Navigation zum Menüplaner	54
Listing 3: Frontend - Speiseplan auswählen	54
Listing 4: Frontend - Speiseplan löschen	54
Listing 5: Frontend - Kurzansicht der Erstellung einer PDF mit jsPDF	55
Listing 6: Frontend - Logo zur Anzeige laden	55
Listing 7: Frontend – Ausgewählten Speiseplan laden	56
Listing 8: Frontend - Gerichte des gewählten Spieseplan laden	56
Listing 9: Frontend - Gerichte in Drop Listen	57
Listing 10: Frontend - Liste der gewählten Gerichte anzeigen	57
Listing 11: Frontend - Drag and drop Funktion	57
Listing 12: Frontend - Aktivieren /Deaktivieren von Gerichten	58
Listing 13: Frontend - Laden von bereits bestehenden Gerichten	59
Listing 14: Frontend - dynamische Veränderung der Formularlogik	59
Listing 15: Frontend - Allergene Logik	60
Listing 16: Frontend - Validierung der Eingaben beim Erstellen von Gerichten	60
Listing 17: Frontend - Speichern des neuen Gerichtes als Menuitem	61
Listing 18: Frontend - Speichern des erstellten Gerichtes	61
Listing 19: Frontend – Regelmäßiges prüfen, geänderter Daten	62
Listing 20: Frontend - regelmäßiges Laden eines Menüs	62
Listing 21: Frontend - Filteroptionen	62
Listing 22: Frontend - Holen der Daten von einem Gericht bzw. einem Menü	63
Listing 23: Frontend - Auswahl eines Gerichtes für den Warenkorb	63
Listing 24: Frontend – Mengensteuerung für den Warenkorb	64
Listing 25: Frontend – Ausprogrammierung increment Methode	64
Listing 26: Frontend - Überprüfung verfügbarer Gerichte	64
Listing 27: Frontend - Absenden der Bestellung	65
Listing 28: Frontend - Alle Bestellungen des jeweiligen Kunden werden geladen	65
Listing 29: Frontend - Trennung zwischen aktiven und abgeschlossenen Bestellungen	66

Listing 30: Frontend - Erstellung und Laden des QR-Codes zur Bestellbestätigung	66
Listing 31: Frontend - dynamische Erstellung des Namens der Bestellung	66
Listing 32: Frontend Kamerazugriff für QR-Code Scan	67
Listing 33: Frontend - Definiere den Scan-Bereich und starte den Scan	68
Listing 34: Frontend - Unterscheidung zwischen Guthaben- und Bestellungscode	68
Listing 35: Frontend - Abschluss der Bestellung	68
Listing 36: Frontend - Bestätigung der Guthabenänderung	69
Listing 37: Frontend - Gesamtdefinition und Ablauf der Bestellbestätigung	69
Listing 38: Frontend - Verwendung eines Paginator für bessere Übersichtlichkeit	70
Listing 39: Frontend - Grundsätzliche Definition für statistische Auswertungen	70
Listing 40: Frontend - Auswahl unterschiedlicher Statistiken.....	71
Listing 41: Frontend - Exportieren der Statistikdaten als PDF	71
Listing 42: Frontend - Routing bei Authentifizierung	72
Listing 43: Frontend - Automatisches Ausfüllen der Namen	72
Listing 44: Frontend - Routing, nachdem die Registrierung bestätigt, wurde	72
Listing 45: Frontend - Passwort zurücksetzen.....	73
Listing 46: Frontend - Neues Passwort und Token senden.....	73
Listing 47: Frontend - Definition freier Routen	75
Listing 48: Frontend - Erstellung der Tokens.....	75
Listing 49: Frontend - Automatischer Logout bei falschen Token	76
Listing 50: Frontend - Einsatz des HTTP-Interceptors	76
Listing 51: Frontend - Verwendung von Route Guard zur Sicherung geschützter Routen	77
Listing 52: Frontend - Routen und Rollen Definition	77
Listing 53: Frontend - Implementierung des HTTP-Interceptors	77
Listing 54: Backend - Entry-Points und Zugriffsrechte	78
Listing 55: Backend - Erstellung und Kontrolle des Tokens.....	79
Listing 56: Backend - Kontrolle des rollenbasierten Zugriffs	80
Listing 57: Backend - Prüfen nach geänderter Klasse - Guard	80
Listing 58: Backend - Prüfen nach geänderter Klasse - UserService	81
Listing 59: Backend - Endpoint für Zugriff auf eigene Daten	81
Listing 60: Backend - AppSettings zur Deaktivierung des Bestellfeatures	82

Listing 61: Backend - Sicherstellen von DB Prinzipien.....	82
Listing 62: Backend - Sicherstellen von DB - Prinzipien - Rollback.....	83
Listing 63: Backend - Ablauf und Prüfen der Bestellungen innerhalb der Transaktion	83
Listing 64: Backend - Abschluss einer Order	84
Listing 65: Backend - Transaktionaler Abschluss von Orders.....	84
Listing 66: Backend - Der Abschluss setzt die Bestellung als delivered	84
Listing 67: Backend - Validierung beim Aufladen des Guthabens	85
Listing 68: Backend - Requests werden während validierung zwischengespeichert.....	85
Listing 69: Backend - Guthaben leeren	86
Listing 70: Backend - Guthaben ändern	86
Listing 71: Backend - Guthabenänderungen werden mitgeloggt	87
Listing 72: Backend - Abschluss der Guthabentransaktion	87
Listing 73: Backend - Nur Inhaber und Admin kann Guthaben ändern	87
Listing 74: Backend - Regeln zur Guthabenänderungen	88
Listing 75: Backend - Erzeugung von Tokens	89
Listing 76: Backend - Speichern von Requests, die sich registrieren wollen	89
Listing 77: Backend - Sessions laden und erneuern	90
Listing 78: Backend - Speicherung vom Token zur Authentifizierung.....	90
Listing 79: Backend - Der Token und das Passwort wird gehasht gespeichert#.....	90
Listing 80: Backend - Email Erstellung und sendenki8	92
Listing 81: Testing - Testaufbau und Mocking.....	102
Listing 82: Testing - Filtern nach Usern	102
Listing 83: Testing – Hinzufügung des QR-Codes zu offenen Bestellungen	102
Listing 84: Testing - Erzeugung QR-Code	103
Listing 85: Testing - HTTP-Fehlerbehandlung OrderService.....	103
Listing 86: Testing - Laden der Bestellungen im OrderOverviewComponent.....	104
Listing 87: Testing - Filtern der Bestellungen nach Status	104
Listing 88: Testing - Berechnung des Gesamtpreises einer Bestellung.....	104
Listing 89: Testing - Fehlerbehandlung beim Laden der Bestellungen	104
Listing 90: Testing - Ein- und Ausblenden von Bestelldetails.....	105
Listing 91: Testing - Navigation vom OrderOverviewComponent zur Startseite.....	105

Listing 92: Testing - Initialisierung des CartPageComponents	106
Listing 93: Testing - Erhöhung und Verringerung von Artikel im Warenkorb.....	106
Listing 94: Testing - Aktualisierung und Entfernung eines Artikels aus dem Warenkorb.....	106
Listing 95: Testing - Berechnung des Gesamtpreises aller Artikel im Warenkorb.....	106
Listing 96: Testing - Fehlerbehandlung bei leerem Warenkorb.....	107
Listing 97: Testing - Fehlerbehandlung unzureichendes Guthaben.....	107
Listing 98: Testing - Abschluss der Bestellung.....	107
Listing 99: Testing – Navigation vom CartPageComponent zur Startseite	107
Listing 100: Testing - Initialisierung: Erstellung eines JWT	109
Listing 101: Testing - Authorization Header	109
Listing 102: Testing - Zugriff Endpoints - Admin	110
Listing 103: Testing - Unautorisierter Zugriff auf Bestellungen	110
Listing 104: Testing - Validierungsfehler bei Orders	110

Literaturverzeichnis

- [1] Proxmox Server Solutions GmbH: Proxmox Virtual Environment (VE) – Open-Source Server-Virtualisierungsplattform. URL: <https://www.proxmox.com/en/products/proxmox-virtual-environment/overview> (Abgerufen am 01.01.2026). Informationen zur Produktübersicht und Eigenschaften der Virtualisierungsplattform.
- [2] Amazon Web Services, Inc.: AWS Fargate – Serverless Compute Engine. URL: <https://aws.amazon.com/de/fargate/> (Abgerufen am 01.01.2026). Produktseite zu AWS Fargate als serverloser Container-Laufzeitdienst.
- [3] Amazon Web Services, Inc.: Amazon RDS für PostgreSQL. URL: <https://aws.amazon.com/de/rds/postgresql/> (Abgerufen am 01.01.2026). Informationen zu relationalem Datenbankservice mit PostgreSQL.
- [4] Microsoft: Azure App Service Plans – Übersicht. URL: <https://learn.microsoft.com/en-us/azure/app-service/overview-hosting-plans> (Abgerufen am 01.01.2026). Überblick über Service-Pläne im Azure App Service.
- [5] Microsoft: Scale Up Features and Capacities – Azure App Service. URL: <https://learn.microsoft.com/en-us/azure/app-service/manage-scale-up> (Abgerufen am 01.01.2026). Skalierungsoptionen im Azure App Service.
- [6] Microsoft: Azure Database for PostgreSQL – Produktübersicht. URL: <https://learn.microsoft.com/en-us/azure/postgresql/overview> (Abgerufen am 01.01.2026). Allgemeine Informationen zum PostgreSQL-Datenbankdienst in Azure.
- [7] World4You Internet Services GmbH: Webhosting Österreich. URL: <https://www.world4you.com/hosting/web-hosting> (Abgerufen am 30.12.2025). Informationen zu Webhosting-Paketen und Serviceangebot.
- [8] World4You Internet Services GmbH: Webhosting-Blog – Kategorie Webhosting. URL: <https://www.world4you.com/blog/kategorie/webhosting/> (Abgerufen am 30.12.2025). Blogbeiträge und Tipps zu Webhosting-Themen.
- [9] Google LLC: Angular – What is Angular? URL: <https://v17.angular.io/guide/what-is-angular> (Abgerufen am 31.12.2025). Grundlegende Einführung in das Angular-Framework.
- [10] Google LLC: Angular Architecture Overview. URL: <https://v17.angular.io/guide/architecture> (Abgerufen am 31.12.2025). Beschreibung der Architektur von Angular-Applikationen.
- [11] Google LLC: Communicating with Backend Services Using HTTP. URL: <https://v17.angular.io/guide/understanding-communicating-with-http> (Abgerufen am 31.12.2025). Erläuterung der HTTP-Kommunikation in Angular.
- [12] NestJS Team: NestJS Documentation. URL: <https://docs.nestjs.com/> (Abgerufen am 31.12.2025). Offizielle Dokumentation des NestJS-Frameworks.

- [13] NestJS Team: Providers – NestJS Core Concepts. URL: <https://docs.nestjs.com/providers> (Abgerufen am 31.12.2025). Dokumentation zu Providern und Abhängigkeitsinjektion in NestJS.
- [14] Microsoft (TypeScript Team): The TypeScript Handbook. URL: <https://www.typescriptlang.org/docs/handbook/intro.html> (Abgerufen am 31.12.2025). Offizielles Handbook zu TypeScript.
- [15] Microsoft (TypeScript Team): TypeScript in 5 Minutes. URL: <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html> (Abgerufen am 31.12.2025). Einstieg in TypeScript für JavaScript-Programmierer.
- [16] Bundesministerium des Innern: Datenschutzerklärung – Datenschutz allgemein. URL: https://www.bmi.bund.de/DE/service/datenschutz/datenschutz_node.html (Abgerufen am 28.12.2025). Überblick zu Datenschutzprinzipien des BMI.
- [17] Landesbeauftragte für Datenschutz und Informationsfreiheit NRW: Datenschutz. URL: <https://www.ldi.nrw.de/datenschutz> (Abgerufen am 28.12.2025). Landesrechtliche Hinweise zum Datenschutz.
- [18] Europäische Union: Datenschutz-Grundverordnung (EU) 2016/679. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Abgerufen am 28.12.2025). Rechtsakt zur EU-DSGVO.
- [19] European Commission: Principles of the GDPR. URL: https://commission.europa.eu/law/law-topic/data-protection/principles-gdpr_en (Abgerufen am 28.12.2025). Erläuterung der Grundprinzipien der DSGVO durch die EU-Kommission.
- [20] National Institute of Standards and Technology (NIST): Digital Identity Guidelines, SP 800-63B. URL: <https://pages.nist.gov/800-63-3/sp800-63b.html> (Abgerufen am 28.12.2025). Richtlinien zu digitalen Identitäten.
- [21] OWASP Foundation: Authentication Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Abgerufen am 28.12.2025). Best Practices zur sicheren Authentifizierung.
- [22] Microsoft: Multi-Factor Authentication – Microsoft Entra ID. URL: <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-mfa-howitworks> (Abgerufen am 28.12.2025). Grundlagen zur Multi-Factor Authentication.
- [23] Jones, M.: Bearer Token Usage, RFC 6750. RFC Editor, 2012. URL: <https://www.rfc-editor.org/rfc/rfc6750> (Abgerufen am 28.12.2025). Spezifikation zur Verwendung von Bearer Tokens.
- [24] Jones, M.; Bradley, J.; Sakimura, N.: JSON Web Token (JWT), RFC 7519. RFC Editor, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519> (Abgerufen am 28.12.2025). Standard für JSON Web Tokens.

- [25] Fielding, R. T.; Taylor, R. N.: Architectural Styles and the Design of Network-based Software Architectures. ACM, 2000. DOI:10.1145/514183.514185 (Abgerufen am 28.12.2025). Grundlegender Artikel zur REST-Architektur.
- [26] OWASP Foundation: Forgot Password Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html (Abgerufen am 28.12.2025). Leitfaden zur sicheren Implementation von Passwort-Recovery.
- [27] World Wide Web Consortium (W3C): Cross-Origin Resource Sharing (CORS). URL: <https://www.w3.org/TR/cors/> (Abgerufen am 28.12.2025). W3C-Standard zur CORS-Policy.
- [28] PortSwigger Ltd.: What is CORS? URL: <https://portswigger.net/web-security/cors> (Abgerufen am 30.12.2025). Erklärung und Sicherheitskontext zu CORS.
- [29] Codd, E. F.: A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6):377–387, 1970. Klassiker zur relationalen Datenbanktheorie.
- [30] Watt, A.: Database Design – Chapter 7: The Relational Data Model. 2nd Edition, 2014. Lehrbuchkapitel zur relationalen Modellierung.
- [31] PostgreSQL Global Development Group: The SQL Language. URL: <https://www.postgresql.org/docs/current/sql.html> (Abgerufen am 30.12.2025). Dokumentation zur SQL-Syntax und -Funktionalität.
- [32] Watt, A.: Database Design – Chapter 12: Normalization. 2nd Edition, 2014. Lehrbuchkapitel zu Datenbanknormalisierung.
- [33] PostgreSQL Global Development Group: Transactions. URL: <https://www.postgresql.org/docs/current/tutorial-transactions.html> (Abgerufen am 30.12.2025). Konzepte zu Transaktionen in PostgreSQL.
- [34] PostgreSQL Global Development Group: Write-Ahead Logging (WAL). URL: <https://www.postgresql.org/docs/current/wal.html> (Abgerufen am 30.12.2025). Logging-Mechanismen in PostgreSQL.
- [35] PostgreSQL Global Development Group: A Brief History of PostgreSQL. URL: <https://www.postgresql.org/docs/current/history.html> (Abgerufen am 30.12.2025). Historische Entwicklung des PostgreSQL-Projekts.
- [36] Chauhan, C.: Understanding the PostgreSQL Architecture. Severalnines Blog. URL: <https://severalnines.com/blog/understanding-postgresql-architecture/> (Abgerufen am 30.12.2025). Architektur und Aufbau von PostgreSQL.
- [37] Bhupathi, S.: PostgreSQL MVCC. OPENSOURCE DB Pvt. Ltd., 2024. Einführung in Multi-version Concurrency Control (MVCC).
- [38] Merkel, D.: Docker – Lightweight Linux Containers for Consistent Development and Deployment. Linux Journal, Juni 2014. DOI: (siehe Linux Journal).

- [39] Docker Inc.: What is Docker? URL: <https://docs.docker.com/get-started/docker-overview/> (Abgerufen am 30.12.2025). Grundlagen zu Docker-Containern.
- [40] Docker Inc.: Docker Networking. URL: <https://docs.docker.com/engine/network/> (Abgerufen am 30.12.2025). Netzwerkmodelle für Docker-Container.
- [41] Docker Inc.: Docker Volumes. URL: <https://docs.docker.com/engine/storage/volumes/> (Abgerufen am 30.12.2025). Persistente Speicherung in Docker.
- [42] Atlassian: Microservices Architecture. URL: <https://www.atlassian.com/microservices/microservices-architecture> (Abgerufen am 30.12.2025). Einführung in Microservices.
- [43] Richardson, C.: Microservice Architecture Pattern. <https://microservices.io/patterns/microservices.html> (Abgerufen am 30.12.2025). Sammlung von Musterlösungen für Microservices.
- [44] Microsoft: Microservices Architecture Style. URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices> (Abgerufen am 30.12.2025). Microservices-Leitfaden von Microsoft.
- [45] Heinrichs, T.: Orchestration vs. Choreography. Camunda Blog. URL: <https://camunda.com/blog/orchestration-vs-choreography/> (Abgerufen am 30.12.2025). Vergleich von Koordinationsmustern in verteilten Systemen.
- [46] Zimmermann, O.; et al.: Introduction to Microservice API Patterns. 2019. Buchtext/Whitepaper zu API-Designmustern.
- [47] GeeksforGeeks: Inter-Service Communication in Microservices. URL: <https://www.geeksforgeeks.org/inter-service-communication-in-microservices/> (Abgerufen am 30.12.2025). Erklärung zu Kommunikationsformen zwischen Microservices.
- [48] Microsoft: Cloud Design Patterns. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/> (Abgerufen am 30.12.2025). Sammlung bewährter Muster für Cloud-Architekturen.
- [49] OWASP Foundation: JSON Web Token Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_Cheat_Sheet.html (Abgerufen am 28.12.2025). Leitfaden zur sicheren Nutzung von JWT.
- [50] Kubernetes: Deployments. URL: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/> (Abgerufen am 31.12.2025). Konzepte zu Deployment-Controllern.
- [51] Kubernetes: Services, Load Balancing and Networking. URL: <https://kubernetes.io/docs/concepts/services-networking/> (Abgerufen am 31.12.2025). Netzwerk- und Servicekonzepte in Kubernetes.

- [52] Bundesamt für Sicherheit in der Informationstechnik (BSI): IT-Grundschutz-Kompendium. URL: <https://www.bsi.bund.de/it-grundschutz> (Abgerufen am 28.12.2025). Deutscher IT-Sicherheitsgrundschutz.
- [53] Microsoft: Saga Design Pattern. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/saga> (Abgerufen am 30.12.2025). Beschreibung des Saga-Musterns für verteilte Transaktionen.
- [54] Mell, P.; Grance, T.: The NIST Definition of Cloud Computing. NIST SP 800-145, 2011. Standarddefinition zum Cloud Computing.
- [55] Fang, L.; et al.: Cloud Computing – Concepts and Challenges. NIST Special Publication 500-292. URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf> (Abgerufen am 30.12.2025). Einführung in Cloud Computing-Konzepte.
- [56] Amazon Web Services, Inc.: What is Web Hosting? URL: <https://aws.amazon.com/what-is/web-hosting/> (Abgerufen am 30.12.2025). Erklärung zu Webhosting im Cloud-Kontext.
- [57] Amazon Web Services, Inc.: AWS Well-Architected Framework. URL: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html> (Abgerufen am 30.12.2025). Leitfaden zu Architektur-Best-Practices.
- [58] Microsoft: Azure Well-Architected Framework. URL <https://learn.microsoft.com/en-us/azure/well-architected/> (Abgerufen am 30.12.2025). Azure-spezifische Architekturprinzipien.
- [59] Microsoft: Azure Well-Architected Framework – Overview. URL: <https://learn.microsoft.com/en-us/azure/well-architected/what-is-well-architected-framework> (Abgerufen am 30.12.2025). Übersicht zu Architekturprinzipien.
- [60] Microsoft: Azure Application Architecture Fundamentals. URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/> (Abgerufen am 30.12.2025). Grundlagen zur Cloud-Architektur.
- [61] Render: Docker on Render. URL: <https://render.com/docs/docker> (Abgerufen am 30.12.2025). Informationen zur Nutzung von Docker bei Render.
- [62] Render: Create and Connect to Render PostgreSQL. URL: <https://render.com/docs/postgresql-creating-connecting> (Abgerufen am 30.12.2025). Anleitung zur PostgreSQL-Integration auf Render.
- [63] Parallax: jsPDF Documentation. URL: <https://parallax.github.io/jsPDF/docs/jsPDF.html> (Abgerufen am 30.12.2025). Dokumentation zu jsPDF.
- [64] Parallax: jsPDF – GitHub Repository. URL: <https://github.com/parallax/jsPDF> (Abgerufen am 30.12.2025). Quellcode-Repository zum jsPDF-Projekt.
- [65] Angular Team: Drag and Drop – Angular CDK. URL: <https://angular.dev/guide/drag-drop> (Abgerufen am 30.12.2025). Funktionen für Drag-and-Drop im Angular-CDK.

- [66] Scanbot SDK: HTML5 QR Code & Barcode Scanner Tutorial. URL: <https://scanbot.io/techblog/html5-barcode-scanner-tutorial/> (Abgerufen am 30.12.2025). Anleitung zur QR-Code-Integration im Browser.
- [67] Google: Gmail API – Node.js Quickstart. URL: <https://developers.google.com/workspace/gmail/api/quickstart/nodejs> (Abgerufen am 30.12.2025). Schnellstart zur Gmail API in Node.js.
- [68] Klensin, J.: Simple Mail Transfer Protocol (SMTP), RFC 5321. RFC Editor, 2008. URL: <https://www.rfc-editor.org/rfc/rfc5321.txt> (Abgerufen am 30.12.2025). Standardbeschreibung des SMTP-Protokolls.
- [69] Netlify, Inc.: What is Netlify? URL: <https://www.netlify.com/products/> (Abgerufen am 02.01.2026). Produktübersicht zu Netlify.
- [70] Netlify, Inc.: Continuous Deployment with Git. URL: <https://docs.netlify.com/site-deploys/overview/> (Abgerufen am 02.01.2026). Informationen zu Git-basierten Deployments.
- [71] Netlify, Inc.: Functions Overview. URL: <https://docs.netlify.com/functions/overview/> (Abgerufen am 02.01.2026). Überblick zu Serverlosen Funktionen bei Netlify.
- [72] Netlify, Inc.: Netlify CDN. URL: <https://www.netlify.com/products/cdn/> (Abgerufen am 02.01.2026). Informationen zum CDN-Service von Netlify.
- [73] *Abbildung 8: Client-Server Kommunikaton.* URL: [https://imgopt.infoq.com/fit-in/3000x4000/filters:quality\(85\)/filters:no_upscale\(\)/articles/api-first-integration/en/resources/1Figure-1-The-role-of-APIs-1604606456889.jpg](https://imgopt.infoq.com/fit-in/3000x4000/filters:quality(85)/filters:no_upscale()/articles/api-first-integration/en/resources/1Figure-1-The-role-of-APIs-1604606456889.jpg)

Anhang

- Diverse Diagramme
- Zwischenpräsentationen
- AGB und Datenschutzverordnung
- Kurzanleitung

Kurzanleitung zur Verwendung der Bestellapp

Dieses Kapitel beschreibt die Bedienung des Bestellsystems aus Sicht eines *Kunden* sowie eines *Admins/Inhabers*. Dabei werden die Abläufe so dokumentiert, dass sie im realen Betrieb direkt nachvollziehbar sind.

Bedienung aus der Sicht des *Kunden*

Zugriff, Registrierung und Login

1. Webanwendung öffnen
2. Falls noch kein Account vorhanden: *Registrieren* (E-Mail + Passwort)
3. Anschließend: *Login* durchführen
4. Nach erfolgreichem Login wird die Menü- bzw. Startseite angezeigt

Hinweis: Ohne Login sind keine Bestellungen möglich, da Bestellungen immer einem Benutzerkonto zugeordnet werden. Der gesamte darauffolgende Ablauf ist in *Abbildung 33* ersichtlich.

Menü ansehen und Speisen auswählen

1. Menü öffnen (Speisenübersicht)
2. Speise auswählen (Name, Preis, ggf. Beschreibung prüfen)
3. Speise über Hinzufügen in den Warenkorb legen
4. Menge im Warenkorb anpassen (+ / -), Positionen entfernen falls nötig

Wichtig: Der Warenkorb dient als letzte Kontrolle, bevor eine verbindliche Bestellung besteht.

Bestellung abschicken

1. Warenkorb öffnen (*Bestellübersicht*)
2. Positionen und Gesamtbetrag prüfen
3. Optional: *Notiz* ergänzen (z. B. „ohne Zwiebeln“, „bitte getrennt verpacken“)
4. *Bestellung absenden*

Nach dem Absenden passiert systemseitig:

- die Bestellung wird gespeichert und erhält den Status *open*
- der Betrag wird abgebucht (keine Abbuchung beim QR-Scan)
- ein QR-Code zur Abholung wird erzeugt
- die Bestellung ist im Bereich *Meine Bestellungen* sichtbar

Bestellung abholen (QR-Code)

1. Meine Bestellungen öffnen
2. Aktive Bestellung auswählen (Status open)
3. QR-Code anzeigen lassen
4. QR-Code bei der Ausgabe vorzeigen (Scan durch berechtigte Person)

Nach erfolgreichem Scan:

- Statuswechsel auf *closed*
- Bestellpositionen werden als *delivered=true* markiert
- die Bestellung ist abgeschlossen und kann nicht erneut „eingelöst“ werden

Aktivitätsdiagramm - User Ablauf (Bestellen & Abholen)

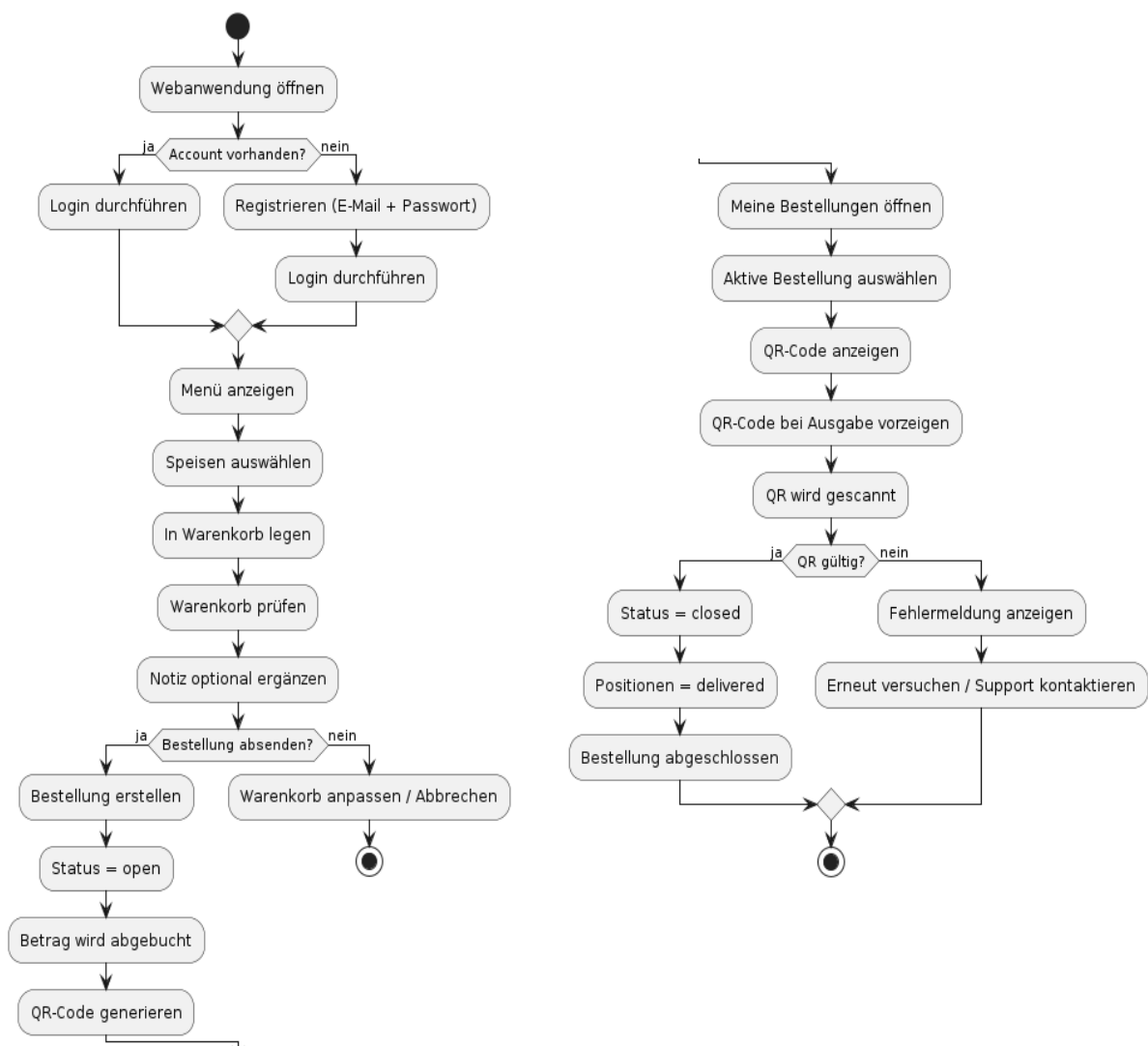


Abbildung 33: Aktivitätsdiagramm User - Anleitung

Bedienung aus der Sicht des Inhabers/Admins

1) Anmeldung und Zugriff auf den Verwaltungsbereich

1. Webanwendung öffnen
2. Login mit einem Account, der die Rolle *ADMIN* oder *INHABER* besitzt
3. Nach erfolgreicher Authentifizierung wird das *Dashboard* angezeigt

Im Dashboard stehen ausschließlich für diese Rolle freigegebene Funktionen zur Verfügung. Die Zugriffskontrolle erfolgt serverseitig über das implementierte Rollen- und Berechtigungskonzept.

2) Speisen verwalten (*Menüpflege*)

Die Menüverwaltung ist eine der zentralen Aufgaben des Admins, da das Angebot unmittelbar Einfluss auf die Bestellprozesse der User hat.

1. Navigieren zum Bereich *Speisenverwaltung*
2. „Neue Speise erstellen“ auswählen
3. Eingabe der erforderlichen Daten:
 - Name der Speise
 - Preis
 - Beschreibung (optional)
4. Speichern der Speise

Nach dem Speichern ist die Speise sofort im Menü sichtbar und kann von Usern bestellt werden.

Bestehende Speise bearbeiten

1. Auswahl einer vorhandenen Speise
2. Anpassung von:
 - Preis
 - Beschreibung
 - Verfügbarkeit
3. Änderungen speichern

Preisänderungen wirken sich unmittelbar auf zukünftige Bestellungen aus, jedoch nicht auf bereits erstellte Bestellungen.

Speise deaktivieren oder entfernen

Falls eine Speise temporär oder dauerhaft nicht angeboten wird:

1. Speise auswählen
2. Deaktivieren oder löschen
3. Bestätigen

Bereits bestehende Bestellungen bleiben unverändert bestehen.

3) Bestellungen überwachen

Der Admin hat Zugriff auf eine Übersicht aller Bestellungen.

1. Navigiere zum Bereich *Bestellungen*
2. Filterung nach Status *open*
3. Anzeige folgender Informationen:
 - Benutzer
 - Bestellzeitpunkt
 - Positionen und Mengen
 - Notizen

Diese Informationen dienen zur Vorbereitung in der Küche bzw. bei der Ausgabe.

Detailansicht einer Bestellung

In der Detailansicht kann der Admin:

- einzelne Positionen prüfen
- Notizen einsehen
- den aktuellen Status kontrollieren

Solange der Status *open* ist, gilt die Bestellung als noch nicht abgeholt.

4) QR-Scan und Abschluss einer Bestellung

Der Abschluss einer Bestellung erfolgt ausschließlich über den QR-Code-Prozess.

1. Öffnen der Scan-Funktion
2. Kamera aktivieren
3. QR-Code des Users scannen

Die enthaltene *Order-ID* wird an das Backend übertragen.

Serverseitige Validierung

Das System prüft automatisch:

- Existiert die Order-ID?
- Gehört sie zu einer gültigen Bestellung?
- Ist der Status noch *open*?
- Wurde die Bestellung bereits abgeschlossen?

Nur wenn alle Prüfungen erfolgreich sind, wird der Abschluss durchgeführt.

Statuswechsel

Bei erfolgreicher Validierung:

- Status wird von *open* auf *closed* gesetzt
- Alle Positionen werden als *delivered = true* markiert
- Die Bestellung gilt als endgültig abgeschlossen

Wichtig: Der QR-Scan löst **keine erneute Abbuchung** aus, da diese bereits beim Erstellen der Bestellung erfolgt ist.

Bei ungültigem oder bereits eingelöstem QR-Code wird eine entsprechende Fehlermeldung angezeigt.

Das nachfolgende Diagramm (*Abbildung 34*) veranschaulicht die möglichen Abläufe des Inhabers.

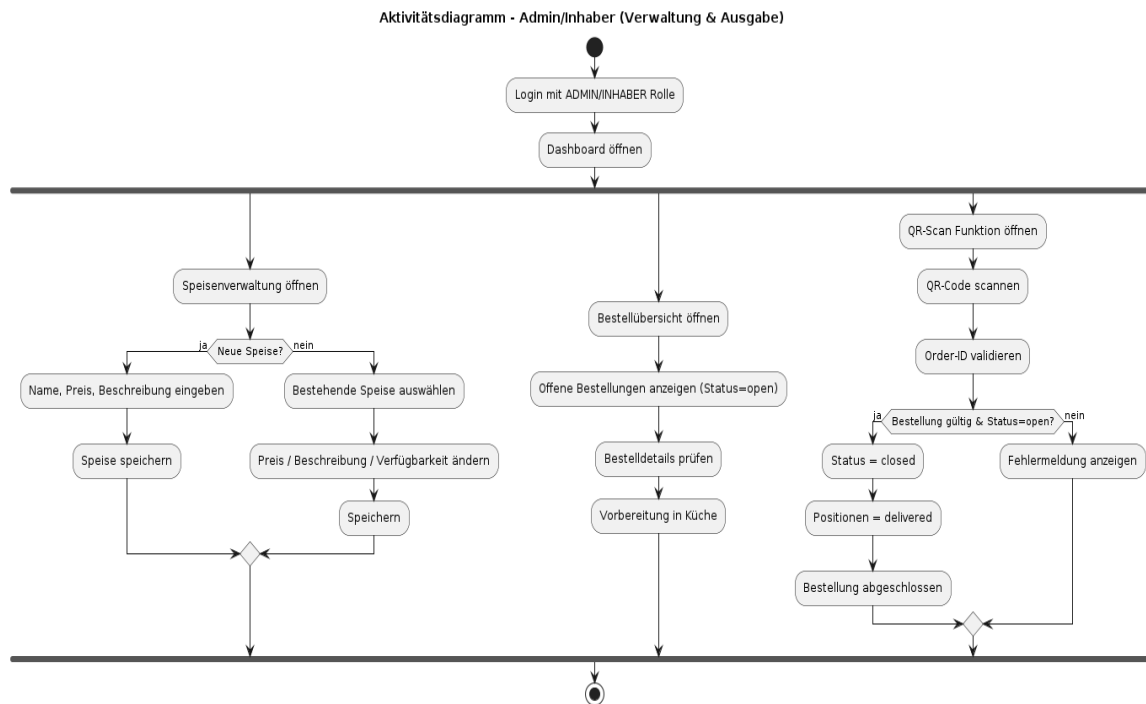


Abbildung 34: Aktivitätsdiagramm ADMIN - Anleitung